tkutils

Tk-Widgets auf tclutils, 0.49.0

2026-10-27

Inhaltsverzeichnis

1 tkutils verstehen 8
1.1 Einführung 8
1.2 Rolle 8
1.3 Der Umbrella 8
1.3.1 tical einbinden 9
1.3.2 Eine Tablelist-Variante 9
1.4 Abgrenzung 10
1.5 Aufbau 10
1.6 Merkliste 10
2 Installation 10
2.1 Pfade 10
2.1.1 Über einen älteren Baum kopieren 11
2.2 Die Umgebungsvariablen 12
2.3 Tests 12
2.3.1 Testlauf 12
2.4 Zusammenfassung 13
3 Dialoge 13
3.1 tkudialog 0.3 13
3.2 Alle Einstiege 13
3.3 Knöpfe mit Ids 14
3.4 Formular 15
3.5 Nicht modal 15
3.6 Parent und Platzierung 15
3.7 Optionen werden geprüft 16
3.8 Versionen 16
3.9 ctrlutils 16
3.10 Zusammenfassung 16
4 Datei-Widgets 16
4.1 Liste und Baum 16
4.2 tkufiletree 17
4.2.1 Versteckte Dateien unter Windows 18
4.3 tkupath 0.3 18
4.4 tkupreview 0.2 18
4.5 tkufiledialog 0.2 19
4.6 Zusammenfassung 20
5 Listen und tablelist 20
5.1 tkutablelist 0.5 20
5.2 Warum -exportselection 0 21
5.3 Zeilen-IDs und Sortieren 21
5.4 CSV 22
5.5 Versionen 22
5.6 Tablelist-Helfer 23
5.7 Zusammenfassung 23
6 Tablelist-Helfer 23
6.1 Sortieren: tkutlsort 24

6.2 Anzeige: tkutlfmt 25
6.3 Zwischenablage: tkutlclip 25
6.4 Fußzeile: tkutlfooter 26
6.5 Suchen: tkutlfind 26
6.6 Layout sichern: tkutlstate 27
6.7 Baum: tkutltree 28
6.8 Filtern: tkutlfilter 29
6.9 Gruppieren: tkutlgroup 30
6.10 Ziehen: tkutldrag 31
6.11 Merkliste 31
7 Geometrie und Units 32
7.1 tkugeometry 0.1 32
7.2 Außerhalb des Bildschirms 33
7.3 tkuunits 0.1 33
7.4 Zusammenfassung 34
8 Aktionen und Layout 34
8.1 tkuaction 0.2 34
8.2 Enable pro Fenster 35
8.3 Gerüst 36
8.4 tkulauncher 0.4 36
8.4.1 Start unter Windows 36
8.4.2 Interpreter für tcl-Einträge 37
8.5 Zusammenfassung 37
9 Gerüst 37
9.1 Leiste: tkutoolbar 38
9.2 Status: tkustatus 38
9.3 Reiter: tkutab 39
9.4 Baum: tkutree 39
9.5 Hinweis: tkuballoon 39
9.6 Kontextmenü: tkucontextmenu 39
9.7 Tasten: tkubind 40
9.8 Fokus: tkukeynav 40
9.9 Suche und Filter 41
9.10 Scrollen 41
9.11 Rahmen aufziehen: tkumarquee 41
9.12 Merkliste 42
10 Eingabe 42
10.1 Formular: tkuform 42
10.2 Eine Zeile: tkulabeled 43
10.3 Prüfung: tkuvalidate 43
10.4 Datum: tkudateentry 43
10.5 Uhrzeit: tkutimeentry 44
10.6 Zahl: tkunumentry 44
10.7 Schlagwörter: tkutags 44
10.8 Merkliste 45
11 Kalender und Notizen 45
11.1 Monat zum Anklicken: tkucalendar 45

11.2 Textkalender: tkucal 46
11.3 tical: tkutical und tkumonthcanvas 46
11.4 Termine: tkuical 47
11.5 Aufgaben: tkutodo 47
11.6 Notizen: tkunotes 48
11.7 Kontakte und LDIF 48
11.8 Rechner: tkucalc 49
11.9 Merkliste 49
12 Daten ansehen 49
12.1 CSV: tkucsv 49
12.2 JSON: tkujson 50
12.3 XML: tkuxml 50
12.4 INI: tkuini 50
12.5 Markdown: tkumd und tkumdview 51
12.6 SQLite: tkusqlite 51
12.7 Diff, Hex, Zeichenketten, Base64 51
12.8 Unscharfe Liste: tkufuzzy 52
12.9 ZIP: tkuzip 52
12.10 PDF-Struktur: tkupdfinspect 52
12.11 DAV 53
12.12 Editor: tkueditor 53
12.13 System: tkusysinfo 54
12.14 Merkliste 55
13 Designer und load 55
13.1 tkurender / tkuload 55
13.2 Feste Größe des Platzhalters 56
13.3 Einsatz 56
13.4 Seitenlayout: tkulayoutcanvas 57
13.5 Zusammenfassung 57
14 Bilder 58
14.1 Ansehen: tkuimage 58
14.2 Symbole: tkuicon 58
14.3 Zeichenfläche als PNG: tkucanvaspng 59
14.4 Windows-Symbol: tkuwinico 59
14.5 Ähnlichkeit: tkudhash 59
14.6 Merkliste 60
15 tkdevtools 60
15.1 Die App 60
15.2 Die Werkzeuge 60
15.3 Werkzeuge seit 0.46.0 61
15.4 Zusammenfassung 61
16 Grenzen 62
16.1 Zusammenfassung 62
17 Fallen 62
17.1 Behoben in 0.47.0 63
17.2 Zusammenfassung 64
18 Widgets (Orientierung) 64

18.1 Auszug 64
18.2 Nur einzeln 65
18.3 Zusammenfassung 65
19 Glossar 65
19.1 Zusammenfassung 66
20 Fehlermeldungen 66
20.1 Zusammenfassung 67
21 API-Katalog: Tkutils-Module 68
21.1 tkuaction 68
21.2 tkuballoon 68
21.3 tkubase64 69
21.4 tkubind 69
21.5 tkucal 69
21.6 tkucalc 69
21.7 tkucalendar 70
21.8 tkucanvaspng 70
21.9 tkucontextmenu 70
21.10 tkucsv 70
21.11 tkudateentry 71
21.12 tkudavaccount 71
21.13 tkudavbrowser 71
21.14 tkudhash 71
21.15 tkudialog 72
21.16 tkudiff 72
21.17 tkueditor 72
21.18 tkufiledialog 73
21.19 tkufilelist 73
21.20 tkufiletree 73
21.21 tkufilterbar 73
21.22 tkuform 74
21.23 tkufuzzy 74
21.24 tkugeometry 74
21.25 tkuhexedit 74
21.26 tkuical 74
21.27 tkuicon 75
21.28 tkuimage 75
21.29 tkuini 76
21.30 tkujson 76
21.31 tkukeynav 76
21.32 tkulabeled 76
21.33 tkulauncher 77
21.34 tkulayoutcanvas 77
21.35 tkuldif 77
21.36 tkuload 78
21.37 tkumarquee 78
21.38 tkumd 78
21.39 tkumdview 78

21.40 tkumonthcanvas 79
21.41 tkunotes 79
21.42 tkunumentry 79
21.43 tkuopts 80
21.44 tkupath 80
21.45 tkupdfinspect 80
21.46 tkupreview 80
21.47 tkurender 80
21.48 tkuscrolledframe 81
21.49 tkusearchbar 81
21.50 tkusqlite 82
21.51 tkustatus 82
21.52 tkustrings 82
21.53 tkusysinfo 82
21.54 tkutab 83
21.55 tkutablelist 83
21.56 tkutags 84
21.57 tkutical 84
21.58 tkutimeentry 84
21.59 tkutlclip 85
21.60 tkutldrag 85
21.61 tkutlfilter 85
21.62 tkutlfind 85
21.63 tkutlfmt 85
21.64 tkutlfooter 85
21.65 tkutlgroup 86
21.66 tkutlsort 86
21.67 tkutlstate 86
21.68 tkutltree 86
21.69 tkutodo 86
21.70 tkutoolbar 87
21.71 tkutree 87
21.72 tkuunits 87
21.73 tkuvalidate 88
21.74 tkuvcard 88
21.75 tkuwheel 88
21.76 tkuwinico 88
21.77 tkuxml 88
21.78 tkuzip 89
22 Praxis: Tk-Integration, Maße und Fensterzustand 89
22.1 Tk zuerst laden 89
22.2 Einheiten vergleichen 89
22.3 Fenstergeometrie 89
22.4 Fehlersuche bei optionalen Widgets 90
23 Testumgebung 90
23.1 Einführung 90
23.2 Umgebung 90

23.3 Ältere Fassungen 91
23.3.1 Testsuite 0.48.0 91
23.3.2 Testsuite 0.46.0 91
23.3.3 Zwei Tablelist-Varianten bis 0.46.0 92
23.3.4 tkufiledialog 0.1 (tkutils 0.46.0) 92
23.4 Noch nicht geprüft 92
24 Lizenz 93

1 tkutils verstehen

1.1 Einführung

tkutils liefert Tk-Widgets und Dialoge: Dialoge, Dateiliste und -baum, Filedialog, tablelist, Geometrie, Units, Actions, Launcher, Designer-Slots und die App tkdevtools. Die Engines bleiben in tclutils, so brauchen Server und CI kein Tk. Der Code nennt Tcl/Tk **8.6** und **9.x**.

Dieses Handbuch beschreibt tkutils **0.49.0** für Desktop-Anwendungen auf Provider und ttk. Die Beispiele und die Testsuite laufen mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux (Xvfb). Unter Windows und macOS ist das Buch nicht geprüft.

Empfohlene Paarung: tclutils **0.69.0** + tkutils **0.49.0** + ctrlutils **0.4**. Den CSV-Export von `tkutablelist` 0.5 (`toCsv`, `saveCsv`) braucht `tclutils::tucsv` 0.2 aus tclutils 0.69.0. Sonst braucht tkutils 0.49.0 mindestens tclutils **0.64.0**: Erst diese Version bringt `tusettings` (für `tkugeometry`), `tutestrun` (Testrunner) und die Funktionen, die tkdevtools braucht.

1.2 Rolle

Stück	Rolle
Präfix	Paket tkutils::tku<name>, Namensraum ::tkutils::tku<name>
Schicht	View
Regel	Engine in tclutils, GUI hier
Laden	package require tkutils oder Einzelmodul

Jedes Modul ist ein Paket `tkutils::tku<name>` in `lib/tm/tkutils/tku<name>-<version>.tm`. Seine Befehle liegen im Namensraum `::tkutils::tku<name>`. Die meisten Module markieren sie mit `namespace export`, aber kein Modul importiert sie in den globalen Namensraum. Rufen Sie sie deshalb immer voll qualifiziert auf: `::tkutils::tkudialog::confirm`, nicht `tkudialog::confirm` — die Kurzform bricht mit `invalid command name "tkudialog::confirm"` ab.

Zwei Ausnahmen haben einen eigenen Namensraum: tkuload lebt in `::tkuload`, tkurender in `::tkurender` (Kapitel Designer und load). Die Pakete heißen trotzdem `tkutils::tkuload` und `tkutils::tkurender`.

1.3 Der Umbrella

`package require tkutils` lädt **53** Module direkt, darunter Dialoge, Dateiliste und -baum, Filedialog, Pfadzeile, Vorschau, Tabs, Toolbar, Statusleiste, Actions, Launcher, Geometrie und Units. Zwei weitere kommen indirekt mit:

- tkutablelist — über `tkufilelist` (und damit `tkufiledialog`). Der Umbrella braucht deshalb **Tablelist** aus tklib auf dem `auto_path`; ohne Tablelist scheitert `package require tkutils` mit `can't find package Tablelist`.
- `tkuopts` — die gemeinsame Optionsprüfung (Kapitel Fehlermeldungen).

Nach `package require tkutils` sind **55** Pakete `tkutils::*` geladen — 53 direkt, dazu `tkutablelist` 0.5 und `tkuopts` 0.1. Der Umbrella fordert `tkupath 0.2` und `tkupreview`

`0.1` an; geladen werden die vorhandenen Versionen 0.3 und 0.2.

Nicht im Umbrella — einzeln requiren:

Modul	**Braucht**
tkuxml	tDOM
tkusqlite	sqlite3
tkuicon	für SVG: tksvg (Tk 8.6) oder Tk 9; PNG/GIF ohne
tkuscrolledframe	scrollutil (tklib)
tkutical, tkumonthcanvas	tical 1.0
tkucanvaspng	tclutils::tupngdraw
tkutltools und tkutlclip, tkutldrag, tkutlfilter, tkutlfind, tkutlfmt, tkutlfooter, tkutlgroup, tkutlsort, tkutlstate, tkutltree	Tablelist (tablelist_tile)
tkuload, tkurender	— (Designer-Kern)
tkumdview, tkupdfinspect, tkulayoutcanvas	tclutils-Engines
tkusysinfo	tclutils::tusysinfo (tclutils 0.69.0)

Das sind **24** Module; zusammen mit den 55 aus dem Umbrella ergibt das die **79** Module des Baums.

```
package require tkutils
puts [package present tkutils::tkutablelist]   ;# -> 0.5, über
    tkufilelist
package require tkutils::tkuload                ;# extra
```

Alle 24 Einzelmodule laden, wenn tDOM, sqlite3, tklib (Tablelist, scrollutil) und tical auf dem Pfad liegen; `tkuicon` lädt auch ohne tksvg.

1.3.1 tical einbinden

tical 1.1 ist die Kalender- und iCalendar-Engine, die das Buch *Utils-Stack* beschreibt. Sie ist kein Teil von tkutils und wird von `setup.tcl` nicht gefunden. Der tical-Baum bringt eine `pkgIndex.tcl` mit; tragen Sie ihn in den `auto_path` ein:

```
lappend auto_path /pfad/zu/tical
package require tkutils::tkutical        ;# -> 0.4
package require tkutils::tkumonthcanvas  ;# -> 0.5
```

`tkutical` braucht `tical::view::month`, `tical::view::week` und `tical::render::canvas` (je 1.0). `tkumonthcanvas` braucht `tical::config`, `tical::locale`, `tical::holidays`, `tical::holidays::de` und `tical::view::month` (je 1.0).

1.3.2 Eine Tablelist-Variante

tklib erlaubt nicht beide Tablelist-Varianten (`tablelist` und `tablelist_tile`) in einem

Programm. tkutils verwendet deshalb überall **tablelist_tile**: `tkutablelist` lädt es zuerst und nur, wenn es fehlt, `Tablelist`. Die `tkutl*`-Helfer laden `tablelist_tile`, wenn noch keine Tablelist geladen ist; hat die Anwendung schon eine der beiden Varianten, nehmen sie diese. Apps (tkdevtools, Wissensbasis), Beispiele, Tests und Doku laden `tablelist_tile`. `tkutils` und danach `tkutltools` lädt ohne Fehler, die umgekehrte Reihenfolge ebenso (Tests `tkutltools-2.1`, `-2.2`). Zum Verhalten bis 0.46.0: Anhang G.

1.4 Abgrenzung

- Kein allgemeines Tk-Lehrbuch — dafür *TclTk-Benutzeroberflächen*
- Keine Controller wie `cufileops` — dafür *Utils-Stack*
- Keine Engines — dafür *tclutils*
- Keine Abschrift aller Widget-Docs — API-Wahrheit: `docs/<modul>.md`

1.5 Aufbau

Kapitel 2 behandelt Installation und Tests. Danach folgen Dialoge, Datei-Widgets, Listen und die Tablelist-Helfer, Geometrie und Units, Aktionen, das Gerüst, die Eingabe, Kalender und Notizen, die Datenansicht, Designer und load, Bilder und tkdevtools. Kapitel 16 nennt die Grenzen. Die Anhänge enthalten Fallen (A), die Widget-Liste (B), das Glossar (C), Fehlermeldungen (D), den API-Katalog (E), Praxisbeispiele (F) und die Testumgebung (G).

1.6 Merkliste

- Widgets auf tclutils-Engines, Version 0.49.0.
- Paarung tclutils 0.69.0 + tkutils 0.49.0 + ctrlutils 0.5; mindestens tclutils 0.64.0, für den CSV-Export von `tkutablelist` 0.69.0.
- Befehle immer mit `::tkutils::tku<name>::…` aufrufen.
- Umbrella braucht Tablelist; 24 Module kommen nur einzeln.
- Überall `tablelist_tile`; beide Tablelist-Varianten in einem Programm lehnt tklib ab.

Getestet mit Tcl/Tk 8.6.14 (Tablelist 6.20, scrollutil 1.17) und 9.0.4 (Tablelist 7.11, scrollutil 2.10) unter Linux und Xvfb, tclutils 0.69.0, tical 1.1.

Nicht getestet: Windows, macOS.

Nächstes Kapitel: Installation

2 Installation

2.1 Pfade

tkutils braucht tclutils auf dem Modulpfad, dazu Tablelist aus tklib (für den Umbrella und alle Tabellen-Widgets). Der übliche Weg ist der Bootstrap `tools/setup.tcl`:

```
source /pfad/zu/tkutils/tools/setup.tcl
package require tkutils
package require tkutils::tkudialog 0.3
```

`setup.tcl` ist in tclutils und tkutils identisch (`diff` ohne Unterschied); welche Kopie Sie

sourcen, ist egal — beide Bäume werden registriert. Gesucht wird in dieser Reihenfolge, nur vorhandene Pfade werden eingetragen:

1. `TCLUTILS_TM` / `TKUTILS_TM` (Override, gewinnt)
2. `lib/tm` des eigenen Baums
3. neben der Tcl-Installation (`<tcl-lib>/<paket>/lib/tm`)
4. `/usr/local/share/tcltk/<paket>/lib/tm`
5. `$XDG_DATA_HOME/tcltk/<paket>/lib/tm`, sonst `$HOME/.local/share/tcltk/<paket>/lib/tm`
6. Geschwisterordner `<paket>/lib/tm`, dann `<paket>-*/lib/tm` (höchste Version)

Ohne `setup.tcl` tragen Sie die Bäume selbst ein:

```
tcl::tm::path add /pfad/zu/tclutils/lib/tm /pfad/zu/tkutils/lib/tm
package require tkutils::tkudialog 0.3
```

Fehlt nur der tclutils-Baum, scheitern Module mit Engine-Bedarf, etwa `tkugeometry` mit `can't find package tclutils::common 0.1`.

2.1.1 Über einen älteren Baum kopieren

Der Umbrella von 0.49.0 ist `lib/tm/tkutils-0.49.0.tm`. Neue Modulversionen in 0.49.0: `tkutablelist` 0.5, `tkutlstate` 0.4, `tkutlclip`, `tkutlfind`, `tkutlsort`, `tkutltree`, `tkutltools` 0.3, `tkuical`, `tkuvcard`, `tkuldif`, `tkuini` 0.3, `tkunotes`, `tkueditor` 0.4, `tkubase64` 0.2, `tkutical` 0.4, `tkutlfooter` 0.4, neu `tkutlfilter`, `tkutlgroup`, `tkutldrag`, `tkusysinfo` 0.1; in 0.48.0: `tkulauncher` und `tkuload` 0.3; in 0.47.0: `tkufiledialog` 0.2; `tkutlsort`, `tkutlclip`, `tkutlfind`, `tkutltree`, `tkutltools` 0.2; `tkutlfooter`, `tkutlfmt`, `tkutlstate` 0.3. Wer 0.49.0 über einen älteren Baum kopiert, löscht vorher die alten Dateien, sonst liegen zwei Versionen nebeneinander.

Über einen Baum von 0.48.0:

- `lib/tm/tkutils-0.48.0.tm`
- in `lib/tm/tkutils/`: `tkutablelist-0.4.tm`, `tkutlclip-0.2.tm`, `tkutlfind-0.2.tm`, `tkutlsort-0.2.tm`, `tkutlstate-0.3.tm`, `tkutltools-0.2.tm`, `tkutltree-0.2.tm`
- ebenso `tkuical-0.2.tm`, `tkuvcard-0.2.tm`, `tkuldif-0.2.tm`, `tkuini-0.2.tm`, `tkutical-0.3.tm`, `tkunotes-0.2.tm`, `tkueditor-0.2.tm`, `tkubase64-0.1.tm`, `tkutlfooter-0.3.tm`, `tkulauncher-0.3.tm`, `tkuhexedit-0.1.tm`

Über einen Baum von 0.47.0 statt `tkutils-0.48.0.tm` die Datei `lib/tm/tkutils-0.47.0.tm`, dazu dieselben Dateien und `tkulauncher-0.2.tm`, `tkuload-0.2.tm`.

Über einen Baum von 0.46.0 statt `tkutils-0.47.0.tm` die Datei `lib/tm/tkutils-0.46.0.tm`, dazu in `lib/tm/tkutils/`:

- `tkufiledialog-0.1.tm`
- `tkutlclip-0.1.tm`, `tkutlfind-0.1.tm`, `tkutlsort-0.1.tm`, `tkutltools-0.1.tm`, `tkutltree-0.1.tm`, `tkutablelist-0.4.tm`
- `tkutlfmt-0.2.tm`, `tkutlfooter-0.2.tm`, `tkutlstate-0.2.tm`
- `tkulauncher-0.2.tm`, `tkuload-0.2.tm`

Die Listen stammen aus dem Vergleich der Bäume von 0.46.0 bis 0.49.0. Bleibt eine ältere Datei liegen, lädt `package require` trotzdem die höhere Version; `tools/check-modules.tcl`

meldet die Doppelung.

2.2 Die Umgebungsvariablen

TCLUTILS_TM und TKUTILS_TM liest Tcl **nicht** selbst. Sie wirken nur dort, wo Code sie ausdrücklich auswertet:

- `tools/setup.tcl` und damit die mitgelieferten Apps (über `apps/_lib/paths.tcl`),
- `tkuload` beim ersten `build` (trägt beide Pfade per `tcl::tm::path add` ein),
- `tests/all.tcl`: TCLUTILS_TM hilft beim Finden des Runners; der Runner selbst stellt die Bäume aus TKUTILS_TM und TCLUTILS_TM nach vorn,
- die einzelnen Testdateien (TKUTILS_TM).

Ein eigenes Skript, das nur `package require tkutils` aufruft, findet trotz gesetzter Variablen nichts und meldet `can't find package tkutils`.

```
export TCLUTILS_TM=/pfad/zu/tclutils/lib/tm
export TKUTILS_TM=/pfad/zu/tkutils/lib/tm
```

Setzen Sie sie, wenn ein bestimmter Arbeitsbaum Vorrang vor einer Installation haben soll — etwa in CI.

2.3 Tests

```
xvfb-run -a tclsh tests/all.tcl
xvfb-run -a tclsh tests/all.tcl -match 'tkuunits-*'
```

Der Runner ist tutestrun aus tclutils (`::tclutils::tutestrun`, ab tclutils 0.64.0). `all.tcl` sucht ihn im eigenen Baum, in TCLUTILS_TM und im Geschwister-Checkout `tclutils*`, nie in einer Installation. Fehlt er, endet `all.tcl` mit Exit-Code 2 und der Meldung `all.tcl: tclutils::tutestrun not found (set TCLUTILS_TM)`.

- Jede `tests/*.test` läuft in einem eigenen Prozess.
- Das Banner `Modules resolve to:` zeigt, aus welchen Dateien tkutils und tclutils geladen wurden, und nennt gesetzte Pfadvariablen.
- Grenze pro Datei: TKUTILS_TESTGRENZE in Sekunden, Standard 120.
- Weitere Argumente gehen an tcltest (`-match`, `-skip`, …).
- Exit-Code 1, wenn eine Datei Fehler meldet, keine Zusammenfassung schreibt, abstürzt oder hängt. Die Dateien stehen am Ende unter `Files with problems:`.

Aufruf mit `tclsh`, nicht mit `wish`. GUI-Tests tragen die Bedingung haveTk; ohne Display werden sie übersprungen. Auch `stack.test`, `tkumdview.test`, `tkupdfinspect.test` und `tkuwinico.test` melden ohne Display `<datei>.test: no display, all tests skipped` und eine Zusammenfassung mit 0 Fehlern.

Tests für Module mit fehlendem Fremdpaket (tDOM, sqlite3, Tablelist, tical) werden übersprungen. tical suchen die Tests über TICAL_DIR oder einen Geschwisterordner. `tkutical.test` findet einen Ordner `tical` neben `tkutils`. `tkumonthcanvas.test` sucht wie `tkutical.test` auch den flachen Checkout; ohne TICAL_DIR bestehen dort 13 von 13 Tests.

2.3.1 Testlauf

tkutils 0.49.0 unter Linux (Xvfb) mit tclutils 0.69.0, tical 1.1 und gesetztem `TICAL_DIR`; Tablelist 6.20 unter Tcl/Tk 8.6.14, 7.11 unter 9.0.4:

Tcl/Tk	Dateien	Tests	bestanden	übersprungen	Fehler
8.6.14	83	947	935	12	0
9.0.4	83	947	944	3	0

Unter 8.6.14 überspringt die Suite die SVG-Tests von `tkuicon` und `tkuwinico` (kein tksvg), dazu je einen Test von `tkuscrolledframe` und `tkuwheel`. Unter macOS (Darwin 27, arm64, Tcl/Tk 9.0.4 mit Aqua) lief die Suite vor den Schreibfehler-Tests ohne Fehler: 83 Dateien, 938 Tests, 896 bestanden, 42 übersprungen. Unter Windows im selben Stand ebenfalls ohne Fehler: Tcl/Tk 8.6.17 927 bestanden, 11 übersprungen; Tcl/Tk 9.0.4 935 bestanden, 3 übersprungen. Umgebung und Ergebnisse von 0.46.0: Anhang G.

2.4 Zusammenfassung

- `setup.tcl` sourcen oder `tcl::tm::path add` für beide Bäume.
- `TCLUTILS_TM`/`TKUTILS_TM` wirken nur über setup.tcl, Apps, tkuload und Testlauf.
- Tests: `xvfb-run -a tclsh tests/all.tcl`; ohne Display werden GUI-Tests übersprungen.
- Beim Kopieren über einen älteren Baum die alten `.tm`-Dateien löschen.

Nächstes Kapitel: Dialoge

3 Dialoge

3.1 tkudialog 0.3

tkudialog ersetzt `tk_messageBox`-Muster mit ttk. Der Meldungstext ist markierbar und kopierbar (Strg-C oder Knopf *Copy*). Dazu kommen Default-Button, Return/Escape, Fokus/Grab und Button-Ids. Reines Tk, keine tclutils-Engine.

```
package require tkutils::tkudialog 0.3
set ok [::tkutils::tkudialog::confirm "Datei löschen?" -parent .]
if {$ok} { puts "wird gelöscht" }
```

`confirm` liefert **1** für *Ja*, sonst **0** — auch bei Escape oder Fenster schließen.

3.2 Alle Einstiege

Aufruf	Art	Rückgabe
showInfo msg ?opt?	modal	ok; "" bei Escape/Schließen
showWarning msg ?opt?	modal	wie showInfo
showError msg ?opt?	modal	wie showInfo; -detail startet eingeklappt
confirm msg ?opt?	modal	1 (Ja) oder 0

Aufruf	Art	Rückgabe
input ?opt?	modal	eingegebener Text; "" bei Abbruch oder leerer Eingabe
form spec ?-title? ?-parent?	modal	Werte-Dict; "" bei Abbruch
show win ?opt?	modal	Id des gewählten Knopfs
build win ?opt?	nicht modal	win
choose win id	Knopf programmatisch wählen	leer
result win	Status	gewählte Id, "" solange offen
getText win	Status	Meldungstext
getDetail win	Status	Detailtext
copyText win	Zwischenablage	kopierter Text (Meldung + Detail)

Alle Befehle liegen in `::tkutils::tkudialog`. Ein `message` oder `warning` gibt es nicht.

Optionen von `build`/`show` (und damit der fertigen Varianten): `-title`, `-message`, `-icon info|warning|error|question`, `-detail`, `-buttons`, `-default`, `-cancel`, `-entry`, `-initial`, `-parent`, `-detailcollapsed`, `-width` (Standard 48), `-maxheight` (Standard 20). `input` kennt nur `-title`, `-message`, `-initial`, `-parent`; `form` nur `-title` und `-parent`. `input` liefert auch bei *OK* mit leerem Feld `""` — Abbruch und leere Eingabe sind am Ergebnis nicht zu unterscheiden.

```
package require tkutils::tkudialog 0.3
::tkutils::tkudialog::showInfo "Export fertig." -parent .
::tkutils::tkudialog::showError "Speichern fehlgeschlagen." \
    -detail "permission denied" -parent .

set name [::tkutils::tkudialog::input -title "Neuer Ordner" \
    -message "Name:" -initial "Neu" -parent .]
if {$name ne ""} { puts "anlegen: $name" }
```

3.3 Knöpfe mit Ids

`-buttons` nimmt Einträge `Label` (Id gleich Label, wie in 0.2) oder `{id Label}`. `show` gibt die **Id** zurück, unabhängig von der Beschriftung — so bleibt der Code gleich, wenn msgcat die Labels übersetzt.

```
package require tkutils::tkudialog 0.3
set id [::tkutils::tkudialog::show .frage -title "Ungespeichert" \
    -icon question -message "Änderungen speichern?" \
    -buttons {{save Speichern} {discard Verwerfen} {cancel Abbrechen}} \
    -default save -cancel cancel -parent .]
switch -- $id {
```

```
    save    { puts speichern }
    discard { puts verwerfen }
    default { puts abbruch }
}
```

- Return / KP_Enter wählt `-default` (ohne Angabe den ersten Knopf).
- Escape und Fenster schließen wählen `-cancel`. Ohne Angabe gilt der erste Knopf mit der Id `No`, `Cancel`, `no` oder `cancel` oder mit dem Label `No` oder `Cancel`; gibt es keinen, ist das Ergebnis `""`.
- `confirm`, `input` und `form` benutzen feste Ids (`yes`/`no`, `ok`/`cancel`).
- Titel und Standardlabels (`OK`, `Yes`, `Copy`, …) laufen über `msgcat`, wenn es geladen ist.
- Modale Dialoge stellen Fokus und Grab des Aufrufers wieder her.

3.4 Formular

`form` bettet ein `tkuform` mit OK/Abbrechen in einen modalen Dialog.

```
package require tkutils::tkudialog 0.3
set v [::tkutils::tkudialog::form {
    {name titel  label "Titel"    type entry}
    {name prio   label "Prio"     type combo values {niedrig normal
    hoch} default normal}
    {name fertig label "Erledigt" type check}
} -title "Neue Notiz" -parent .]
if {$v ne ""} { puts "prio=[dict get $v prio]" }
```

3.5 Nicht modal

`build` baut den Dialog und kehrt sofort zurück. Den Rest verdrahten Sie selbst.

```
package require tkutils::tkudialog 0.3
::tkutils::tkudialog::build .info -title "Protokoll" \
    -message "Lauf beendet." -detail "3 Dateien kopiert" \
    -buttons {{ok OK}} -default ok
puts [::tkutils::tkudialog::getDetail .info]   ;# -> 3 Dateien kopiert
::tkutils::tkudialog::copyText .info
::tkutils::tkudialog::choose .info ok
puts [::tkutils::tkudialog::result .info]      ;# -> ok
destroy .info
```

3.6 Parent und Platzierung

`-parent w` macht den Dialog transient zum Toplevel von `w` und zentriert ihn darüber. Ohne `-parent` gilt das Toplevel mit dem Tastaturfokus, sonst`.`. Geben Sie `-parent` an, wann immer das Fenster bekannt ist — bei mehreren Monitoren ist es die einzige verlässliche Antwort. Ein `-parent`, das es nicht gibt, ist ein Fehler `{TKUTILS TKUDIALOG PARENT <w>}` mit der Meldung `bad window path name ".nix" given as -parent` (für `-parent .nix`). Ein Parent, der noch nicht sichtbar ist, macht den Dialog nicht transient; er wird dann auf dem

Bildschirm zentriert.

3.7 Optionen werden geprüft

Eine unbekannte Option ist ein Fehler mit der Liste der bekannten:

```
package require tkutils::tkudialog 0.3
catch {::tkutils::tkudialog::showInfo "x" -parnet .} msg opts
puts [dict get $opts -errorcode]   ;# -> TKUTILS TKUDIALOG OPTION
    -parnet
```

Die Meldung lautet `unknown option "-parnet"`, in der zweiten Zeile `Known:` mit den gültigen Optionen in der Reihenfolge des Codes (`-title -message -icon -detail -buttons -entry -initial -parent -default -cancel -detailcollapsed -width -maxheight`). Fehlt ein Wert, lautet sie `missing value for option "…"` mit dem Code `{TKUTILS TKUDIALOG OPTION}`.

3.8 Versionen

Die Datei **0.2** ist entfernt; `package require tkutils::tkudialog 0.2` lädt 0.3 (kompatibel für einfache Labels). Doc: `docs/tkudialog.md`.

3.9 ctrlutils

`cufileops` verlangt seit 0.3 `tkudialog` 0.3 für Bestätigungen; der aktuelle Stand in ctrlutils 0.5 ist `cufileops` 0.6. Siehe *Utils-Stack*.

3.10 Zusammenfassung

- Einstiege: `showInfo`, `showWarning`, `showError`, `confirm`, `input`, `form`, `build`/`show`, dazu `choose`, `result`, `getText`, `getDetail`, `copyText`.
- `confirm` → 1/0, `show` → Id, `input`/`form` → `""` bei Abbruch.
- Immer `-parent` angeben.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux und Xvfb: alle Beispiele; die modalen Dialoge wurden dabei per `choose` geschlossen.

Nicht getestet: Darstellung, Fokus und Platzierung auf einem echten Bildschirm.

Nächstes Kapitel: Datei-Widgets

4 Datei-Widgets

4.1 Liste und Baum

Widget	Rolle
tkufilelist 0.2	Detailliste eines Verzeichnisses über Provider
tkufiletree 0.2	Lazy-Baum; lokal oder mit -provider

Widget	Rolle
tkupath 0.3	Pfadzeile (Breadcrumb)
tkupreview 0.2	Vorschau (Text, Markdown, Meldung, …)

Die Widgets sprechen einen Provider aus `tclutils::tuprovider`, nicht das feste Dateisystem. Ohne `-provider` legt `tkufilelist` selbst einen lokalen an.

```
package require tclutils::tuprovider
package require tkutils::tkufilelist

set prov [::tclutils::tuprovider open local]
::tkutils::tkufilelist::widget .l -provider $prov -dir [pwd] \
    -onactivate {apply {e {
        if {[dict get $e type] eq "dir"} {
            ::tkutils::tkufilelist::setDir .l [dict get $e path]
        }
    }}}
pack .l -fill both -expand 1
::tkutils::tkufilelist::setFilter .l *.txt
```

Callbacks bekommen den Eintrag des Providers als Dict mit `name`, `path`, `type` (`file`/`dir`), `size` und `mtime`; `size` und `mtime` können leer sein. Weitere Befehle: `dir`, `refresh`, `selectedEntry`, `selectedEntries`. `tkufilelist` baut auf `tkutablelist` auf und braucht deshalb Tablelist.

Optionen von `widget`: `-provider`, `-dir` (Standard `/`, nicht das aktuelle Verzeichnis), `-files` (Standard 1), `-showhidden` (Standard 0), `-filter`, `-onactivate` (Doppelklick oder Return), `-onselect`, `-height` (Standard 16). `setFilter` nimmt **ein** Glob-Muster, ohne Beachtung der Groß-/Kleinschreibung; Ordner bleiben immer sichtbar, damit die Navigation möglich bleibt. Die Sortierung nach Kopfklick hält die Zuordnung Zeile → Eintrag, weil der Eintrag als Zeilenattribut der Tablelist gespeichert ist.

Controller (`cufileops`, Filter, Monitor) verdrahten Verhalten — Buch *Utils-Stack*. ZIP/Remote: Caps des Providers steuern Schreibaktionen.

4.2 tkufiletree

tkufiletree liest Unterverzeichnisse erst beim Aufklappen. Optionen: `-root`, `-filter` (Glob-Liste, ohne Beachtung der Groß-/Kleinschreibung), `-files`, `-showhidden`, `-provider`, `-onactivate`, `-onselect`, `-height`, `-isolatekeys`.

```
package require tkutils::tkufiletree
set t [::tkutils::tkufiletree::widget .t -root [pwd] -showhidden 1 \
    -filter {*.txt *.md} \
    -onactivate {apply {p {puts "öffnen: $p"}}}]
pack $t -fill both -expand 1
::tkutils::tkufiletree::reveal $t [file join [pwd] README.md]
```

Item-Ids sind die normalisierten vollen Pfade. `reveal` klappt bis zum Ziel auf und liefert 1, wenn genau dieses Ziel gewählt wurde, sonst 0. Weitere Befehle: `setRoot`, `root`, `up` (auch

BackSpace), `refresh`, `refreshDir`, `selectedPath`, `treeview`, `volumesRoot`.

4.2.1 Versteckte Dateien unter Windows

Mit `-showhidden 1` liest der Baum `*` und `.*`. Unter Windows trifft `*` die Punkt-Dateien bereits, sie kamen also doppelt, und das Treeview brach mit `ITEM_EXISTS` ab. Seit 0.46.0 werden die Treffer vor dem Einfügen mit `lsort -unique` entdoppelt (Test `tkufiletree-1.7`).

Ohne `-root` gilt das aktuelle Verzeichnis. `volumesRoot` liefert einen Sonderwert für `-root`: einen virtuellen Knoten *This PC* mit allen Laufwerken. `up` steigt an einer Laufwerkswurzel dorthin, wenn es mehr als ein Laufwerk gibt.

4.3 tkupath 0.3

tkupath zeigt einen Pfad als Reihe von Knöpfen. Ein Klick setzt die Zeile auf diesen Teilpfad und ruft `-onnavigate` mit ihm auf. Die Zeile berührt kein Dateisystem; sie trennt nur an `/`.

```
package require tkutils::tkupath
::tkutils::tkupath::widget .bar -onnavigate {apply {p {puts "gehe:
    $p"}}}
pack .bar -fill x
::tkutils::tkupath::setPath .bar /srv/daten/docs
puts [::tkutils::tkupath::getPath .bar]   ;# -> /srv/daten/docs
```

Die Zeile hat sich schon umgestellt, bevor sie den Callback ruft. Der Callback braucht deshalb kein `setPath`. Nur wenn die Navigation woanders endet (Umleitung, Wechsel fehlgeschlagen), setzt er mit `setPath` den echten Ort.

Optionen: `-onnavigate`, `-rootlabel` (Beschriftung der Wurzel `/`, Standard `/`). Seit 0.3 erkennt die Zeile Laufwerks- und UNC-Pfade; Backslashes gelten als `/`:

Pfad	Segmente	Ziele
/srv/daten/docs	/ srv daten docs	/ /srv /srv/daten /srv/daten/docs
S:/projekte/daten	S: projekte daten	S:/ S:/projekte S:/projekte/daten
//srv/share/x	//srv/share x	//srv/share/ //srv/share/x

Bis 0.2 stand vor jedem Ziel ein `/` (`/S:/projekte`); Windows liest das als Pfad auf dem aktuellen Laufwerk. `S:\projekte\daten` ergibt dieselben Segmente und Ziele wie `S:/projekte/daten`. Die Zerlegung ist reine Zeichenkettenarbeit.

4.4 tkupreview 0.2

`tkupreview` zeigt Inhalte an, die der Aufrufer liefert (`text`, `markdown`, `message` und weitere Arten); welche Datei welche Art bekommt, entscheidet die App. Seit 0.2 laufen die Meldungen, die das Widget selbst schreibt (Vorschau nicht verfügbar, Datei nicht lesbar, Rendern fehlgeschlagen), über `msgcat`, wenn es geladen ist. Übersetzungen legt die App im Namensraum `::tkutils` ab:

```
package require msgcat
namespace eval ::tkutils {
    ::msgcat::mcset {} "Image preview unavailable." "Bildvorschau
    nicht verfügbar."
}
```

Ohne Katalog bleiben die Texte englisch.

4.5 tkufiledialog 0.2

tkufiledialog ist ein modaler Dialog auf `tkupath` + `tkufilelist`, im Look des Explorer-Stacks. Lokaler Provider als Standard, optional `-provider` für ZIP/Remote.

Befehl	Rückgabe
open ?opt?	Pfad; mit -multiple 1 Liste von Pfaden
save ?opt?	Pfad (Datei muss nicht existieren)
directory ?opt?	gewählter Ordner, sonst der aktuelle

Bei Abbruch ist das Ergebnis `""` (bzw. eine leere Liste).

Option	Bedeutung
-parent w	Toplevel für transient und Platzierung
-title t	Fenstertitel (Standard Open, Save, Choose folder, über msgcat)
-initialdir d	Startordner; Standard $HOME, sonst pwd
-provider p	tuprovider; Standard ein frischer lokaler
-filetypes {{Label {.ext ...}} ...}	Typ-Combobox; filtert nach der ersten Endung; ohne Angabe ein Eintrag All files
-multiple 0/1	nur open: mehrere Dateien
-showhidden 0/1	Punkt-Dateien zeigen
-initialfile name	nur save: Name vorbelegen
-defaultextension .ext	nur save: anhängen, wenn der Name keine Endung hat

```
package require tkutils::tkufiledialog

set f [::tkutils::tkufiledialog::open -parent . -title "Text öffnen" \
    -initialdir [pwd] -filetypes {{Text {.txt .md}} {Alle *}}]
if {$f ne ""} { puts "gewählt: $f" }

set s [::tkutils::tkufiledialog::save -parent . -initialdir [pwd] \
    -initialfile bericht -defaultextension .txt]

set d [::tkutils::tkufiledialog::directory -parent . -initialdir [pwd]]
```

- Doppelklick auf einen Ordner wechselt hinein.
- Doppelklick auf eine Datei schließt den Dialog und liefert den Pfad (seit `tkufiledialog` 0.2; in 0.1 lieferte er `""`).
- *OK* oder Return im Namensfeld bestätigt, Escape oder *Cancel* bricht ab.
- `directory` listet nur Ordner.
- Unbekannte Option: `{TKUTILS TKUFILEDIALOG OPTION <opt>}`, Meldung `unknown option "<opt>"` mit `Known:`-Liste; unbekannter Parent: `{TKUTILS TKUFILEDIALOG PARENT <w>}`, Meldung `bad window path name "<w>" given as -parent`.

Der Doppelklick läuft wie *OK* und Return über `_finishOk` (Test `tkufiledialog-1.4a`). Die Bedienung Schritt für Schritt und der Fehler in 0.1: Anhang G.

Der Dialog ist kein voller Dateimanager.

4.6 Zusammenfassung

- Liste/Baum sprechen Provider, nicht festes FS.
- `tkufiletree -showhidden 1` zeigt Punkt-Dateien auch unter Windows einmal.
- `tkupath` 0.3 zeigt Laufwerks- und UNC-Pfade mit echten Zielen.
- Filedialog: `open`/`save`/`directory`, `""` bei Abbruch; Doppelklick auf eine Datei liefert den Pfad.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux und Xvfb: die Beispiele zu `tkufilelist`, `tkufiletree` und `tkupath`, die Tabelle der Segmente und Ziele, `tkufiledialog.test` (7 von 7).

Nicht getestet: Windows (versteckte Dateien im Baum, Navigation mit Laufwerks- und UNC-Pfaden).

Nächstes Kapitel: Listen und tablelist

5 Listen und tablelist

5.1 tkutablelist 0.5

Wrapper um tablelist aus tklib: Zeilen-/Spalten-API, Sortieren per Kopfklick, editierbare Zellen, Auswahlhilfen, Zeilen-IDs, CSV laden und speichern, Fußzeile, Hierarchie. Nicht im Umbrella direkt, kommt aber über `tkufilelist` mit.

```
package require tkutils::tkutablelist
set w [::tkutils::tkutablelist::widget .t -columns {Name Größe Typ}]
pack $w -fill both -expand 1
::tkutils::tkutablelist::setRows $w {
    {a.txt 120 Text} {b.md 80 Markdown} {c.png 4096 Bild}
}
::tkutils::tkutablelist::selectRows $w {0 2}
puts [::tkutils::tkutablelist::selection $w]   ;# -> 0 2
```

Optionen von `widget`: `-columns`, `-stretch`, `-titlecolumns`, `-sortable`, `-editable`, `-selectmode` (Standard `extended`), `-exportselection` (Standard **0**), `-stripes`, `-selectcommand`, `-doublecommand`, `-editendcommand`, `-footer`. Weitere Standardwerte: `-stretch all`, `-titlecolumns 0`, `-sortable 1`, `-editable 0`, `-stripes ""` (keine Streifen), `-footer 0`. Die rohe Tablelist liefert `tableWidget` (Pfad `<w>.tbl`).

5.2 Warum `-exportselection 0`

Tk exportiert die Auswahl eines Listen-Widgets normalerweise als X-Selection PRIMARY. Nimmt ein anderes Widget PRIMARY — ein Eintrag in einem Menü, eine Combobox, ein Entry in der Toolbar —, verliert die Tabelle ihre Auswahl. Bei einem Dateimanager heißt das: Mehrere Dateien markieren, Rechtsklick, und die Markierung ist weg, bevor *Kopieren* greift.

Mit exportselection 0 bleibt die Auswahl stehen. Diese Vorgabe kam mit **0.4**.

```
package require tkutils::tkutablelist
set w [::tkutils::tkutablelist::widget .alt -columns {A B} \
    -exportselection 1]
puts [[::tkutils::tkutablelist::tableWidget $w] cget -exportselection]
      ;# -> 1
```

Wer das alte Tk-Verhalten braucht, setzt `-exportselection 1`.

5.3 Zeilen-IDs und Sortieren

Eine Zeilennummer gilt nur, bis der Benutzer sortiert. Seit **0.5** trägt jede Zeile auf Wunsch eine ID in ihrer Zeilenoption `-name`. `setRows` nimmt die IDs mit `-ids`, `insert` als drittes Argument.

```
::tkutils::tkutablelist::setRows $w {
    {c.png 4096 Bild} {a.txt 120 Text} {b.md 80 Markdown}
} -ids {f3 f1 f2}
::tkutils::tkutablelist::sortBy $w 0
puts [::tkutils::tkutablelist::findId $w f3]          ;# -> 2
puts [::tkutils::tkutablelist::selectIds $w {f2 f9}]  ;# -> f2
puts [::tkutils::tkutablelist::selectedIds $w]        ;# -> f2
```

Befehl	Wirkung
rowId w index	ID der Zeile, leer ohne ID

Befehl	Wirkung
findId w id	aktuelle Zeilennummer, -1 ohne Treffer
selectedIds w	IDs der ausgewählten Zeilen; eine Zeile ohne ID fehlt, curselection liefert alle
selectIds w ids	wählt diese Zeilen aus, aktiviert die erste und rollt zu ihr; liefert die gefundenen IDs

`setRows` fügt alle Zeilen mit einem `insertlist` ein und stellt danach die Sortierung wieder her, die der Benutzer zuletzt gewählt hat. Passt die Zahl der IDs nicht zur Zahl der Zeilen, wirft es `{TKUTILS TKUTABLELIST IDS}` mit `-ids needs one id per row (2 rows, 1 ids)`. `selectRows` aktiviert ebenfalls die erste Zeile und rollt zu ihr.

Mit `-sortable 1` (Vorgabe) sortiert ein Klick auf den Kopf über `tablelist::sortByColumn`: Der zweite Klick dreht die Richtung, der Pfeil erscheint, `<<TablelistColumnSorted>>` kommt. Umschalt+Klick nimmt eine weitere Spalte dazu (`tablelist::addToSortColumns`).

5.4 CSV

`toCsv` liefert, was der Benutzer sieht: sichtbare Zeilen, sichtbare Spalten, Rohwerte der Zellen. `saveCsv` schreibt dasselbe als UTF-8.

```
set tbl [::tkutils::tkutablelist::tableWidget $w]
::tkutils::tkutablelist::setRows $w {
    {a.txt 120 Text} {b.md 80 Markdown} {c.png 4096 Bild}
}
$tbl rowconfigure 1 -hide 1
$tbl columnconfigure 2 -hide 1
puts [::tkutils::tkutablelist::toCsv $w]
;# Name,Größe
;# a.txt,120
;# c.png,4096
::tkutils::tkutablelist::saveCsv $w liste.csv -profile excel-de
```

Option	Vorgabe	Wirkung
-header	1	Spaltentitel als erste Zeile
-all	0	1: alle Zeilen und Spalten, auch versteckte
-formatted	0	1: angezeigte Werte statt Rohwerte
-encoding	utf-8	nur saveCsv

Alle anderen Optionen gehen an `tclutils::tucsv::text`, etwa `-delimiter ;` oder `-profile excel-de` (Semikolon, CRLF, BOM). Dafür braucht `tkutablelist` `tclutils::tucsv` 0.2 aus tclutils 0.69.0.

5.5 Versionen

Es gibt nur **0.5**. `package require tkutils::tkutablelist 0.4` und `… 0.3` laden 0.5 (gleiche Major-Version 0). Liegt eine alte `tkutablelist-0.4.tm` noch im Baum, lädt `package require` trotzdem die höhere Version; die alte Datei beim Überkopieren löschen.

Unbekannte Option: `{TKUTILS TKUTABLELIST OPTION -colums}` mit dem Optionsnamen, wie bei `tkuopts`. Die Meldung hat zwei Zeilen:

```
unknown option "-colums"
Known: -columns -doublecommand -editable -editendcommand
    -exportselection -footer -selectcommand -selectmode -sortable
    -stretch -stripes -titlecolumns
```

Bis 0.46.0 fehlten Optionsname und `Known:`-Liste (Anhang Fallen). Ein fehlender Wert meldet `{TKUTILS TKUTABLELIST OPTION}` mit `missing value for option "-columns"`.

5.6 Tablelist-Helfer

Sortieren, Anzeige, Zwischenablage, Fußzeile, Suche, Spaltenlayout, Baum, Filtern, Gruppieren und Ziehen stehen im Kapitel Tablelist-Helfer. `tkutltools` lädt alle zehn. Wrapper und Helfer laden `tablelist_tile`; die Reihenfolge zum Umbrella ist frei (Kapitel tkutils verstehen).

5.7 Zusammenfassung

- Auswahl bleibt bei Menüs (`-exportselection 0`); altes Verhalten über `-exportselection 1`.
- Zeilen-IDs mit `setRows -ids`; `findId`, `selectIds` gelten auch nach dem Sortieren.
- `toCsv` und `saveCsv` exportieren nur Sichtbares; `-all 1` nimmt alles.
- `require … 0.4` lädt 0.5.
- `tkutltools` lädt alle zehn Helfer. Befehle: Kapitel Tablelist-Helfer.

Getestet mit Tcl/Tk 8.6.14 (Tablelist 6.20) und 9.0.4 (Tablelist 7.11) unter Linux und Xvfb: alle Beispiele und Meldungen dieses Kapitels.

Nächstes Kapitel: Tablelist-Helfer

6 Tablelist-Helfer

Dieses Kapitel setzt Listen und tablelist fort. Die zehn Module hier arbeiten auf einer fertigen Tablelist, nicht auf dem Wrapper `tkutablelist`. Für den Wrapper gilt `[tkutils::tkutablelist::tableWidget $w]`.

tkutltools 0.3 lädt alle zehn und `tclutils::tunum`. Es steht außerhalb des Umbrellas. Ist weder `tablelist` noch `tablelist_tile` geladen, fordert es `tablelist_tile` an. Ist eine der beiden Varianten schon da, bleibt sie. Wrapper und Helfer laden dieselbe Variante; die Reihenfolge zum Umbrella ist frei (Kapitel tkutils verstehen).

Modul	Version	Aufgabe
tkutlsort	0.3	Vergleich je Spalte, leere Zellen
tkutlfmt	0.3	Anzeige je Spalte

Modul	Version	Aufgabe
tkutlclip	0.3	Auswahl als TSV oder CSV, Strg+C
tkutlfooter	0.4	Fußzeile, Summe, weitere Aggregate
tkutlfind	0.3	Suche mit Markierung
tkutlstate	0.4	Spaltenlayout und Sortierung sichern
tkutltree	0.3	verschachtelte Dicts als Baum, Nachladen
tkutlfilter	0.1	Zeilen verstecken, die nicht passen
tkutlgroup	0.1	flache Zeilen gruppieren, mit Summen
tkutldrag	0.1	Zeilen mit der Maus umsortieren

`tkutltools` fordert `tkutlfmt` ab 0.1 an; die Datei im Baum ist 0.3.

6.1 Sortieren: tkutlsort

```
::tkutils::tkutlsort::column  tbl spalte typ ?befehl? ?-empty
    first|last?
::tkutils::tkutlsort::columns tbl {spalte typ spalte typ ...} ?-empty
    first|last?
```

Typ	Tablelist-Modus
string	dictionary
nocase	asciinocase
integer	integer
real	real
num	Befehl über tclutils::tunum
command	eigener Vergleich mit zwei Argumenten

`num` liest Gruppierung und Währung. Unlesbare Zellen stehen hinter den Zahlen. Das Modul setzt nur den Vergleich. Sortiert wird mit `sortbycolumn` oder einem Klick auf die Spaltenüberschrift.

Eine leere Zelle in einer `integer`- oder `real`-Spalte bringt den Tablelist-Vergleich zum Fehler. Seit 0.3 legt `-empty first` oder `-empty last` fest, wo leere und unlesbare Zellen stehen, und zwar in beiden Richtungen. `-empty` gilt für `integer`, `real` und `num`; `columns` gibt es nur an diese Spalten weiter.

```
tablelist::tablelist .t -columns {10 Artikel left 6 Menge right}
.t insertlist end {{Schrauben 100} {Muttern {}} {Dübel 20}}
::tkutils::tkutlsort::column .t 1 integer -empty last
.t sortbycolumn 1 -decreasing
puts [lmap r [.t get 0 end] {lindex $r 1}]   ;# -> 100 20 {}
```

Ein unbekannter Typ wirft `{TKUTILS TKUTLSORT TYPE}`, eine `command`-Spalte ohne Befehl

{TKUTILS TKUTLSORT CMD}, eine ungerade Liste an columns {TKUTILS TKUTLSORT SPEC}, ein anderer Wert für -empty {TKUTILS TKUTLSORT EMPTY} mit bad -empty "middle": must be first or last, eine andere Option OPTION.

6.2 Anzeige: tkutlfmt

```
::tkutils::tkutlfmt::column  tbl spalte typ ?option wert ...?
::tkutils::tkutlfmt::columns tbl {spalte typ ?{option wert ...}? ...}
```

Die Zelle behält den Rohwert. Die Anzeige kommt über -formatcommand. integer, number, currency und percent stehen rechts, sofern -align fehlt. date formatiert mit clock.

Option	Vorgabe	Wirkung
-locale	eu	eu mit Punkt als Tausender und Komma als Dezimal, oder us
-group	aus -locale	Tausendertrenner
-decimal	aus -locale	Dezimalzeichen
-decimals	je Typ	Nachkommastellen: integer 0, number und currency 2, percent 1
-symbol	€	nur Währung
-symbolpos	post	pre oder post
-align	automatisch	right, left, center
-informat	leer	clock scan, sonst Zahl oder erkennbare Zeit
-outformat	%Y-%m-%d	clock format

Ein unbekannter Typ wirft {TKUTILS TKUTLFMT TYPE}, eine unbekannte Option {TKUTILS TKUTLFMT OPTION -name} und nennt die bekannten, ein fremdes -locale {TKUTILS TKUTLFMT LOCALE} mit locale must be eu or us.

6.3 Zwischenablage: tkutlclip

```
::tkutils::tkutlclip::copySelection   tbl ?option wert ...?
::tkutils::tkutlclip::copyAll         tbl ?option wert ...?
::tkutils::tkutlclip::asText          tbl zeilen ?option wert ...?
::tkutils::tkutlclip::installBindings tbl ?option wert ...?
```

copySelection und copyAll liefern die Zahl der kopierten Zeilen; copyAll nimmt nur sichtbare Zeilen, keine versteckten und keine in zugeklappten Knoten. asText liefert den Text und lässt die Zwischenablage in Ruhe. installBindings bindet <<Copy>> auf den Körper der Tabelle (bodytag) und beendet die Bindung mit break. Ohne break liefe danach die eigene <<Copy>>-Bindung von Tablelist und überschriebe die Zwischenablage mit ihrem TSV; bis 0.2 kam deshalb bei Strg+C nie das gewünschte Format an. Die Optionen prüft installBindings sofort, nicht erst beim ersten Strg+C.

Option	Vorgabe	Wirkung
-format	tsv	tsv oder csv
-header	0	Spaltentitel als erste Zeile
-formatted	1	angezeigte Werte, bei 0 die Rohzellen
-columns	visible	visible oder all
-separator	,	Trennzeichen für CSV, ein Zeichen; TSV nimmt immer den Tabulator

```
::tkutils::tkutlclip::installBindings .t -format csv -separator {;}
    -header 1
puts [::tkutils::tkutlclip::asText .t {0 1} -format csv -separator {;}]
```

TSV ersetzt Tabulator und Zeilenumbruch in einer Zelle durch ein Leerzeichen. CSV behält sie und läuft immer über `tclutils::tucsv` 0.2, wie `tkutablelist::toCsv`: Dieselbe Zeile ergibt in beiden denselben Text. Fehlercodes: `{TKUTILS TKUTLCLIP OPTION}` (`unknown option "-foo"`, ohne Optionsnamen im Code und ohne `Known:`-Zeile), `FORMAT` (`format must be tsv or csv`), `COLUMNS`, `SEPARATOR` (`separator must be one character`).

6.4 Fußzeile: tkutlfooter

```
::tkutils::tkutlfooter::attach  haupt fuss ?-autowire 1?
::tkutils::tkutlfooter::update  haupt fuss
::tkutils::tkutlfooter::setvals fuss {wert ...}
::tkutils::tkutlfooter::autosum haupt fuss ?-columns {1 2}? ?-label
    SUM:? ?-format %.2f?
::tkutils::tkutlfooter::autoagg haupt fuss ?-columns {1 sum ...}?
    ?-label text? ?-format %.2f?
::tkutils::tkutlfooter::detach  haupt fuss
```

`attach` macht aus `fuss` eine einzelne Zeile ohne Auswahl und übernimmt die Spalten. `-autowire 1` hängt das horizontale Scrollen an das bestehende `-xscrollcommand` der Haupttabelle, statt es zu ersetzen. Seit 0.2 übernimmt die Fußzeile auch `-titlecolumns`, versteckte Spalten und `-stretch`. Den ersten Abgleich plant `attach` für die Leerlaufzeit; `detach` bricht ihn ab, und ist eine der Tabellen bis dahin zerstört, tut er nichts (0.4; vorher kam ein Hintergrundfehler `invalid command name`).

`autosum` summiert die genannten Spalten, Vorgabe alle, und schreibt `-label` in Spalte 0. Zahlen liest `tclutils::tunum`. Nach einer Datenänderung läuft `autosum` erneut. `autoagg` nimmt je Spalte `sum`, `avg`, `min`, `max`, `count` oder `countnum`. Zähler bleiben ganzzahlig. Fehler tragen `{TKUTILS TKUTLFOOTER <GRUND>}`, etwa `SPEC` für eine ungerade `-columns`-Liste bei `autoagg` und `FUNC` für ein unbekanntes Aggregat. Unbekannte Optionen prüft das Modul nicht: `attach`, `autosum` und `autoagg` übergehen sie ohne Fehler.

6.5 Suchen: tkutlfind

```
::tkutils::tkutlfind::find    tbl text ?option wert ...?
::tkutils::tkutlfind::next    tbl
::tkutils::tkutlfind::prev    tbl
::tkutils::tkutlfind::matches tbl
::tkutils::tkutlfind::clear   tbl
```

`find` liefert die Trefferzahl und färbt die Zellen. `next` und `prev` laufen im Kreis und rufen `seecell`; sie liefern `{zeile spalte}` mit der aktuellen Zeilennummer. Gesucht wird im angezeigten Text, also nach `tkutlfmt`. Seit 0.3 merkt sich das Modul die Treffer als Zeilenschlüssel: Nach einem Sortieren stimmen `matches`, `next` und `prev` weiter, gelöschte Zeilen fallen heraus. Versteckte Zeilen, etwa von `tkutlfilter`, übergeht `find`, außer mit `-hidden 1`. Liegt ein Treffer in einem zugeklappten Knoten, klappt `next` die Vorfahren auf. `clear` stellt die Zellfarben wieder her, die vor `find` gesetzt waren.

Option	Vorgabe	Wirkung
-columns	alle sichtbaren	Spalten
-mode	substring	substring, exact, glob, regexp
-nocase	1	ohne Großschreibung
-highlightbg	#ffe9a8	Hintergrund der Treffer
-highlightfg	leer	Vordergrund, optional
-hidden	0	1: auch versteckte Zeilen

```
::tkutils::tkutlfind::find .t schr      ;# -> 1
.t sortbycolumn 0 -increasing
::tkutils::tkutlfind::next .t           ;# -> 2 0  (Schrauben steht
    jetzt unten)
```

Fehlercodes: `{TKUTILS TKUTLFIND MODE}` (`unknown mode "xx"`) und `OPTION` (`unknown option "-foo"`, ohne Optionsnamen im Code).

6.6 Layout sichern: tkutlstate

```
::tkutils::tkutlstate::save           tbl
::tkutils::tkutlstate::restore        tbl zustand
::tkutils::tkutlstate::saveToFile     tbl pfad
::tkutils::tkutlstate::restoreFromFile tbl pfad
::tkutils::tkutlstate::saveVia        tbl schlüssel setPrefix
::tkutils::tkutlstate::restoreVia     tbl schlüssel getPrefix
::tkutils::tkutlstate::autosave       tbl schlüssel setPrefix ?-delay
    ms? ?-onerror befehl?
::tkutils::tkutlstate::cancelAutosave  tbl
::tkutils::tkutlstate::lastError      tbl
```

`save` liefert seit 0.4 ein Dict der Version 2: `version`, `ncols`, `columns` in der angezeigten Reihenfolge (je Spalte `id`, `width`, `hide`) und `sort` mit allen Sortierspalten und Richtungen.

Die `id` ist der Spaltenname (`-name`), sonst `#` und der Titel. Breite und Sichtbarkeit hängen so an der Spalte, nicht an ihrer Nummer; `restore` stellt die Reihenfolge mit `movecolumn` wieder her.

```
.t columnconfigure 0 -name artikel
::tkutils::tkutlstate::save .t
;# -> version 2 ncols 3 columns {{id artikel width 10 hide 0}
;#    {id #Menge width 6 hide 0} {id #Preis width 8 hide 0}}
;#    sort {columns {{#Menge}} orders decreasing}
```

Zustände der Version 1 aus 0.1 bis 0.3 (nach Spaltennummer) stellt `restore` weiter her. `autosave` speichert auch nach Verstecken und Sortieren. Scheitert das Speichern, schickt `autosave` der Tabelle `<<TkutlStateSaveFailed>>` (%d ist die Meldung), ruft `-onerror` mit Tabelle und Meldung und merkt sich die Meldung für `lastError`, bis ein Speichern gelingt; bis dahin verschwand der Fehler in einem `catch`. `restoreFromFile` und `restoreVia` liefern `1`, wenn ein Zustand ankam, und `0`, wenn er fehlt. `saveVia` hängt Schlüssel und Dict an den Befehlspräfix. `restore` ändert nur Spalten, die in beiden Zuständen vorkommen. Ein Wert, der kein Zustand ist, wirft `{TKUTILS TKUTLSTATE STATE}` mit `not a tkutlstate dict`. Eine unbekannte Option an `autosave` meldet `{TKUTILS TKUTLSTATE badoption}` — als einziger Grund im Modul klein geschrieben.

6.7 Baum: tkutltree

Ein Knoten ist ein Dict der Felder plus optional `children`, eine Liste weiterer Knoten. Die oberste Ebene ist eine Liste von Knoten.

```
::tkutils::tkutltree::fromData tbl daten -fields {feld ...} ?option
    wert ...?
::tkutils::tkutltree::toData   tbl       -fields {feld ...}
    ?-childrenkey children? ?-node root? ?-idfield f?
::tkutils::tkutltree::clear    tbl ?-node root?
::tkutils::tkutltree::lazy     tbl ladebefehl -fields {feld ...}
    ?option wert ...?
::tkutils::tkutltree::reveal   tbl index
::tkutils::tkutltree::findId   tbl id
```

Feld *i* füllt Spalte *i*. Ein fehlendes Feld wird eine leere Zelle. `toData` liest die Rohzellen, nicht den formatierten Text. Hin und zurück ergibt dieselbe Struktur. `fromData` fügt jede Ebene mit einem `insertchildlist` ein und liefert seit 0.3 die Schlüssel der obersten Knoten (bis 0.2 den Tabellenpfad).

Option von fromData	Wirkung
-childrenkey	Schlüssel der Kindliste, Vorgabe children
-parent	Knoten, unter dem eingefügt wird, Vorgabe root
-idfield f	Wert von Feld f wird -name der Zeile

Option von fromData	Wirkung
-expand	none alles zu, all alles auf, eine Zahl N: N Ebenen sichtbar; ohne Angabe wie Tablelist einfügt (aufgeklappt)
-lazykey f	ein Knoten mit wahrem Feld f und ohne Kinder bekommt einen Pfeil und lädt später nach

`lazy` setzt `-expandcommand` und `-populatecommand` der Tabelle. Beim ersten Aufklappen eines solchen Knotens, oder wenn `searchcolumn -descend` ihn erreicht, ruft es einmal `ladebefehl tbl schlüssel` und fügt die gelieferten Knoten ein.

```
proc ladeOrdner {tbl key} {
    set dir [$tbl rowcget $key -name]
    lmap f [lsort [glob -nocomplain -directory $dir *]] {
        list pfad $f name [file tail $f] lazy [file isdirectory $f]
    }
}
tablelist::tablelist .baum -columns {30 Name left} -treecolumn 0
::tkutils::tkutltree::lazy .baum ladeOrdner -fields {name} \
    -idfield pfad -lazykey lazy
::tkutils::tkutltree::fromData .baum [list [list pfad $start name
    $start lazy 1]] \
    -fields {name} -idfield pfad -lazykey lazy
```

`findId` sucht eine ID im ganzen Baum (`findrowname -descend`), `reveal` klappt die Vorfahren einer Zeile auf und rollt zu ihr. Fehlercodes: `{TKUTILS TKUTLTREE FIELDS}` (`-fields is required`), `OPTION` und `EXPAND` (`bad -expand "x": must be none, all or a level count`).

6.8 Filtern: tkutlfilter

tkutlfilter versteckt die Zeilen, die nicht passen. Die Zeilen bleiben in der Tabelle; IDs, Attribute und eingebettete Fenster bleiben erhalten.

```
::tkutils::tkutlfilter::filter tbl text ?option wert ...?
::tkutils::tkutlfilter::later  tbl ms text ?option wert ...?
::tkutils::tkutlfilter::clear  tbl
::tkutils::tkutlfilter::query  tbl
```

Eine Zeile bleibt sichtbar, wenn der Text in einer der Spalten vorkommt, jede Bedingung aus `-where` passt und `-command` 1 liefert. `filter` liefert die Zahl der passenden Zeilen. Ein leerer Text ohne `-where` und `-command` hebt den Filter auf. `later` filtert erst nach `ms` Millisekunden Ruhe und schickt danach `<<TkutlFilterApplied>>` an die Tabelle; das passt zu `<KeyRelease>` an einem Eingabefeld. Wird die Tabelle vorher zerstört, entfällt der wartende Aufruf.

Option	Vorgabe	Wirkung
-columns	alle sichtbaren	Spalten für den Text

Option	Vorgabe	Wirkung
-mode	substring	substring, exact, glob, regexp
-nocase	1	ohne Großschreibung
-where	leer	{spalte text ...}, alle müssen passen; Spalte als Nummer, -name oder Titel
-command	leer	eigener Test befehl tbl schlüssel zeile, liefert 0 oder 1

```
::tkutils::tkutlfilter::filter .t "" -where {Artikel u}   ;# -> 2
puts [lmap r [.t get 0 end -viewable] {lindex $r 0}]      ;# ->
    Muttern Schrauben
::tkutils::tkutlfilter::clear .t
```

Das Dict, das tkufilterbar an sein `-command` gibt, passt direkt in `-where`. Zeilen, die die Anwendung selbst versteckt hat, bleiben versteckt und zählen nicht mit. Im Baum bleibt ein Elternknoten sichtbar, solange ein Nachfahre passt. `size` und `get 0 end` zählen weiter alle Zeilen; die sichtbaren liefert `get 0 end -viewable`. `tkutablelist::toCsv` und `tkutlclip::copyAll` nehmen nur sichtbare Zeilen. Fehlercodes: `{TKUTILS TKUTLFILTER <GRUND>}` mit `OPTION`, `MODE`, `REGEXP`, `WHERE`, `COLUMN`.

6.9 Gruppieren: tkutlgroup

tkutlgroup macht aus flachen Zeilen einen Baum: je Wert einer Schlüsselspalte eine Elternzeile, die Zeilen selbst als Kinder.

```
::tkutils::tkutlgroup::group   tbl zeilen schlüsselspalte ?option wert
    ...?
::tkutils::tkutlgroup::ungroup tbl
```

```
set orders {
    {{Tämo GmbH} Schrauben 100 200,00}
    {{Quarz KG}  Muttern    50  50,00}
    {{Tämo GmbH} Dübel      20  60,50}
}
tablelist::tablelist .o -treecolumn 0 \
    -columns {12 Kunde left 10 Artikel left 6 Menge right 8 Betrag
    right 4 N right}
::tkutils::tkutlgroup::group .o $orders 0 -sum {2 3} -count 4
puts [.o get 0]      ;# -> {Tämo GmbH} {} 120 260.50 2
```

Option	Vorgabe	Wirkung
-sum	leer	Spalten mit Summe in der Elternzeile
-count	leer	Spalte für die Zahl der Zeilen
-blank	1	Schlüsselspalte in den Kindzeilen leer

Option	Vorgabe	Wirkung
-format	%.2f	format für Summen, die keine ganze Zahl sind
-font	fett	Schrift der Elternzeilen
-expand	1	0: Gruppen zugeklappt
-replace	1	vorher alle Zeilen entfernen; 0 hängt die Gruppen an

Ein zweiter Aufruf von `group` ersetzt die Gruppen; bis dahin stapelte er sie unter die ersten. Zahlen wie `200,00` liest `tclutils::tunum`. Eine Zelle ohne Zahl zählt als 0. Gruppen erscheinen in der Reihenfolge, in der ihr Schlüssel zuerst vorkommt. Beim Sortieren sortiert Tablelist die Elternzeilen nach ihren Summen und die Kinder in jeder Gruppe; die Gruppen bleiben zusammen. `ungroup` liefert die Zeilen in der angezeigten Reihenfolge zurück, mit gefüllter Schlüsselspalte. Fehlercodes: `{TKUTILS TKUTLGROUP OPTION}` und `COLUMN` (`bad column "9": must be 0..4`).

6.10 Ziehen: tkutldrag

tkutldrag lässt den Benutzer Zeilen mit der Maus umsortieren und meldet jede Verschiebung.

```
::tkutils::tkutldrag::enable  tbl ?option wert ...?
::tkutils::tkutldrag::disable tbl
::tkutils::tkutldrag::order   tbl ?eltern?
```

`enable` setzt `-movablerows`. Das wirkt nur im Auswahlmodus `single` oder `multiple`; `enable` macht aus `browse` `single` und aus `extended` `multiple` und liefert den Modus. `order` liefert die IDs (`-name`) der Kinder von `eltern` (Vorgabe `root`) in der angezeigten Reihenfolge, für Zeilen ohne ID den Schlüssel.

Option	Vorgabe	Wirkung
-command	leer	nach dem Ablegen: befehl tbl id eltern position
-samelevel	1	nur unter Geschwistern umsortieren, keine neuen Eltern
-acceptchild	leer	eigenes -acceptchildcommand, ersetzt -samelevel
-acceptdrop	leer	eigenes -acceptdropcommand, ersetzt -samelevel

```
proc verschoben {tbl id eltern pos} {
    speichereReihenfolge [::tkutils::tkutldrag::order $tbl]
}
::tkutils::tkutldrag::enable .t -command verschoben
```

Escape bricht das Ziehen ab. Mit `-samelevel 1` darf eine Zeile im Baum nur vor einem Geschwister landen; das Ende einer Kindliste unterhalb der obersten Ebene ist dann kein Ziel. Eine unbekannte Option wirft `{TKUTILS TKUTLDRAG OPTION}`.

6.11 Merkliste

- `tkutltools` 0.3 lädt die zehn Helfer und `tunum`.
- Sortieren und Formatieren lassen den Rohwert in der Zelle.
- Leere Zellen in Zahlenspalten: `tkutlsort … -empty last`.
- Strg+C mit eigenem Format: `tkutlclip::installBindings`, seit 0.3 wirksam.
- Die Fußzeile ersetzt ein vorhandenes horizontales Scrollen nicht.
- `tkutlstate` kennt nur Spalten, die es noch gibt; mit `-name` überlebt eine Spalte auch einen neuen Titel.
- Filtern versteckt, Suchen markiert; beide zählen über Schlüssel.

Getestet mit Tcl/Tk 8.6.14 (Tablelist 6.20) und 9.0.4 (Tablelist 7.11) unter Linux und Xvfb: Laden von `tkutltools`, die Beispiele, die Fehlercodes und Meldungen. Das Ziehen mit der Maus ist nur über das Ereignis `<<TablelistRowMoved>>` geprüft, nicht mit echter Maus.

Nächstes Kapitel: Geometrie und Units

7 Geometrie und Units

7.1 tkugeometry 0.1

tkugeometry merkt sich Größe, Position und Zoom-Zustand eines Toplevels über `tclutils::tusettings`, in einer eigenen Datei `<Konfig-Ordner>/<app>/geometry.ini`. Eine App, die beim Beenden ihr eigenes Settings-Dict speichert, kann die Geometrie so nicht überschreiben. Braucht mindestens tclutils **0.64.0** (dort kam `tusettings` dazu); empfohlen ist 0.69.0.

Der Konfig-Ordner kommt aus `tusettings::dir config`: unter Linux `$XDG_CONFIG_HOME/<app>`, sonst `$HOME/.config/<app>`; laut Code unter Windows `%APPDATA%\<app>` und unter macOS `$HOME/Library/Application Support/<app>` (nur unter Linux geprüft).

Befehl	Wirkung	Rückgabe
track w app ?-key k? ?-default geo?	jetzt herstellen, beim Zerstören speichern	angewandte Geometrie oder ""
restore w app ?-key k? ?-default geo?	gespeicherte Geometrie anwenden	angewandte Geometrie oder ""
save w app ?-key k?	jetzt speichern	gespeicherte Geometrie
key w	Schlüssel aus dem Pfad	. → main, .dlg → dlg

```
package require tkutils::tkugeometry
::tkutils::tkugeometry::track . meineapp -default 800x600         ;#
    -> 800x600
toplevel .vorschau
::tkutils::tkugeometry::track .vorschau meineapp -default 400x300 ;#
    -> 400x300
```

`track` stellt selbst her — ein vorheriges `restore` ist nicht nötig. `-default` (eine Größe wie `800x600` oder eine volle Geometrie) gilt, wenn noch nichts gespeichert ist. Gespeichert wird beim `<Destroy>` des Fensters über einen eigenen Bindtag; die `<Destroy>`-Bindings der App

bleiben unberührt. `save` ruft man nur, wenn zu einem anderen Zeitpunkt gesichert werden soll. Mit `-key` wählen Sie einen anderen Abschnittsnamen als den aus `key`.

Unter Xvfb ohne Fenstermanager steht nach dem Beenden in `geometry.ini`:

```
[main]
geometry=800x600+0+0
zoomed=0
```

7.2 Außerhalb des Bildschirms

Eine gespeicherte Position, die nicht mehr auf dem Bildschirm liegt — der zweite Monitor fehlt, die Auflösung ist kleiner —, wird nicht benutzt. Das Fenster behält seine Größe (falls nötig auf den Bildschirm verkleinert) und wird **zentriert**. "Auf dem Bildschirm" heißt: der obere Streifen des Fensters (Titelleiste, 100×30 Pixel) liegt in Tks virtueller Wurzel — unter X11 alle Monitore als ein Rechteck, unter Windows der virtuelle Desktop. Eine Lücke zwischen verschieden großen Monitoren erkennt die Prüfung nicht.

Beispiel bei einem Bildschirm von 1280×1024 Pixeln: Gespeichert ist `500x400+5000+5000`, `restore` setzt `500x400+390+312` — gleiche Größe, zentriert.

Der Zoom-Zustand wird als `zoomed` gespeichert und dort wiederhergestellt, wo Tk ihn kennt (`wm state zoomed` unter Windows/macOS, `wm attributes -zoomed` unter X11).

Fehler: `{TKUTILS TKUGEOMETRY WINDOW}`, Meldung `".nix" is not an existing toplevel` (kein vorhandenes Toplevel). Eine unbekannte Option meldet `{TCLUTILS COMMON OPTION <opt>}` mit `unknown option "<opt>"` und `Known: -key -default`, weil das Modul die Optionsprüfung von tclutils benutzt. Ungültige App-Namen und unlesbare Dateien melden die `tusettings`-Fehler, etwa `{TCLUTILS TUSETTINGS APP}` mit `bad app name "a/b": must be a plain directory name`.

7.3 tkuunits 0.1

tkuunits rechnet Bildschirmmaße zwischen px, pt, mm, cm und inch um. Tk versteht die Einheiten selbst (`winfo fpixels. 10m`), aber Pixel zurück in Millimeter oder Einheit in Einheit bleibt dem Aufrufer. Für Etiketten- und Druckvorschauen.

Befehl	Rückgabe
units	px pt mm cm inch
toPixels wert einheit ?fenster?	Pixel als Fließkommazahl
fromPixels px ?fenster?	Dict mit px pt mm cm inch
convert wert von nach ?fenster?	Fließkommazahl

```
package require tkutils::tkuunits
puts [::tkutils::tkuunits::convert 62 mm cm]       ;# -> 6.2
puts [::tkutils::tkuunits::convert 1 inch pt]      ;# -> 72.0
set px [::tkutils::tkuunits::toPixels 62 mm]       ;# Etikettenbreite
    am Schirm
puts [dict get [::tkutils::tkuunits::fromPixels 300] mm]
```

Neben `px pt mm cm inch` gelten die Tk-Buchstaben `p m c i` und `in`. `pt` ist der Druckerpunkt, 1/72 Zoll, wie in Tk. Pixelwerte richten sich nach der Auflösung des Fensters (Standard.) und damit nach `tk scaling`; Einheit-zu-Einheit-Ergebnisse hängen davon nicht ab.

Fehler (Meldungen aus dem Test):

Code	Meldung
{TKUTILS TKUUNITS UNIT}	unknown unit "furlong": must be px, pt, mm, cm, inch
{TKUTILS TKUUNITS VALUE}	expected a number but got "2c"
{TKUTILS TKUUNITS WINDOW}	bad window path name ".nix"

Bei `tk scaling` 1,39 (Xvfb-Standard) und 2,0 bleibt `convert 1 inch pt` bei `72.0`; `convert 62 mm cm` liefert bei 1,39 `6.2`, bei 2,0 `6.199999999999999` — Rundungsrest der Fließkommarechnung. Runden Sie Ausgaben selbst, etwa mit `format %.2f`.

7.4 Zusammenfassung

- `track` stellt her und speichert beim Schließen; `-default` für den ersten Start.
- Position außerhalb des Bildschirms → zentriert, Größe bleibt.
- Units für geräteunabhängige Maße am Bildschirm.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux, Xvfb ohne Fenstermanager, Bildschirm 1280×1024: alle Beispiele, Fehlercodes und Meldungen dieses Kapitels.

Nicht getestet: Konfig-Ordner unter Windows und macOS, mehrere Monitore, Zoom-Zustand (Xvfb ohne Fenstermanager kennt keinen).

Nächstes Kapitel: Aktionen und Layout

8 Aktionen und Layout

8.1 tkuaction 0.2

tkuaction beschreibt eine Aktion einmal — Label, Befehl, Icon, Tastenkürzel, Enable/Check-Zustand — und hält alle registrierten Widgets und Menüeinträge synchron. Das Modul ist das Model; gezeichnet wird in den Widgets.

```
package require tkutils::tkuaction

proc speichern {} { puts gespeichert }
::tkutils::tkuaction::define save -label "Speichern" -command
```

```
    speichern \
    -accelerator "Ctrl+S"

foreach top {.a .b} {
    toplevel $top
    ttk::button $top.save -text Speichern \
        -command {::tkutils::tkuaction::invoke save}
    pack $top.save
    ::tkutils::tkuaction::register save $top.save
    menu $top.m -tearoff 0
    ::tkutils::tkuaction::addMenuItem $top.m save
}

::tkutils::tkuaction::setEnabled save 0 .b      ;# nur Fenster .b
puts [::tkutils::tkuaction::getEnabled save .a] ;# -> 1
puts [::tkutils::tkuaction::getEnabled save .b] ;# -> 0
::tkutils::tkuaction::setEnabled save 1         ;# global, hebt .b auf
```

8.2 Enable pro Fenster

- `setEnabled id 0|1` ohne Fenster wirkt auf alle gebundenen Widgets, wie in 0.1. Dieser Aufruf löscht zugleich alle Fenster-Overrides.
- `setEnabled id 0|1 $w` graut nur Widgets und Menüeinträge im Toplevel von `$w`. Ein Kind-Widget wird mit `winfo toplevel` aufgelöst; bei Menüs, die selbst Toplevels sind, gilt der erste Vorfahr, der kein Menü ist.
- `getEnabled` und `groupSet` nehmen dasselbe optionale Fenster.
- `invoke` prüft immer das **globale** Flag.
- Unbekanntes Fenster: `{TKUTILS TKUACTION WINDOW}`.

Menüzeilen sind keine Widgets; `addMenuItem $menu id` legt die Zeile an und registriert ihren Index, `registerMenu`/`unregisterMenu` binden eine bestehende Zeile. Gruppen schalten mehrere Aktionen auf einmal:

```
package require tkutils::tkuaction
::tkutils::tkuaction::define zoom_in  -label "Vergrößern" -command
    {puts +}
::tkutils::tkuaction::define zoom_out -label "Verkleinern" -command
    {puts -}
::tkutils::tkuaction::groupDefine bild {zoom_in zoom_out}
::tkutils::tkuaction::groupSet bild 0
```

Fehlercodes: `{TKUTILS TKUACTION <GRUND>}` mit `OPTION`, `NOACTION`, `NOGROUP`, `WINDOW`. Eine unbekannte Option bei `define` meldet den Code `{TKUTILS TKUACTION OPTION -lable}` mit dem Optionsnamen und zwei Zeilen wie bei `tkuopts`:

```
unknown option "-lable"
Known: -accelerator -checkable -checked -command -compound -enabled
    -icon -iconChecked -label -tooltip
```

Die übrigen Meldungen: `unknown action 'zz'`, `unknown group 'zz'`, `unknown window '.nix'`. `getEnabled` für eine unbekannte Aktion liefert `0` statt eines Fehlers. Zum Stand bis 0.46.0: Anhang Fallen.

8.3 Gerüst

Leiste, Status, Reiter, Hinweis, Kontextmenü, Tasten, Fokus, Suche, Filter, Scrollen und der Auswahlrahmen stehen im Kapitel Gerüst. Formulare stehen im Kapitel Eingabe. Viewer für CSV, JSON und Diff stehen im Kapitel Daten ansehen.

8.4 tkulauncher 0.4

tkulauncher ist ein Programmstarter als Menü (`-mode menu`) oder Knopfliste (`-mode list`). Einträge: `app` (auch `command`), `url`, `open` (auch `file`), `tcl`, `system`, `menu`, `separator`, `calendar`, `calc`. URLs und Dateien öffnet `tclutils::tuopen` mit dem Standardprogramm des Systems. `app`- und `tcl`-Einträge kennen zusätzlich `dir` (Arbeitsverzeichnis) und `terminal 1` (im Terminalfenster starten).

Optionen von `widget`: `-mode` (Standard `list`), `-text` (Knopftext im Menümodus, Standard `Start`), `-items`, `-file` (Einträge aus `.json` oder `.ini`), `-onlaunch`, `-settings`, `-system`, `-scroll`, `-height` (Standard 400), `-tooltips`, `-columns` (Standard 1), `-editable`.

```
package require tkutils::tkulauncher
set spec {
    {type menu label Werkzeuge items {
        {type app label Editor cmd {xterm}}
        {type url label "Tcl-Handbuch" target
    https://www.tcl.tk/man/tcl9.0/}
    }}
    {type separator}
    {type tcl label "Meine App" target app.tcl interp wish8}
}
::tkutils::tkulauncher::widget .l -mode list -items $spec \
    -onlaunch {apply {{kind argv} {puts "$kind: $argv"; return 0}}}
pack .l -fill both -expand 1
```

`-onlaunch` bekommt `kind argv`; ein falscher Rückgabewert unterdrückt den Start — gut für Protokoll, Rückfrage oder Tests. `calendar`- und `calc`-Einträge öffnen eingebaute Fenster und laufen nicht über `-onlaunch`.

Mit `-scroll 1` liegt die Knopfliste in einem `tkuscrolledframe` und braucht damit scrollutil. Das Mausrad über den Knöpfen rollt die Liste, auch über Knöpfen, die `reload` neu anlegt; bis 0.2 fehlte ihnen diese Bindung. Eine unbekannte Option an `widget` meldet `unknown option "-foo"` mit der `Known:`-Liste aus `tkuopts`.

8.4.1 Start unter Windows

`app`-Einträge laufen unter Windows über `cmd /c start`, damit unter *App Paths* registrierte Programme ohne vollen Pfad starten; das Arbeitsverzeichnis geht über `start /D`. `start` liest ein `/` als Beginn eines Schalters. Seit 0.46.0 bekommen deshalb das Programm (erstes Wort)

und das `/D`-Verzeichnis **Backslashes**; die Argumente bleiben, wie sie sind (sie können URLs oder Optionen sein). Ein Programm mit `://` behält seine Schreibweise. Unter Linux/macOS wird das Programm direkt ausgeführt; ein `dir` wird dort per `cd` vor dem Start gesetzt.

8.4.2 Interpreter für `tcl`-Einträge

Ein `tcl`-Eintrag startet ein Skript mit `interp` (Standard `wish9`). Optional setzt `tcllibpath` die Variable `TCLLIBPATH` für den Kindprozess.

Schlüssel	Linux	Windows
wish9	wish9.0	wish90.exe
wish8	wish8.6	wish86t.exe
tclsh9	tclsh9.0	tclsh90.exe
tclsh8	tclsh8.6	tclsh86t.exe

Jede Plattform außer Windows benutzt die Spalte *Linux*, auch macOS.

```
package require tkutils::tkulauncher
puts [::tkutils::tkulauncher::tclInterpreters]                ;# ->
    wish9 wish8 tclsh9 tclsh8
puts [::tkutils::tkulauncher::resolveTclInterp wish8 windows] ;# ->
    wish86t.exe
::tkutils::tkulauncher::setTclInterp wish9 -linux
    /usr/local/tcl9/bin/wish9.0
puts [::tkutils::tkulauncher::resolve \
    {type tcl label Demo target app.tcl interp wish9 tcllibpath {lib}}]
```

`resolve` zeigt ohne Nebenwirkung, was gestartet würde (`kind`, `argv`, optional `env`, `dir`, `terminal`). Die letzte Zeile des Beispiels gibt `kind exec argv {/usr/local/tcl9/bin/wish9.0 app.tcl} env {TCLLIBPATH lib}` aus. Ein führendes `~` in Pfaden erweitert der Launcher selbst, weil Tcl 9 das nicht mehr tut. Ein unbekannter Interpreter-Schlüssel meldet `unknown interpreter: <key>` (ohne Fehlercode).

8.5 Zusammenfassung

- Actions zentral; Enable global oder pro Fenster (`setEnabled id 0 $w`).
- `invoke` folgt dem globalen Flag.
- Launcher: Windows-Start mit Backslashes, Interpreter-Mapping per `setTclInterp`.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux und Xvfb: alle Beispiele und Meldungen dieses Kapitels, `tkulauncher.test` (100 Tests).

Nicht getestet: Start von `app`-Einträgen unter Windows (nur im Code nachgelesen), Interpreter unter macOS.

Nächstes Kapitel: Gerüst

9 Gerüst

Aktionen und der Launcher stehen im Kapitel Aktionen und Layout. Hier liegen die Leisten,

Reiter, Hinweise und die kleinen Helfer um Fokus, Rad und Auswahlrahmen. Die Module gehören zum Umbrella, außer `tkuscrolledframe`.

9.1 Leiste: tkutoolbar

tkutoolbar 0.2 baut eine Leiste aus Knöpfen, Schaltern, Aufklappmenüs, Trennern und eingebetteten Widgets. Jedes Stück hat eine Id.

```
set tb [::tkutils::tkutoolbar::widget .tb \
    ?-orient horizontal|vertical? ?-displaymode icon|text|both? \
    ?-spacing px? ?-padding px? ?-buttonstyle flat|raised?]
::tkutils::tkutoolbar::addButton   $tb id label befehl ?-icon bild?
    ?-tooltip text? ?-shortcut taste?
::tkutils::tkutoolbar::addToggle   $tb id label varname ?-onvalue v?
    ?-offvalue v?
::tkutils::tkutoolbar::addDropdown $tb id label ?-menu {{label befehl}
    ...}?
::tkutils::tkutoolbar::addSeparator $tb
::tkutils::tkutoolbar::addWidget    $tb id kind ?seite?
```

`-buttonstyle` ist `flat`. `raised` gibt Knöpfen einen Rand. Schalter bleiben flach. `-shortcut` bindet auf das Toplevel und fällt weg, wenn die Leiste zerstört wird. Hinweise gehen über `tkuballoon`. Ohne Icon zeigt der Modus `icon` den Text.

```
::tkutils::tkutoolbar::setEnabled     $tb id 0|1
::tkutils::tkutoolbar::setDisplayMode $tb icon|text|both
::tkutils::tkutoolbar::buttonWidget   $tb id
::tkutils::tkutoolbar::items          $tb
```

Fehler `{TKUTILS TKUTOOLBAR <GRUND>}`: `OPTION`, `ORIENT`, `DISPLAYMODE`, `BUTTONSTYLE`, `NOITEM`, `NOTOOLBAR`.

9.2 Status: tkustatus

tkustatus 0.2 hat eine Hauptzeile, benannte Felder und einen Balken. Die Zugriffe heißen `setText` und `getText`, damit `set` frei bleibt.

```
set st [::tkutils::tkustatus::widget .st]
::tkutils::tkustatus::setText   $st text
::tkutils::tkustatus::getText   $st
::tkutils::tkustatus::addField  $st id ?-width n?
::tkutils::tkustatus::setField  $st id text
::tkutils::tkustatus::fieldText $st id
::tkutils::tkustatus::progress  $st ?0..100?
::tkutils::tkustatus::flash     $st text ?ms?
```

`progress` ohne Wert oder mit leerem Wert blendet den Balken aus. `flash` zeigt den Text und stellt danach die vorige Zeile wieder her.

9.3 Reiter: tkutab

tkutab 0.2 ist ein `ttk::notebook` mit einem Knopf "+" und schließbaren Reitern. `add` liefert den Rahmen des neuen Reiters. `current` liefert den Rahmen des aktiven, `tabs` die Rahmen in Reihenfolge, `count` die Zahl.

Option	Wirkung
-onnew	Skript beim "+"
-onclose	Skript, der schließende Rahmen wird angehängt
-onselect	Skript, der gewählte Rahmen wird angehängt

Schließen geht über `close` und über den mittleren Mausknopf.

9.4 Baum: tkutree

tkutree 0.2 ist ein dünner `ttk::treeview`. `tkufiletree` sitzt darauf. Das Dateifenster steht im Kapitel Datei-Widgets.

```
set w [::tkutils::tkutree::widget .tr ?-columns ids? ?-headings texte?
    \
    ?-show spec? ?-height n? ?-command befehl?]
::tkutils::tkutree::insert   $w eltern text ?-values liste? ?-open
    0|1? ?-id id?
::tkutils::tkutree::loadTree $w knoten
::tkutils::tkutree::selection $w
::tkutils::tkutree::itemText  $w id
::tkutils::tkutree::children  $w ?id?
::tkutils::tkutree::treeview  $w
```

9.5 Hinweis: tkuballoon

tkuballoon 0.1 hängt einen Hinweis an ein beliebiges Widget.

```
::tkutils::tkuballoon::add       $w text ?-delay ms? ?-wraplength px?
::tkutils::tkuballoon::clear     $w
::tkutils::tkuballoon::configure ?-delay ms? ?-wraplength px?
::tkutils::tkuballoon::cget      option
::tkutils::tkuballoon::enable
::tkutils::tkuballoon::disable
```

Die Vorgabe ist 600 ms und 320 Pixel Umbruch. `disable` schaltet alle Hinweise ab und blendet einen offenen aus.

9.6 Kontextmenü: tkucontextmenu

tkucontextmenu 0.1 baut Menüs und bindet sie an den Kontextknopf der Plattform: Knopf 3 unter X11 und Windows, Knopf 2 und Control-Klick unter macOS.

```
set m [::tkutils::tkucontextmenu::create .m ?-tearoff 0|1? ?-dynamic
    0|1?]
::tkutils::tkucontextmenu::addItem     $m label ?-command befehl?
    ?-accelerator text?
::tkutils::tkucontextmenu::addCheckItem $m label ?-variable v?
::tkutils::tkucontextmenu::addRadioItem $m label -variable v -value
    wert
::tkutils::tkucontextmenu::addSeparator $m
::tkutils::tkucontextmenu::addSubmenu   $m label $unter
::tkutils::tkucontextmenu::attach       $m $w ?-button 2|3?
::tkutils::tkucontextmenu::show         $m x y
```

`enable` und `disable` treffen einen Eintrag über sein Label. `createStandardEdit` legt die üblichen Bearbeiten-Einträge an. `createFromSpec` nimmt eine Liste von Einträgen.

9.7 Tasten: tkubind

tkubind 0.1 schreibt `Mod` als Control unter X11 und Windows und als Command unter macOS. Globale Kürzel bleiben stehen, solange ein Eingabefeld, ein Text oder eine Tablelist den Fokus hat.

```
set id [::tkutils::tkubind::key "Mod-s" {speichern} ?-toplevel .? ?-id
    name?]
::tkutils::tkubind::unbind $id
::tkutils::tkubind::accelerator "Mod-Shift-p"
::tkutils::tkubind::isEditing
::tkutils::tkubind::platform ?schlüssel?
::tkutils::tkubind::context     $w {menü %X %Y}
::tkutils::tkubind::doubleClick $w {öffnen %W}
```

`accelerator` liefert unter X11 und Windows `Ctrl+Shift+P`, unter macOS `@^P`. Gruppen schalten mehrere Kürzel gemeinsam:

```
::tkutils::tkubind::group define editKeys {{Mod-b {fett}} {Mod-i
    {kursiv}}}
::tkutils::tkubind::group enable  editKeys ?-toplevel .?
::tkutils::tkubind::group enabled editKeys
```

`clear` löst alle Bindungen und Gruppen.

9.8 Fokus: tkukeynav

tkukeynav 0.1 bewegt den Fokus und bindet Return und Escape.

```
::tkutils::tkukeynav::next $w
::tkutils::tkukeynav::prev $w
::tkutils::tkukeynav::enable $w ?-onreturn befehl? ?-onescape befehl?
    ?-tab 0|1?
::tkutils::tkukeynav::disable $w
::tkutils::tkukeynav::form $rahmen ?-onsubmit befehl? ?-wrap 0|1?
    ?-onescape befehl?
```

`next` und `prev` liefern das Widget, das den Fokus bekommt. `form` bindet die Eingabefelder im Rahmen.

9.9 Suche und Filter

tkusearchbar 0.1 ist ein Feld mit Löschen-Knopf und optionaler Filterliste. Tippen ruft `-command` nach `-delay` Millisekunden Ruhe. Vorgabe 300, Breite 28. Löschen und ein Filterwechsel feuern sofort. `setText` setzt den Text und feuert nicht.

```
set s [::tkutils::tkusearchbar::widget .s ?-command befehl? ?-delay
    ms? ?-filters liste?]
::tkutils::tkusearchbar::getText $s
::tkutils::tkusearchbar::clear   $s
::tkutils::tkusearchbar::getFilter $s
```

tkufilterbar 0.2 legt je Spalte ein Feld an. `-command` bekommt ein Dict der nicht leeren Felder. Vorgabe der Pause 300 ms, Feldbreite 10. `setFilter` feuert nicht, `clear` schon.

```
set f [::tkutils::tkufilterbar::widget .f ?-command befehl?]
::tkutils::tkufilterbar::setColumns $f {name ort}
::tkutils::tkufilterbar::getFilters $f
```

9.10 Scrollen

tkuscrolledframe 0.1 braucht `scrollutil` aus tklib und steht außerhalb des Umbrellas. `widget` liefert den Außenrahmen, `content` den Rahmen zum Packen.

```
set sf [::tkutils::tkuscrolledframe::widget .sf ?-width n? ?-height n?]
set innen [::tkutils::tkuscrolledframe::content $sf]
::tkutils::tkuscrolledframe::see $sf $kind ?ecke?
```

tkuwheel 0.2 leitet das Mausrad von einem Widget an ein scrollbares Ziel.

```
::tkutils::tkuwheel::redirect ziel $w ?-orient y|x|both? ?-amount n?
    ?-dynamic 0|1?
::tkutils::tkuwheel::unbind   $w ?-recursive 0|1?
```

`-dynamic 1` trifft auch Widgets, die später in `$w` entstehen.

9.11 Rahmen aufziehen: tkumarquee

tkumarquee 0.1 zieht auf einer Zeichenfläche ein Rechteck.

```
::tkutils::tkumarquee::enable  .c ?-onselect befehl? ?-onstart befehl?
    ?-button n?
::tkutils::tkumarquee::disable .c
::tkutils::tkumarquee::region  .c
::tkutils::tkumarquee::active  .c
```

`region` liefert `{x1 y1 x2 y2}` oder den leeren String. `active` ist `1`, solange die Taste gehalten ist.

9.12 Merkliste

- `tkutoolbar` spricht Widgets über Ids an. Hinweise kommen von `tkuballoon`.
- `tkustatus` heißt `setText`, nicht `set`.
- `tkutab::add` liefert den Inhaltsrahmen.
- Mod in `tkubind` ist Control oder Command, je nach Plattform.
- `tkuscrolledframe` braucht `scrollutil` und ein eigenes `package require`.

Nächstes Kapitel: Eingabe

10 Eingabe

Formulare, einzelne beschriftete Felder und die spezialisierten Eingaben für Datum, Uhrzeit, Zahl und Schlagwörter. Die Prüfung hängt an `tclutils::tuvalidate`. Alle Module dieses Kapitels liegen im Umbrella.

10.1 Formular: tkuform

tkuform 0.1 baut aus einer Liste von Feld-Dicts ein Formular und liest die Werte als Dict. `tkudialog::form` nutzt dasselbe Modul. Der Dialog steht im Kapitel Dialoge.

Ein Feld braucht `name`. Dazu `label`, `type`, `default`, `values` bei `combo`, `state` bei `combo` (`normal` oder `readonly`), `from` und `to` bei `spin`, `height` bei `text`. Der Typ ist `entry`, `password`, `check`, `combo`, `spin` oder `text`, Vorgabe `entry`.

```
set w [::tkutils::tkuform::widget .w $felder ?-padding 8? ?-columns 1?]
::tkutils::tkuform::values    $w
::tkutils::tkuform::get       $w name
::tkutils::tkuform::setField  $w name wert
::tkutils::tkuform::setValues $w dict
::tkutils::tkuform::fieldNames $w
::tkutils::tkuform::widgetOf  $w name
```

`values` liefert ein Dict von Name zu Wert. `setValues` übergeht unbekannte Schlüssel. `setField` heißt so, damit `set` frei bleibt. `-columns` legt mehrere Paare aus Beschriftung und Feld nebeneinander. Ein Feld vom Typ `text` nimmt immer eine eigene Zeile über die volle Breite.

Code	Anlass
{TKUTILS TKUFORM OPTION}	unbekannte Widget-Option

Code	Anlass
{TKUTILS TKUFORM VALUE}	-columns unbrauchbar
{TKUTILS TKUFORM SPEC}	Feld ohne name
{TKUTILS TKUFORM TYPE}	unbekannter Feldtyp

10.2 Eine Zeile: tkulabeled

tkulabeled 0.1 ist eine Zeile aus Beschriftung und Steuerelement, wenn ein ganzes Formular zu viel ist. Der Typ ist ein Wert: `entry`, `combo`, `spin`, `check` oder `text`.

```
::tkutils::tkulabeled::add $pfad $typ \
    ?-label text? ?-labelwidth n? ?-orient horizontal|vertical?
    ?weitere Optionen?
::tkutils::tkulabeled::value       $pfad ?neu?
::tkutils::tkulabeled::control     $pfad
::tkutils::tkulabeled::labelwidget $pfad
```

Ein Argument liest den Wert, zwei setzen ihn. Optionen außer `-label`, `-labelwidth` und `-orient` gehen an das innere Steuerelement.

10.3 Prüfung: tkuvalidate

tkuvalidate 0.1 hängt eine Prüfung an Entry, Combo oder Spinbox. Ein ungültiger Wert färbt den Text rot, setzt den ttk-Zustand `invalid` und zeigt den Grund über `tkuballoon`.

```
::tkutils::tkuvalidate::attach $w $prüfer \
    ?-message text? ?-when focusout|key|both? ?-allowempty 0|1? \
    ?-onvalid befehl? ?-oninvalid befehl?
::tkutils::tkuvalidate::check    $w
::tkutils::tkuvalidate::valid    $w
::tkutils::tkuvalidate::clear    $w
::tkutils::tkuvalidate::allValid {w ...}
```

`$prüfer` ist ein Name aus `tclutils::tuvalidate` (`email`, `url`, `ipv4`, `port`, `alpha`, `alnum`, `numeric`, `integer`) oder ein Befehlspräfix. Der Wert des Feldes wird angehängt. Der Präfix liefert einen Wahrheitswert. `check` und `allValid` liefern `0` oder `1`. `clear` nimmt Markierung und Bindung weg.

10.4 Datum: tkudateentry

tkudateentry 0.1 ist ein Feld plus ein Monatskalender zum Aufklappen. Der Montag steht vorn. `getDate` liefert immer `jjjj-mm-tt` oder `""`, wenn das Feld leer oder ungültig ist. Die Anzeige folgt `-dateformat`.

```
set w [::tkutils::tkudateentry::widget .d \
    ?-dateformat fmt? ?-width n? ?-textvariable var? ?-command befehl?]
::tkutils::tkudateentry::getText $w
::tkutils::tkudateentry::getDate $w
::tkutils::tkudateentry::setDate $w iso
::tkutils::tkudateentry::today   $w
::tkutils::tkudateentry::clear   $w
```

`-command` bekommt das ISO-Datum, beim Löschen den leeren String. `setDate` mit einem Text, der kein ISO-Datum ist, wirft `{TKUTILS TKDATEENTRY DATE}`. Ein leeres `setDate` leert das Feld.

Der Monat zum Anklicken ohne Eingabezeile ist `tkucalendar` im Kapitel Kalender und Notizen.

10.5 Uhrzeit: tkutimeentry

tkutimeentry 0.1 zeigt Stunde und Minute als Spinbox, mit `-seconds` auch die Sekunde. `getTime` und `setTime` nutzen `HH:MM` oder `HH:MM:SS`.

```
set w [::tkutils::tkutimeentry::widget .t \
    ?-time HH:MM? ?-increment min? ?-seconds 0|1? ?-command befehl?]
::tkutils::tkutimeentry::getTime $w
::tkutils::tkutimeentry::setTime $w 09:30
```

`-increment` ist der Minutenschritt, Vorgabe 5. `-command` bekommt die Zeit bei jeder Änderung. Ein unbrauchbares `setTime` wirft `{TKUTILS TKTIMEENTRY TIME}`.

10.6 Zahl: tkunumentry

tkunumentry 0.1 lässt nur eine Zahl mit höchstens `-decimals` Nachkommastellen tippen. Klemmen und Neuformatieren laufen beim Commit, also Return oder Fokusverlust.

```
set w [::tkutils::tkunumentry::widget .n \
    ?-decimals n? ?-min v? ?-max v? ?-textvariable var? ?-width n?
    ?-command befehl?]
::tkutils::tkunumentry::getValue $w
::tkutils::tkunumentry::setValue $w 3.14
::tkutils::tkunumentry::clear    $w
```

`getValue` liefert die Zahl oder `""`, wenn das Feld leer ist. `setValue` mit einem Text, der keine Zahl ist, wirft `{TKUTILS TKNUMENTRY VALUE}`. `-command` bekommt den Wert beim Commit.

10.7 Schlagwörter: tkutags

tkutags 0.1 zeigt Schlagwörter als entfernbare Plättchen. Return oder "+" fügt eines hinzu. Doppelte und leere entfallen. Mit `-readonly` fehlen Eingabe und Entfernen-Knopf. `-suggestions` macht aus dem Feld eine Combobox.

```
set w [::tkutils::tkutags::widget .t \
    ?-tags liste? ?-suggestions liste? ?-textvariable var? ?-readonly
    0|1? ?-command befehl?]
::tkutils::tkutags::getTags   $w
::tkutils::tkutags::setTags   $w liste
::tkutils::tkutags::addTag    $w tag
::tkutils::tkutags::removeTag $w tag
::tkutils::tkutags::clear     $w
```

`-textvariable` spiegelt die Liste. Ist `-tags` leer, setzt die Variable den Anfang. `-command` bekommt die Liste bei jeder Änderung.

10.8 Merkliste

- `tkuform::values` ist ein Dict. `setField` setzt ein Feld.
- `tkuvalidate` nimmt die Namen aus `tclutils::tuvalidate` oder einen eigenen Befehl.
- `tkudateentry::getDate` ist ISO, die Anzeige folgt `-dateformat`.
- `tkunumentry` formatiert beim Commit, nicht bei jedem Tastendruck.

Nächstes Kapitel: Kalender und Notizen

11 Kalender und Notizen

Monat, Woche und Aufgaben in der Oberfläche. Die Engines bleiben in tclutils (`tucal`, `tuical`, `tunotes`, `tuvcard`, `tuldif`). `tkutical` und `tkumonthcanvas` brauchen zusätzlich tical und stehen außerhalb des Umbrellas. Wie tical auf den `auto_path` kommt, steht im Kapitel tkutils verstehen; tical selbst, seine Views und Renderer beschreibt das Buch *Utils-Stack*.

11.1 Monat zum Anklicken: tkucalendar

tkucalendar 0.2 ist ein Monat aus Tagesknöpfen.

```
::tkutils::tkucalendar::widget       pfad ?option wert ...?
::tkutils::tkucalendar::getDate      pfad
::tkutils::tkucalendar::getFormatted pfad
::tkutils::tkucalendar::setDate      pfad iso
::tkutils::tkucalendar::today        pfad
::tkutils::tkucalendar::next         pfad
::tkutils::tkucalendar::prev         pfad
::tkutils::tkucalendar::getMonth     pfad
```

`getDate` liefert das gewählte ISO-Datum oder `""`. `getFormatted` nutzt `-dateformat`. `getMonth` liefert `jjjj mm` des angezeigten Monats. `next` und `prev` blättern einen Monat.

Option	Vorgabe	Wirkung
-date	keines	anfangs gewählter Tag, ISO
-firstday	monday	monday oder sunday

Option	Vorgabe	Wirkung
-dateformat	%Y-%m-%d	Format für getFormatted
-locale	die aktuelle	Monats- und Wochentagsnamen
-command	leer	Skript, das ISO-Datum wird angehängt

Die Eingabezeile mit demselben Kalender als Aufklappfenster ist `tkudateentry` im Kapitel Eingabe.

11.2 Textkalender: tkucal

tkucal 0.1 zeigt den Monat als Text über `tclutils::tucal`.

```
set w [::tkutils::tkucal::widget .w ?-year j? ?-month m? ?-weeknumbers
    0|1?]
::tkutils::tkucal::setMonth $w jahr monat
::tkutils::tkucal::getMonth $w
::tkutils::tkucal::next     $w
::tkutils::tkucal::prev     $w
::tkutils::tkucal::today    $w
::tkutils::tkucal::getText  $w
```

`setMonth` und `getMonth` liefern `{jahr monat}`. Ein ungültiger Monat wirft den Fehler von `tucal`, etwa `{TCLUTILS TUCAL MONTH}` mit `invalid month 13`. Die Woche beginnt am Sonntag, wie bei `cal`. `getText` ist der gezeichnete Kalendertext.

11.3 tical: tkutical und tkumonthcanvas

tkutical 0.4 zeichnet Monat oder Woche über tical.

```
set w [::tkutils::tkutical::widget .w ?-year j? ?-month m? \
    ?-weeknumbers 0|1? ?-fontsize n? ?-holidays region? \
    ?-selectmode none|single|multiple? ?-command befehl? ?-events
    liste?]
::tkutils::tkutical::setMonth $w jahr monat
::tkutils::tkutical::getSelection $w
::tkutils::tkutical::setSelection $w {daten und A..B}
```

`setMonth` liefert `{jahr monat}` und wirft bei einem ungültigen Monat. `getSelection` liefert sortierte ISO-Daten. `-events` und `setEvents` (seit 0.4) nehmen Termine im Format von `tical::event`; sie erscheinen als Punkte im Monat und als Titel in der Woche, eine Serie an jedem ihrer Tage. Das braucht tical 1.1 oder neuer, sonst `{TKUTILS TKUTICAL EVENTS}`.

tkumonthcanvas 0.5 zeichnet Monat, Quartal und Jahr auf eine bestehende Zeichenfläche. `init` setzt Schrift und Thema, bevor gezeichnet wird.

```
::tkutils::tkumonthcanvas::init ?-fontsize 11? ?-font Arial?
::tkutils::tkumonthcanvas::drawMonth   $c jahr monat ?x0 y0?
::tkutils::tkumonthcanvas::drawQuarter $c jahr monat ?x0 y0?
::tkutils::tkumonthcanvas::drawYear    $c jahr ?x0 y0 spalten?
::tkutils::tkumonthcanvas::clear       $c
::tkutils::tkumonthcanvas::setSelectMode none|single|multiple
::tkutils::tkumonthcanvas::getSelection
::tkutils::tkumonthcanvas::setTheme default|dark|light
::tkutils::tkumonthcanvas::addNote datum text
```

`drawQuarter` nimmt irgendeinen Monat des Quartals. `setCallback` kennt `day`, `week`, `month` und `select`. Themen sind `default`, `dark` und `light`, dazu `defineTheme`.

11.4 Termine: tkuical

tkuical 0.3 zeigt die VEVENT-Einträge eines iCalendar-Textes in einer Tabelle: Summary, Start, Ende, Ort. Die Engine ist `tclutils::tuical`.

```
set w [::tkutils::tkuical::widget .w]
::tkutils::tkuical::loadText      $w text
::tkutils::tkuical::loadFile      $w datei
::tkutils::tkuical::setComponents $w komponenten
::tkutils::tkuical::events        $w
::tkutils::tkuical::count         $w
::tkutils::tkuical::toText        $w
::tkutils::tkuical::save          $w datei
```

`loadText` liefert die Zahl der Termine. `-editable 1` ist die Vorgabe und zeigt eine Leiste für Summary, Start, Ende und Ort. `addEvent`, `removeEvent` und `setField` ändern einen Termin. `setField` kennt `summary`, `start`, `end` und `location`. Andere Eigenschaften und Komponenten, die keine Termine sind, bleiben beim Schreiben erhalten, an ihrer Stelle: Eine Aufgabe nach dem Termin steht danach wieder dort. `loadFile` und `save` arbeiten in UTF-8, gleich welches Encoding das System hat; `save` schreibt die Zeilenenden CRLF einmal (bis 0.2 `\r\r\n`). Mit `tuical` 0.2 endet auch die letzte Zeile mit CRLF; eine Datei, die ohne Änderung gelesen und geschrieben wird, bleibt dann bis auf die Faltung langer Zeilen gleich.

11.5 Aufgaben: tkutodo

tkutodo 0.2 zeigt VTODO-Komponenten: Summary, Fälligkeit, Priorität, Prozent. `-editable 1` lässt den Status umschalten, auch mit Leertaste und Doppelklick.

```
set w [::tkutils::tkutodo::widget .t ?-editable 1? ?-onchange befehl?]
::tkutils::tkutodo::loadText $w text
::tkutils::tkutodo::setTodos $w komponenten
::tkutils::tkutodo::todos    $w
::tkutils::tkutodo::count    $w
::tkutils::tkutodo::toggle   $w ?index?
```

Erledigt setzt `STATUS` auf `COMPLETED`, `PERCENT-COMPLETE` auf `100` und einen Zeitstempel

COMPLETED. Zurück setzt NEEDS-ACTION, PERCENT-COMPLETE auf 0 und entfernt COMPLETED.

11.6 Notizen: tkunotes

tkunotes 0.4 ist ein Baum links und ein Editor rechts. Speichern und Suche kommen von tclutils::tunotes.

```
set w [::tkutils::tkunotes::widget .w ?-file notizen.json? ?-toolbar
    0?]
::tkutils::tkunotes::addRoot  $w titel inhalt ?tags?
::tkutils::tkunotes::addChild $w eltern titel inhalt ?tags?
::tkutils::tkunotes::commit   $w ?id?
::tkutils::tkunotes::delete   $w id ?cascade?
::tkutils::tkunotes::search   $w text
::tkutils::tkunotes::load     $w datei
::tkutils::tkunotes::save     $w ?datei?
::tkutils::tkunotes::store    $w
```

addRoot liefert die Id. search liefert Ids. load liefert die Zahl. commit schreibt die Felder des Editors in den Speicher. store ist das Dict von tunotes. Die Leiste ist an; -toolbar 0 blendet sie aus. Ihr Knopf "Save" übernimmt die Felder und schreibt seit 0.3 auch die JSON-Datei (-file, die zuletzt geladene oder gespeicherte, sonst fragt er); bis 0.2 tat er nur commit. Im Feld "Tags" trennt ein Komma die Tags, Arbeit Büro, privat sind zwei (bis 0.2 war jedes Wort ein Tag). Seit 0.4 bleiben Änderungen erhalten, wenn eine andere Notiz gewählt wird (Klick im Baum, select, neue Notiz, Suchtreffer): Titel, Tags und Inhalt der bisherigen gehen in den Speicher. Bis 0.3 waren sie ohne "Save" verloren. Die Datei schreibt weiter nur "Save" oder save.

11.7 Kontakte und LDIF

tkuvcard 0.3 zeigt jede Karte als Knoten mit FN und die Eigenschaften als Kinder. Die Engine ist tclutils::tuvcard.

```
set w [::tkutils::tkuvcard::widget .w]
::tkutils::tkuvcard::loadText $w text
::tkutils::tkuvcard::cards    $w
::tkutils::tkuvcard::count    $w
```

loadText liefert die Zahl der Karten. -editable 1 ist die Vorgabe. setProperty ändert Name, Wert und TYPE einer Zeile und behält die übrigen Parameter wie PREF (bis 0.2 gingen sie verloren); mehrere Typen stehen durch Komma getrennt ("CELL, VOICE"). Dateien liest und schreibt tkuvcard in UTF-8 mit CRLF.

tkuldif 0.3 zeigt jeden LDIF-Eintrag als Knoten mit dn und jede Attributzeile als Kind. Die Engine ist tclutils::tuldif. loadText liefert die Zahl der Einträge. -editable 1 ist die Vorgabe. entries liefert die Einträge. Dateien in UTF-8. Mit tuldif 0.3 behalten Binärattribute wie jpegPhoto beim Speichern ihre Bytes.

11.8 Rechner: tkucalc

tkucalc 0.2 ist ein Taschenrechner. `evaluate` rechnet ohne Fenster, das Paket fordert trotzdem Tk an.

```
::tkutils::tkucalc::widget       pfad ?-onresult befehl? ?-history 0?
::tkutils::tkucalc::evaluate     text
::tkutils::tkucalc::clear        pfad
::tkutils::tkucalc::getHistory   pfad
::tkutils::tkucalc::clearHistory pfad
```

`evaluate` ersetzt das deutsche Dezimalkomma durch einen Punkt, `^` durch `**` und die Zeichen × ÷ − durch `* / -`. Außer Ziffern, Operatoren, Klammern, Punkt, `e`, Leerzeichen, `^` und `%` wirft der Text `invalid characters in expression`. Ein leerer Text wirft `empty expression`. Beide ohne eigenen `-errorcode`.

Ohne `%` wird jede ganze Zahl zu `n.0`, damit `10/4` eine Gleitkommadivision ist. Mit `%` bleibt die Zahl ganz, weil Modulo ganze Operanden braucht. Die Rechnung läuft in einem sicheren Interpreter. `Inf`, `-Inf` und `NaN` werden zu `divide by zero`. Ein Ergebnis mit Endung `.0` verliert diese Endung.

`-history 1` zeigt die bisherigen Zeilen. `-onresult` bekommt das Ergebnis.

11.9 Merkliste

- `tkucalendar::getDate` ist ISO. `getFormatted` folgt `-dateformat`.
- `tkutical` und `tkumonthcanvas` brauchen tical und ein eigenes `package require`.
- `tkuical` und `tkutodo` lesen iCalendar über `tuical`.
- `tkucalc::evaluate` rechnet in einem sicheren Interpreter.

Nächstes Kapitel: Daten ansehen

12 Daten ansehen

Die Widgets dieses Kapitels zeigen, was die Engines in tclutils schon lesen: CSV, JSON, XML, INI, Markdown, SQLite, Diff, Hex, Zeichenketten, Base64, unscharfe Suche, ZIP und PDF-Struktur. Dazu der DAV-Browser, der Texteditor und die Systemübersicht. Die API der Engine steht im Handbuch tclutils.

`tkuxml`, `tkusqlite`, `tkusysinfo` und der Editor laden extra. Die übrigen Viewer dieses Kapitels liegen im Umbrella, soweit das Kapitel es nicht anders sagt.

12.1 CSV: tkucsv

tkucsv 0.1 zeigt eine Tabelle über `tclutils::tucsv`.

```
set w [::tkutils::tkucsv::widget .w ?-height n? ?-header 0|1?]
::tkutils::tkucsv::setData  $w text ?option wert ...?
::tkutils::tkucsv::loadFile $w pfad ?option wert ...?
::tkutils::tkucsv::getRows  $w
```

Optionen von `setData` und `loadFile` gehen an `tucsv::parse`. `getRows` liefert alle

Zeilen, die Kopfzeile eingeschlossen.

12.2 JSON: tkujson

tkujson 0.1 zeigt den Baum von `tujson::parseTyped`. Objekte, Arrays und Skalare bleiben unterschieden.

```
set w [::tkutils::tkujson::widget .w ?-height n?]
::tkutils::tkujson::setJson  $w text
::tkutils::tkujson::loadFile $w pfad
::tkutils::tkujson::getTree  $w
```

`setJson` liefert den Typ der Wurzel. `getTree` liefert `{typ wert}`. Ein Objekt erscheint als `object (n)`, ein Array als `array (n)`. Schlüssel behalten ihre Reihenfolge. Array-Elemente heißen `[0]`, `[1]` und so fort.

12.3 XML: tkuxml

tkuxml 0.1 braucht tDOM und steht außerhalb des Umbrellas.

```
set w [::tkutils::tkuxml::widget .w ?-height n?]
::tkutils::tkuxml::setXml   $w text
::tkutils::tkuxml::loadFile $w pfad
::tkutils::tkuxml::getRoot  $w
```

`setXml` liefert den Namen des Wurzelelements. Der Baum zeigt Elementnamen, Attribute und Text.

12.4 INI: tkuini

tkuini 0.3 zeigt Abschnitte links und die Paare des gewählten Abschnitts rechts. Der Abschnitt `""` heißt `(global)`. Die Engine ist `tclutils::tuini`.

```
set w [::tkutils::tkuini::widget .w]
::tkutils::tkuini::loadText       $w text
::tkutils::tkuini::loadFile       $w datei
::tkutils::tkuini::sections       $w
::tkutils::tkuini::selectSection  $w abschnitt
::tkutils::tkuini::currentSection $w
::tkutils::tkuini::data           $w
```

`loadText` liefert die Zahl der Abschnitte. `loadFile` und `save` arbeiten in UTF-8, gleich welches Encoding das System hat (seit 0.3).

Mit `tuini` 0.2 werfen `setKey` und `addSection` `{TCLUTILS TUINI VALUE}`, wenn INI den Wert nicht fassen kann, etwa einen Zeilenumbruch; die Daten bleiben dann unverändert. Die Bearbeitungsleiste schneidet Leerzeichen am Rand von Schlüssel und Abschnitt ab und zeigt einen solchen Fehler als Meldung.

12.5 Markdown: tkumd und tkumdview

tkumd 0.1 zeigt die Überschriften als Baum über `tclutils::tumd`.

```
set w [::tkutils::tkumd::widget .w ?-height n?]
::tkutils::tkumd::setMarkdown $w text
::tkutils::tkumd::loadFile    $w pfad
::tkutils::tkumd::getHeadings $w
```

`getHeadings` liefert eine Liste `{stufe titel}`.

tkumdview 0.1 zeigt dieselbe Gliederung und eine Vorschau.

```
set w [::tkutils::tkumdview::widget .w ?-width n? ?-height n?]
::tkutils::tkumdview::loadFile    $w pfad
::tkutils::tkumdview::setMarkdown $w text
::tkutils::tkumdview::getMarkdown $w
::tkutils::tkumdview::headings    $w
::tkutils::tkumdview::toHtml      $w
```

`loadFile` liest UTF-8. `loadFile` und `setMarkdown` liefern die Zeichenzahl. `tocWidget` und `textWidget` liefern die beiden inneren Widgets.

12.6 SQLite: tkusqlite

tkusqlite 0.1 braucht das Paket `sqlite3` und steht außerhalb des Umbrellas. Links stehen die Tabellen, rechts die Zeilen der gewählten Tabelle. `tclutils::tusqlite` ist die Engine ohne Oberfläche.

```
set w [::tkutils::tkusqlite::widget .w ?-height n?]
::tkutils::tkusqlite::openFile  $w datei
::tkutils::tkusqlite::tables    $w
::tkutils::tkusqlite::showTable $w name
::tkutils::tkusqlite::getRows   $w
::tkutils::tkusqlite::closeDb   $w
```

`openFile` nimmt auch `:memory:` und liefert die Zahl der Tabellen. `showTable` liefert die Zahl der Zeilen.

12.7 Diff, Hex, Zeichenketten, Base64

tkudiff 0.1 zeigt einen Zeilenvergleich über `tclutils::tudiff`.

```
set w [::tkutils::tkudiff::widget .w ?-width n? ?-height n?]
::tkutils::tkudiff::setTexts  $w alt neu ?option wert ...?
::tkutils::tkudiff::loadFiles $w alt neu ?option wert ...?
::tkutils::tkudiff::getOps    $w
```

Optionen gehen an `tudiff::text`. `getOps` liefert eine Liste `{op token}`.

tkuhexedit 0.2 zeigt und ändert Bytes über `tuhexdump` und `tubin`. `saveFile` schreibt über eine temporäre Datei; scheitert das Schreiben, bleibt die alte Datei stehen.

```
set w [::tkutils::tkuhexedit::widget .hex]
::tkutils::tkuhexedit::loadFile   $w datei
::tkutils::tkuhexedit::gotoOffset $w 0x100
::tkutils::tkuhexedit::findHex    $w {50 4B 03 04}
::tkutils::tkuhexedit::patchHex   $w 0x100 {00 FF}
::tkutils::tkuhexedit::saveFile   $w
```

tkustrings 0.1 listet lesbare ASCII-Folgen über `tclutils::tustrings`. `setData` und `loadFile` nehmen `-minlength` und liefern die Zahl. `getStrings` liefert die Folgen.

tkubase64 0.2 kodiert und dekodiert über `tclutils::tubase64`. `encode` und `decode` liefern den Ausgabetext. `setInput`, `getInput` und `getOutput` lesen und schreiben die beiden Felder. Die Felder halten Text: `encode` wandelt ihn zuerst nach UTF-8, `decode` liest die Bytes als UTF-8, so wird `Jörg €` zu `SsO2cmcg4oKs` und kommt unverändert zurück. Bis 0.1 brach `€` unter Tcl 9 ab, und `ö` wurde ein Byte.

12.8 Unscharfe Liste: tkufuzzy

tkufuzzy 0.1 filtert eine Liste über `tclutils::tufuzzy`.

```
set w [::tkutils::tkufuzzy::widget .w ?-height n?]
::tkutils::tkufuzzy::setItems     $w liste
::tkutils::tkufuzzy::filter       $w muster
::tkutils::tkufuzzy::getMatches   $w
::tkutils::tkufuzzy::getSelection $w
```

`filter` liefert die Treffer nach Rang und setzt die angezeigte Liste. `getSelection` liefert den gewählten Eintrag oder `""`.

12.9 ZIP: tkuzip

tkuzip 0.1 listet die Mitglieder eines Archivs über `tclutils::tuzip`: Name, Größe ungepackt, Größe gepackt, Methode.

```
set w [::tkutils::tkuzip::widget .w ?-height n?]
::tkutils::tkuzip::openFile       $w archiv
::tkutils::tkuzip::getEntries     $w
::tkutils::tkuzip::selectedMember $w
```

`openFile` liefert die Zahl der Mitglieder. `getEntries` liefert die Dicts von `tuzip::entries`. `selectedMember` liefert den Namen oder `""`.

12.10 PDF-Struktur: tkupdfinspect

tkupdfinspect 0.1 zeigt den Aufbau einer PDF-Datei über `tclutils::tupdf`. Der Text der Seiten ist eine andere Aufgabe (`tupdftext` im Handbuch tclutils).

```
set w [::tkutils::tkupdfinspect::widget .w ?-width n? ?-height n?]
::tkutils::tkupdfinspect::loadFile    $w pfad
::tkutils::tkupdfinspect::currentFile $w
::tkutils::tkupdfinspect::summary     $w
```

`loadFile` füllt den Baum und liefert den Pfad. `summary` ist das Dict des letzten Ladens. `currentFile` ist `""`, solange nichts geladen ist. `treeWidget` und `textWidget` liefern Baum und Detailtext. Das Detail ist nur lesbar.

12.11 DAV

tkudavaccount 0.1 ist das Formular für URL, Benutzer, Passwort und Art (`carddav`, `caldav`, `webdav`). Der Prüfknopf macht ein PROPFIND über `tclutils::tudav` und blockiert so lange.

```
set a [::tkutils::tkudavaccount::widget .a ?-type carddav? ?-command
    befehl?]
::tkutils::tkudavaccount::setConfig $a {url https://h/c/ user alice
    password pw}
::tkutils::tkudavaccount::testConnection $a
```

`testConnection` liefert `1` oder `0`. `-command` bekommt `ok` und eine Meldung.

tkudavbrowser 0.2 listet Sammlungen eines CalDAV- oder CardDAV-Servers, gruppiert nach Art. Er ist nur lesend, außer mit `-editable 1`.

```
set b [::tkutils::tkudavbrowser::widget .b ?-oncollection befehl?]
::tkutils::tkudavbrowser::setClient .b $client
::tkutils::tkudavbrowser::refresh    .b
::tkutils::tkudavbrowser::setData    .b $sammlungen
```

`refresh` ruft `discover` und `listCollections` und blockiert. Dieselbe Href erscheint einmal, auch wenn ein Server beide Arten aus einem Home-Set liefert. `setData` baut den Baum aus fertigen Dicts `{href displayname kind}` ohne Netz. `-oncollection` bekommt das Widget und das Dict der Auswahl. `collections` liefert die flache Liste, `selected` das Dict oder `{}`.

12.12 Editor: tkueditor

tkueditor 0.4 ist ein Texteditor mit Leiste und Statuszeile. Er fordert `tkuicon`, `tkutoolbar` und `tkustatus` an, dazu `tclutils::tuiconv`.

```
set w [::tkutils::tkueditor::widget .w ?-width n? ?-height n? ?-wrap
    modus?]
::tkutils::tkueditor::setText    $w text
::tkutils::tkueditor::getText    $w
::tkutils::tkueditor::loadFile   $w pfad ?-encoding name?
::tkutils::tkueditor::saveFile   $w ?pfad? ?-encoding name? ?-eol
    lf|crlf|cr?
::tkutils::tkueditor::find       $w nadel ?-from index? ?-nocase?
::tkutils::tkueditor::replace    $w nadel ersatz ?-all? ?-nocase?
::tkutils::tkueditor::gotoLine   $w n
::tkutils::tkueditor::isModified $w
```

`setText` löscht die Markierung "geändert". `find` liefert einen Index oder `""`. `replace` liefert die Zahl. `readonly` sperrt die Eingabe. `setText` bleibt dabei möglich. `statusbarWidget` liefert die `tkustatus`-Leiste oder `""`, wenn sie aus ist.

Zeilenenden erkennt `loadFile` als `lf`, `crlf`, `cr` oder `mixed` (mehrere Arten in einer Datei); `eol` liefert den Stil. Eine gemischte Datei, unverändert gespeichert, bleibt Byte für Byte gleich; nach einer Änderung schreibt `saveFile` die häufigste Art und sagt das in der Statuszeile. Bis 0.2 machte ein einziges CRLF die ganze Datei zu CRLF, ein einzelnes CR wurde LF.

Die Kodierung läuft über `tclutils::tuiconv`. Mit `tuiconv` 0.2 ist sie auch unter Tcl 8.6 streng: `loadFile` wirft `{TCLUTILS TUICONV INVALID enc}`, wenn die Datei in der Kodierung ungültig ist, `saveFile` wirft `{TCLUTILS TUICONV UNMAPPABLE enc}`, wenn ein Zeichen darin nicht darstellbar ist; die Datei bleibt dann, wie sie war. Die Knöpfe "Open" und "Save" der Leiste fangen diese Fehler ab, ebenso eine Datei, die sich nicht lesen oder schreiben lässt: Ein Meldungsfenster und die Statuszeile nennen Datei und Grund, der Text im Editor bleibt (seit 0.4; bis 0.3 ging der Fehler an `bgerror`).

12.13 System: tkusysinfo

tkusysinfo 0.1 zeigt, was `tclutils::tusysinfo` über den Rechner sammelt: Betriebssystem, Prozessor, Speicher, Modell, Festplatten und Monitore, dazu einen Abschnitt "Screen" aus Tk selbst (Pixel, Millimeter, DPI, `tk scaling`, Farbtiefe, Fenstersystem). Knöpfe: *Refresh*, *Copy* (Textbericht in die Zwischenablage), *Save...* (Text oder JSON). Nicht im Umbrella, weil es `tusysinfo` aus tclutils 0.69.0 braucht.

```
set w [::tkutils::tkusysinfo::widget .si ?option wert ...?]
::tkutils::tkusysinfo::refresh $w            ;# neu sammeln -> Dict;
    ein Fehler steht in der Statuszeile
::tkutils::tkusysinfo::data    $w            ;# letztes Dict
::tkutils::tkusysinfo::report  $w ?-anonymize 0|1?
::tkutils::tkusysinfo::json    $w
::tkutils::tkusysinfo::save    $w datei ?-format text|json?
    ?-anonymize 0|1?
::tkutils::tkusysinfo::screen  fenster       ;# Tk-Bildschirmdaten
```

Option	Vorgabe	Wirkung
-sections	alle	Abschnitte von tusysinfo

Option	Vorgabe	Wirkung
-serials	0	1 zeigt Seriennummern
-timeout	10000	Grenze je Programmaufruf in ms
-autoload	1	gleich beim Aufbau sammeln
-height	16	sichtbare Zeilen
-buttons	1	0 ohne Knopfleiste

```
package require tkutils::tkusysinfo
pack [::tkutils::tkusysinfo::widget .si -sections {os cpu memory}] \
    -fill both -expand 1
dict keys [::tkutils::tkusysinfo::screen .]
;# -> pixels millimeters dpi scaling depth windowing
```

Tk sieht nur einen Bildschirm: Bei mehreren Monitoren ist `pixels` unter Linux die ganze X-Arbeitsfläche, unter Windows und macOS der Hauptmonitor. Die Monitore selbst stehen unter "Monitors", aus ihrer EDID. Unter macOS dauert das Sammeln einige Sekunden; der Mauszeiger zeigt solange `watch`. Eine unbekannte Option wirft `{TKUTILS TKUSYSINFO OPTION <opt>}`, ein anderes Format bei `save {TKUTILS TKUSYSINFO FORMAT}`.

12.14 Merkliste

- Die Viewer rufen die gleichnamige Engine in tclutils. XML braucht tDOM, SQLite das Paket `sqlite3`.
- `tkujson::setJson` liefert den Typ der Wurzel, `tkuxml::setXml` den Namen des Wurzelelements.
- `tkudavbrowser::setData` baut den Baum ohne Netz. `refresh` blockiert.
- `tkueditor` behält gemischte Zeilenenden, solange der Text unverändert ist.
- `tkusysinfo` braucht tclutils 0.69.0 und liegt deshalb außerhalb des Umbrellas.

Nächstes Kapitel: Designer und load

13 Designer und load

13.1 tkurender / tkuload

tkurender ist der Render-Kern von tkudesigner und tkuload; er kennt den Widget-Typ placeholder (Optionen `text`, `role`, `width`, `height`). tkuload baut aus einem `.tkd`-Entwurf eine lebende Oberfläche und bietet slot: Platzhalter-Rahmen leeren, der Host packt das echte Widget hinein.

Beide Module haben eigene Namensräume (`::tkurender`, `::tkuload`) und sind nicht im Umbrella. Die Pakete heißen `tkutils::tkurender` (0.2) und `tkutils::tkuload` (0.3).

```
package require tkutils::tkuload
package require tkutils::tkurender
package require tkutils::tkufilelist
```

```
# Entwurf mit einem benannten Platzhalter (sonst aus tkudesigner, .tkd)
::tkurender::newModel
set p [::tkurender::addNode placeholder $::tkurender::D(root)]
::tkurender::setName $p filelist
::tkurender::setOpt  $p role filelist
set spec [::tkurender::serialize]

ttk::frame .host
pack .host -fill both -expand 1
set ui [::tkuload::build .host $spec]

set box [::tkuload::slot $ui filelist]
::tkutils::tkufilelist::widget $box.l -dir [pwd]
pack $box.l -fill both -expand 1
pack propagate $box 1
```

Aus einer Datei: `::tkuload::buildFromFile $parent $pfad`.

`build` liefert ein Dict mit `root`, `byId`, `byType` und `byName`. Als `$parent` ist auch `.` erlaubt; `build` legt dann einen eigenen Rahmen `.tkuloadHost<n>` an. Jeden Entwurf in einen eigenen, leeren Parent bauen: Die Widget-Namen kommen aus den Knoten-Ids, ein zweiter Bau in denselben Parent kollidiert. Beim ersten `build` trägt `tkuload` die Pfade aus `TCLUTILS_TM` und `TKUTILS_TM` in den Modulpfad ein, falls gesetzt.

Weitere Befehle für benannte Widgets: `widgetByName`, `setValue`, `getValue`, `fill` (Dict Name → Wert, unbekannte Namen werden übersprungen) und `collect` (Werte zurück als Dict). Seit 0.3 nennt `namespace export` alle acht Befehle, `namespace import ::tkuload::*` holt sie also vollständig; bis 0.2 waren nur `build`, `buildFromFile` und `slot` exportiert.

13.2 Feste Größe des Platzhalters

Der Platzhalter ist ein `ttk::frame` mit fester Größe — Standard **200×120** Pixel — und `pack propagate 0`. `slot` entfernt nur die Kinder; Größe und Propagation bleiben. Ein hineingepacktes Widget wird also auf 200×120 beschnitten. Der Rahmen aus dem Beispiel ist vor `pack propagate $box 1` genau 200×120 groß bei Propagation 0, danach 199×251 (Tcl/Tk 9.0.4: 199×231). Abhilfe:

- `pack propagate $box 1` — der Rahmen wächst mit dem Inhalt, oder
- im Designer `width`/`height` passend setzen, oder
- den Platzhalter im Layout mit `-fill both -expand 1` (bzw. Grid-Gewicht) strecken.

13.3 Einsatz

Die App **Dateiblick** bleibt laut Buch *Utils-Stack* handverdrahtet (kein `.tkd`); ihr Code liegt nicht in tclutils, tkutils oder ctrlutils und ist hier nicht geprüft. tkudesigner zeigt Placeholder in der Palette (Gruppe *Widget*; im Code `defType placeholder Widget`).

Fehler von tkuload:

Code	Meldung
{TKULOAD PARENT}	no such parent widget: .nix
{TKULOAD NONAME}	no widget named 'foo' (aus slot und widgetByName)

Details: `docs/tkuload.md` im tkutils-Baum.

13.4 Seitenlayout: tkulayoutcanvas

tkulayoutcanvas 0.1 setzt benannte Blöcke auf eine Seite in Millimetern. Die Engine ist `tclutils::tulayout`. Feine Schrift und Spalten bleiben in der Anwendung.

```
::tkutils::tkulayoutcanvas::widget     $pfad ?option wert ...?
::tkutils::tkulayoutcanvas::getBlocks  $pfad
::tkutils::tkulayoutcanvas::setBlocks  $pfad $blöcke
::tkutils::tkulayoutcanvas::select     $pfad $id
::tkutils::tkulayoutcanvas::getCanvas  $pfad
::tkutils::tkulayoutcanvas::redraw     $pfad
```

Option	Vorgabe	Wirkung
-paper	a4	a4, a5, letter
-orientation	portrait	portrait oder landscape
-margin	20.0	Rand in mm
-gridmm	5.0	Raster in mm
-scale	3.0	Pixel je mm
-blocks	{}	Dict der Blöcke
-definitions	{}	Katalog: Beschriftung und Größe je Id

`mergePreset` übernimmt aus einer Vorlage nur die genannten Schlüssel, Vorgabe `x`, `y`, `w` und `show`.

13.5 Zusammenfassung

- placeholder/slot für Host-Apps und Designer.
- Platzhalter 200×120, `pack propagate 0` — beim Füllen bedenken.
- `build` liefert `root`, `byId`, `byType`, `byName`; Werte über `setValue/getValue/fill/collect`.
- Pflicht für Dateiblick: nein.
- `tkulayoutcanvas` setzt Blöcke in Millimetern. Die Engine ist `tulayout`.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux und Xvfb: das Beispiel mit `slot`, die Größen des Platzhalters, die Fehler von `tkuload`.

Nicht getestet: die App Dateiblick.

Nächstes Kapitel: Bilder

14 Bilder

Laden, skalieren und in eine Datei schreiben. `tkuimage`, `tkuwinico` und `tkudhash` liegen im Umbrella. `tkuicon` und `tkucanvaspng` kommen einzeln: `tkuicon` braucht für SVG eine Erweiterung, `tkucanvaspng` die Engine `tclutils::tupngdraw`.

14.1 Ansehen: tkuimage

tkuimage 0.2 liefert Hilfen und einen Viewer mit Scrollen und Zoom. Skalieren nimmt `imgtools`, wenn es geladen ist, sonst `subsample` und `zoom` des Tk-Photo. `imgtools` ist optional.

```
::tkutils::tkuimage::fit      bildB bildH kastenB kastenH
    ?contain|cover|none?
::tkutils::tkuimage::load     datei ?ziel?
::tkutils::tkuimage::fromData bytes ?ziel?
::tkutils::tkuimage::scale    quelle b h ?ziel?
::tkutils::tkuimage::thumbnail quelle max ?ziel?
```

`fit` liefert `{neueB neueH maßstab}`. `load` und `fromData` kennen PNG und GIF aus Tk selbst. `contain` passt das Bild in den Kasten, `cover` füllt ihn.

```
set v [::tkutils::tkuimage::view .v ?-fitmode contain? ?-onchange
    befehl?]
::tkutils::tkuimage::openFile  .v datei.png
::tkutils::tkuimage::zoomIn    .v
::tkutils::tkuimage::zoomOut   .v
::tkutils::tkuimage::zoom1     .v
::tkutils::tkuimage::fitView   .v
::tkutils::tkuimage::getImage  .v
::tkutils::tkuimage::zoomLevel .v
```

`zoomLevel` ist `1.0` bei 100 Prozent. `getImage` ist das Photo in Originalgröße.

14.2 Symbole: tkuicon

tkuicon 0.1 lädt SVG und PNG und erzeugt einen festen Satz aus `tclutils::tusvg`. Ergebnisse bleiben im Cache. SVG unter Tk 8.6 braucht `tksvg`. Tk 9 zeichnet SVG selbst. Ohne SVG laden Dateien weiterhin PNG und GIF. `create` wirft dann `{TKUTILS TKUICON NOSVG}`.

```
::tkutils::tkuicon::hassvg
::tkutils::tkuicon::available
::tkutils::tkuicon::create name größe ?-color farbe? ?-strokeWidth n?
    ?-name bild?
::tkutils::tkuicon::load   datei ?-height h? ?-width b? ?-name bild?
::tkutils::tkuicon::rescale bildname neueHöhe
::tkutils::tkuicon::loadToolbarSet größe ?-color farbe? ?-icons {namen
    ...}?
::tkutils::tkuicon::clearCache
```

`create` liefert den Namen des Photo. `loadToolbarSet` liefert ein Dict. `rescale` heißt so,

weil `scale` ein Tk-Befehl ist. Mit SVG zeichnet es neu, sonst skaliert es das vorhandene Bild. Die Leiste, die das Photo als `-icon` nimmt, steht im Kapitel Gerüst.

14.3 Zeichenfläche als PNG: tkucanvaspng

tkucanvaspng 0.2 schreibt eine Tk-Zeichenfläche über `tclutils::tupngdraw`. Echte Schrift braucht Glyphs und `-fontmap`.

```
set png [::tkutils::tkucanvaspng::render $canvas \
    ?-region {x1 y1 x2 y2}? ?-scale n? ?-background farbe? \
    ?-textcmd befehl? ?-fontmap dict?]
::tkutils::tkucanvaspng::write aus.png $canvas ?dieselben Optionen?
```

Die Vorgabe für `-region` ist `[$canvas bbox all]`. Eine leere Fläche wirft `{TKUTILS TKCANVASPNG EMPTY}` mit `canvas is empty`. Ein Fenster, das es nicht gibt, wirft `{TKUTILS TKCANVASPNG WINDOW}` mit `not a canvas window: .nix`. Ein vorhandenes Widget, das keine Zeichenfläche ist, prüft das Modul nicht; es scheitert mit dem Tk-Fehler `bad command "bbox"`. Eine unbekannte Option meldet `{TCLUTILS COMMON OPTION <opt>}`. `-scale` ist eine positive ganze Zahl. `-background` ist der Hintergrund der Fläche, sonst weiß. `write` liefert den Dateinamen.

14.4 Windows-Symbol: tkuwinico

tkuwinico 0.1 lässt Tk die einzelnen Größen zeichnen und gibt die PNG-Daten an `tclutils::tuico`. Die Transparenz bleibt, weil die Nutzlast PNG ist.

```
::tkutils::tkuwinico::fromSvg    logo.svg app.ico
::tkutils::tkuwinico::fromPhoto  $photo app.ico ?-sizes {32 16}?
::tkutils::tkuwinico::fromPhotos {{32 $groß} {16 $klein}} app.ico
```

`fromSvg` zeichnet jede Größe aus der Vorlage. `fromPhoto` skaliert ganzzahlig. `fromPhotos` nimmt je Größe ein fertiges Bild und skaliert hier nichts. SVG-Ausgabe braucht dieselbe SVG-Unterstützung wie `tkuicon`.

14.5 Ähnlichkeit: tkudhash

tkudhash 0.1 lädt eine Bilddatei als Photo und gibt die Pixel an `tclutils::tudhash`. PNG, GIF und PPM gehen mit Tk allein. JPEG und TIFF brauchen das Paket `Img`.

```
::tkutils::tkudhash::fromFile     pfad ?-sample n?
::tkutils::tkudhash::similarFiles pfadA pfadB ?abstand?
::tkutils::tkudhash::distance     a b
::tkutils::tkudhash::similar      a b ?abstand?
```

`fromFile` liefert 16 Hexzeichen, den 64-Bit-Hash. `-sample` ist die Kantenlänge der Probe. `distance` und `similar` kommen von `tudhash`. `distance` liegt zwischen 0 und 64. `similarFiles` und `similar` nehmen als Vorgabe den Abstand 10.

14.6 Merkliste

- `tkuimage::fit` liefert Breite, Höhe und Maßstab. `imgtools` ist optional.
- `tkuicon` und `tkuwinico` brauchen für SVG `tksvg` oder Tk 9.
- `tkucanvaspng` schreibt PNG über `tupngdraw`. Eine leere Fläche wirft `EMPTY`.
- `tkudhash` rechnet nicht selbst. Den Hash liefert `tudhash`.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux und Xvfb: Laden der Module, `tkuimage::fit`, die Fehler von `tkucanvaspng` und `tkuicon`. Unter 8.6.14 ohne tksvg liefert `tkuicon::hassvg 0` und `create` den Fehler `NOSVG`; unter 9.0.4 liefert `hassvg 1`.

Nächstes Kapitel: tkdevtools

15 tkdevtools

15.1 Die App

tkdevtools (`apps/tkdevtools/tkdevtools.tcl`) ist ein Nachschlage- und Probier-Browser für Tk-Entwicklung: Referenztabellen, Cheat-Sheets und Proben in einer App mit Navigationsbaum. Reine Anzeige — im normalen Betrieb schreibt die App keine Dateien, nur die Zwischenablage. Ausnahme ist der Screenshot-Modus.

```
wish apps/tkdevtools/tkdevtools.tcl
tclsh apps/tkdevtools/tkdevtools.tcl --shot out.png shadow
```

Der Screenshot-Modus rendert ein Werkzeug (Schlüssel wie `shadow`, `colors`, `regexp`) in eine PNG-Datei mit 900×600 Pixeln und beendet sich. Er braucht das Paket `img::window` (Img); fehlt es, versucht die App ImageMagicks `import -window root`. Ohne Schlüssel bleibt das erste Werkzeug (*Tcl / Tk*) gewählt.

Mit `tclsh` und den Schlüsseln `shadow`, `colors` und `regexp` entstehen drei PNG mit 900×600 Pixeln, Exit-Code 0.

Voraussetzungen: Tcl/Tk 8.6 oder 9.x, mindestens **tclutils 0.64.0** (`tucolor`, `tustr` 0.2, `tudate` 0.2, `tupkgfinder` 0.2, `tuappinfo`, `tupath`, `tunum` 0.4; empfohlen 0.69.0), aus tkutils `tkutablelist` ab 0.4 und `tkuunits`, dazu Tablelist. Die App lädt `tablelist_tile` vor `tkutablelist`; die Helfer `tkutl*` nehmen dieselbe Variante (Kapitel tkutils verstehen). Die Helfer `tkutlsort`, `tkutlfind`, `tkutlclip` werden genutzt, wenn vorhanden. Mit `tkutlclip` 0.3 (tkutils 0.49.0) kopiert Strg+C in den Tabellen die markierten Zeilen als TSV; vorher überschrieb die eigene Bindung von Tablelist die Zwischenablage. Die Bibliotheken findet die App über `apps/_lib/paths.tcl` (dieselbe Logik wie `tools/setup.tcl`).

Der Knopf *Copy report* im Werkzeug *Tcl / Tk* kopiert einen Umgebungsbericht (`tuappinfo::buildReport`) in die Zwischenablage.

15.2 Die Werkzeuge

Gruppe	Werkzeuge
Runtime	Tcl /Tk, OS /platform, Packages, Package shadows, Paths, Display, Environment, Channels, Photo formats, External tools

Gruppe	Werkzeuge
Reference	Colors, Characters, Graphemes, Fonts, Bitmaps, Font aliases
Conversion / time	Units, Timezones, clock format, Number format
Platform	Paths, glob, Programs, Encoding profiles
Tk widgets / style	Widget tree, Clipboard, Cursors, Relief /Anchor, Keysym probe, ttk Themes, Widget explorer
Layout	pack, grid
Text / patterns	regexp, format / scan, Encodings, Virtual events

Fett: seit 0.46.0. *Paths* gibt es zweimal: unter *Runtime* die Suchpfade (`auto_path`, `tcl_pkgPath`, `tcl::tm::path`, Encoding-Ordner), unter *Platform* die Pfadprobe.

15.3 Werkzeuge seit 0.46.0

Package shadows. Jede Kopie eines Pakets in Modulpfaden und `auto_path` (`tupkgfinder::shadows`). Rot markiert ist eine zweite Kopie derselben Version, die nie geladen wird — etwa eine Installation neben dem Arbeitsbaum. Die Zeile ist hellrot hinterlegt, die aktive Kopie grün geschrieben, andere Versionen grau. Welche gewinnt, hängt von der Pfadreihenfolge ab.

Paths (Platform). `file pathtype`, `split`, `dirname`, `normalize`, `nativename` und `tupath::clean` für einen Pfad, dazu die Einordnung nach Windows- und Unix-Regeln (`/x` und `C:x` sind unter Windows volumerelativ).

glob. Was ein Muster in einem Verzeichnis trifft, mit `-types`. Zeigt, wie viele Punkt-Dateien `*` hier auslässt; unter Windows trifft `*` sie, und `*` plus `.*` liefert sie doppelt — der Fehler, den `tkufiletree` behoben hat (Kapitel Datei-Widgets).

Programs. `auto_execok` für einen Namen und die PATH-Einträge mit Treffern; Trennzeichen und unter Windows PATHEXT.

Encoding profiles. Bytes (hex) in einer Kodierung mit den Profilen `strict`, `replace` und `tcl8` gelesen (Tcl 9). `strict` bricht beim ersten ungültigen Byte ab, wie ein Kanal mit Standardprofil. Unter Tcl 8.6 gibt es keine Profile; die Tabelle zeigt dann eine Zeile `(8.6)` mit dem Ergebnis ohne Fehler.

Font aliases. Was Tk aus Helvetica, Times, Courier, Arial und den `Tk*Font`-Namen macht (Familie, Größe, Metriken). Gelb: der Name kommt unverändert zurück (Tk 9 unter Windows bei Helvetica) und kann in einem PDF-Export mit der gleichnamigen Standardschrift zusammenfallen. Die Zeile ist dann gelb hinterlegt.

Number format. Dieselben Werte mit `format %.Nf` (Rundung der C-Laufzeit; glibc und Windows unterscheiden sich bei x,5) und `tunum::format` (kaufmännisch, 0,5 weg von null), dazu der gespeicherte double-Wert.

15.4 Zusammenfassung

- tkdevtools: Referenz und Proben, schreibt nichts.

- Seit 0.46.0: Package shadows, Paths, glob, Programs, Encoding profiles, Font aliases, Number format.
- Braucht mindestens tclutils 0.64.0 (empfohlen 0.69.0) und Tablelist.
- `--shot` braucht Img (`img::window`) oder ImageMagick.

Getestet mit Tcl/Tk 8.6.14 (Img 1.4.16) und 9.0.4 (Img 2.1.1) unter Linux und Xvfb: `--shot` mit `shadow`, `colors` und `regexp`.

Nicht getestet: Anzeige der Kodierungsprofile unter Tcl 9, Schrift-Aliase unter Windows.

Nächstes Kapitel: Grenzen

16 Grenzen

- Ohne tclutils kein sinnvolles Pairing (Engines, tusettings, tutestrun); mindestens 0.64.0, empfohlen 0.69.0.
- Der Umbrella braucht Tablelist (über `tkufilelist` → `tkutablelist`).
- tkutils lädt überall `tablelist_tile`; eine Anwendung, die selbst zusätzlich `tablelist` anfordert, scheitert an tklib.
- Nicht jedes Modul im Umbrella: `tkuxml`, `tkusqlite`, `tkuicon`, `tkuscrolledframe`, `tkuload`, `tkurender`, die `tkutl*`-Helfer und weitere extra requiren (Kapitel tkutils verstehen).
- `tkufiledialog` ist kein Explorer; `-filetypes` filtert nur nach der ersten Endung eines Typs.
- tablelist kommt von außen (tklib, z. B. `apt install tklib`).
- tical kommt von außen; `setup.tcl` findet es nicht.
- `tkugeometry` erkennt Lücken zwischen verschieden großen Monitoren nicht.
- Die GUI-Tests brauchen ein Display; ohne Display werden sie übersprungen.
- Nicht jedes Modul prüft seine Optionen: `tkutlfooter` übergeht unbekannte Optionen.
- Controller und Undo: ctrlutils, nicht hier.
- Geprüft mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux (Xvfb). Windows, macOS und die echte Bildschirmdarstellung sind nicht geprüft.

16.1 Zusammenfassung

- View-Schicht; Model und Controller daneben.
- API-Wahrheit: Modul-Docs `docs/<modul>.md`.
- Die Lücken von 0.46.0 (Tablelist-Varianten, Doppelklick im Filedialog, Tests ohne Display) sind in 0.47.0 behoben; Einzelheiten in den Anhängen A und G.

Nächstes Kapitel: Fallen

17 Fallen

- Kurzform `tkudialog::confirm` statt `::tkutils::tkudialog::confirm` — `invalid command name "tkudialog::confirm"`.
- TCLUTILS_TM/TKUTILS_TM setzen und `setup.tcl` weglassen — Tcl liest die Variablen nicht; `can't find package tkutils` bzw. `can't find package tclutils::…`.

- Umbrella ohne Tablelist — `can't find package Tablelist`.
- `tkudialog` 0.2 erwarten — Datei entfernt, Verhalten = 0.3.
- `tkudialog::message`/`warning` aufrufen — heißen `showInfo`/`showWarning`.
- `input` liefert `""` — Abbruch oder leeres Feld, beides gleich.
- `tkutablelist` 0.3 requiren und alte PRIMARY-Auswahl erwarten — es kommt immer 0.4; `-exportselection 1` setzen.
- `tkufilelist` ohne `-dir` — zeigt `/`, nicht das aktuelle Verzeichnis.
- `tkugeometry::restore` vor `track` aufrufen — doppelt; `track` stellt selbst her.
- `-default 800x600` gibt nur eine Größe; die Position wählt der Window-Manager (unter Xvfb ohne Fenstermanager `+0+0`; unter einem echten Fenstermanager nicht geprüft). Der Test `tkugeometry-3.2` vergleicht deshalb nur die Größe.
- `setEnabled id 0` (global) nach `setEnabled id 0 $w` — löscht die Fenster-Overrides; `invoke` folgt nur dem globalen Flag.
- Placeholder füllen und wundern, dass der Inhalt abgeschnitten ist — 200×120 und `pack propagate 0`.
- Tests mit `wish` starten — `tclsh tests/all.tcl`.
- Dateioperationen im Widget suchen — gehören in ctrlutils.
- Unbekannte Optionen an `tkutlfooter` — werden ohne Fehler übergangen.
- `tkutablelist::toCsv` oder `saveCsv` mit versteckten Zeilen oder Spalten — seit 0.49.0 exportieren beide nur Sichtbares; alles mit `-all 1`.
- Eine `integer`- oder `real`-Spalte mit leeren Zellen sortieren — Tablelist bricht mit einem Fehler ab; `tkutlsort::column … -empty last`.
- Von `tkutltree::fromData` den Tabellenpfad zurück erwarten — seit 0.3 kommen die Schlüssel der obersten Knoten.
- Strg+C mit `tkutlclip::installBindings` bis tkutils 0.48.0 — die eigene `<<Copy>>`-Bindung von Tablelist überschrieb die Zwischenablage; seit `tkutlclip` 0.3 behoben.
- 0.49.0 über einen älteren Baum kopieren und die alten `.tm`-Dateien liegen lassen — dann liegen zwei Versionen nebeneinander (Liste im Kapitel Installation).
- Auf den Fehlercode `{TKUTILS <MODUL> OPTION}` ohne viertes Element prüfen — ab 0.47.0 trägt er auch in `tkuaction` (`define`), `tkuballoon`, `tkuform`, `tkukeynav`, `tkulayoutcanvas`, `tkumarquee`, `tkuscrolledframe`, `tkutablelist`, `tkutoolbar` und `tkuvalidate` den Optionsnamen als viertes Element: `{TKUTILS <MODUL> OPTION <opt>}`. Bis 0.46.0 fehlte er dort, und die Meldung lautete `unknown option '<opt>'` ohne `Known:`-Zeile. Code, der den ganzen Code oder den Meldungstext vergleicht, muss angepasst werden; `lrange $code 0 2` passt für beide Stände. Nur ein fehlender Wert meldet weiterhin `OPTION` ohne Optionsnamen.

17.1 Behoben in 0.47.0

Diese Fallen gelten für 0.46.0. In 0.47.0 sind sie behoben:

- Doppelklick auf eine Datei in `tkufiledialog` 0.1 — Dialog schließt, das Ergebnis ist `""`; bis 0.46.0 nötig: *OK* oder Return benutzen. Behoben in `tkufiledialog` 0.2.
- Tests ohne Display starten — vier Dateien brechen ab, Exit-Code 1.
- `tkumonthcanvas.test` ohne `TICAL_DIR` — alle Tests übersprungen, wenn tical flach neben tkutils liegt; bis 0.46.0 nötig: `TICAL_DIR` setzen.
- `tkutltools` oder ein `tkutl*`-Helfer **nach** dem Umbrella — `can't set "usingTile": it is not supported to use both Tablelist and Tablelist_tile in the`

`same application`; bis 0.46.0 nötig: `tablelist` zuerst laden. Jetzt laden alle `tablelist_tile`.

17.2 Zusammenfassung

- Die meisten Fallen sind Pfad- und Ladefragen: setup.tcl, Tablelist, Ladereihenfolge.
- Zwei Fallen sind Code-Fehler in 0.46.0: Doppelklick im Filedialog und Tests ohne Display. Beide sind in 0.47.0 behoben.
- Code, der Optionsfehler am Fehlercode erkennt, muss ab 0.47.0 in zehn weiteren Modulen den Optionsnamen als viertes Element erwarten.

Nächstes Kapitel: Widgets

18 Widgets (Orientierung)

Übersicht im tkutils-Baum: `docs/guide/tkutils-modules.md`. **79** Module, je mit Test, Doc und Manpage (`tools/check-docman.tcl` meldet keine fehlenden Dateien). Der Umbrella lädt 53 direkt und `tkutablelist` und `tkuopts` indirekt; maßgeblich ist der Code des Umbrellas. Keine Parallelversionen: `tkutablelist` nur 0.5, `tkudialog` nur 0.3.

18.1 Auszug

Widget	Rolle
tkudialog 0.3	showInfo/Warning/Error, confirm, input, form, build/show
tkufiledialog 0.2	open/save/directory
tkufilelist 0.2 / tkufiletree 0.2	Provider-UI
tkupath 0.3	Pfadzeile, auch Laufwerk und UNC
tkupreview 0.2	Vorschau, Texte über msgcat
tkutablelist 0.5	tablelist-Wrapper, -exportselection 0, Zeilen-IDs
tkugeometry 0.1	Fenstergeometrie merken
tkuunits 0.1	px/pt/mm/cm/inch
tkuaction 0.2	Actions, Enable pro Fenster
tkutoolbar 0.2	Leiste, addAction
tkulauncher 0.4	Programmstarter (Menü/Liste)
tkuload 0.3 / tkurender 0.2	UI-Laden / placeholder (nicht im Umbrella)
tkuopts 0.1	Optionsprüfung (indirekt geladen)
tkuwinico 0.1	Windows-Symbol aus Tk-Bildern
tkudhash 0.1	Bildähnlichkeit über tudhash

18.2 Nur einzeln

Modul	Braucht
tkuxml 0.1	tDOM
tkusqlite 0.1	sqlite3
tkuicon 0.1	tksvg / Tk 9 für SVG
tkuscrolledframe 0.1	scrollutil
tkutical 0.4, tkumonthcanvas 0.5	tical 1.0 (-events: 1.1)
tkucanvaspng 0.2	tclutils::tupngdraw
tkutltools 0.3, tkutl* (0.1 bis 0.4)	Tablelist (tablelist_tile)
tkumdview, tkupdfinspect, tkulayoutcanvas (je 0.1)	tclutils-Engines
tkusysinfo 0.1	tclutils::tusysinfo (tclutils 0.69.0)

18.3 Zusammenfassung

- 79 Module: 55 über den Umbrella, 24 einzeln.
- Die Versionen dieses Anhangs sind die Dateien im Baum von 0.49.0.

Nächstes Kapitel: Glossar

19 Glossar

Umbrella
Paket `tkutils`, lädt 53 Kernmodule direkt; braucht Tablelist.

Namensraum
`::tkutils::tku<name>` — Befehle immer voll qualifiziert aufrufen.

Provider
Speicher-API aus tclutils (`tuprovider`); speist File-Widgets.

exportselection
Tk-PRIMARY-Export; Default in tkutablelist 0.4 aus.

Button-Id
Rückgabe von `tkudialog::show`; `{id Label}` in `-buttons`.

Fenster-Override
`tkuaction::setEnabled id 0 $w` — Enable nur für ein Toplevel.

placeholder
tkurender-Typ: Rahmen mit Beschriftung (Standard 200×120) als Host-Slot; `slot` leert ihn.

slot
`::tkuload::slot` — Platzhalter leeren für Host-Widget.

tutestrun
Testrunner aus tclutils; `tclsh tests/all.tcl`.

Package shadow
Zweite Kopie eines Pakets auf dem Suchpfad; tkdevtools zeigt sie.

Pairing
Empfohlene Versionskombination: tclutils 0.69.0 + tkutils 0.49.0 + ctrlutils 0.5.

tablelist_tile
ttk-Variante von Tablelist; `tkutablelist` und die `tkutl*`-Helfer laden sie. Beide Varianten schließen sich aus.

19.1 Zusammenfassung

- Kurzdefinitionen der Begriffe, die im Buch wiederkehren.

Nächstes Kapitel: Fehlermeldungen

20 Fehlermeldungen

Code / Symptom	Bedeutung
invalid command name "tkudialog::..."	Namensraum fehlt; ::tkutils::tkudialog::...
can't find package tclutils::...	tclutils nicht auf Modulpfad
can't find package tkutils::...	setup.tcl / tcl::tm::path add fehlt
can't find package Tablelist	Umbrella oder Tabellen-Widget ohne tklib
can't set "usingTile": ...	bis 0.46.0: tkutl* nach tkutablelist geladen, tablelist zuerst laden; behoben in 0.47.0. Heute nur noch, wenn eine Anwendung selbst beide Varianten anfordert
{TKUTILS <MODUL> OPTION <opt>}	unbekannte Option (Module mit tkuopts, tkudialog, tkufiledialog, ab 0.47.0 zehn weitere, siehe unten)
{TKUTILS <MODUL> OPTION}	Wert zur Option fehlt; bis 0.46.0 in zehn Modulen auch unbekannte Option
{TCLUTILS COMMON OPTION <opt>}	unbekannte Option in tkugeometry, tkucanvaspng, tkudavbrowser::refresh
{TKUTILS TKUDIALOG PARENT <w>}	-parent existiert nicht
{TKUTILS TKUFILEDIALOG PARENT <w>}	dito im Filedialog
{TKUTILS TKUGEOMETRY WINDOW}	kein Toplevel für save/track/restore
{TCLUTILS TUSETTINGS ...}	Geometrie-Persistenz (tusettings), z. B. APP
{TKUTILS TKUUNITS UNIT}	unbekannte Einheit
{TKUTILS TKUUNITS VALUE}	keine Zahl (2c)

Code / Symptom	Bedeutung
{TKUTILS TKUUNITS WINDOW}	kein solches Fenster
{TKUTILS TKUACTION WINDOW}	unbekanntes Fenster bei Enable
{TKUTILS TKUACTION NOACTION} / NOGROUP	unbekannte Aktion / Gruppe
{TKULOAD PARENT} / {TKULOAD NONAME}	tkuload: Parent / Name fehlt
unknown interpreter: <key>	tkulauncher: kein solcher Interpreter-Schlüssel
ITEM_EXISTS in tkufiletree	vor 0.46.0: Punkt-Dateien doppelt unter Windows
{TKUTILS TKCANVASPNG WINDOW} / EMPTY	tkucanvaspng: Fenster fehlt / Fläche leer
{TKUTILS TKUICON NOSVG}	tkuicon: SVG ohne tksvg unter Tk 8.6

Die Optionsprüfung von tkuopts benutzen 20 Module, darunter `tkufilelist`, `tkufiletree`, `tkupath` und `tkulauncher`. Sie melden eine unbekannte Option als `{TKUTILS <MODUL> OPTION <opt>}` mit `unknown option "<opt>"` und in der zweiten Zeile `Known:` mit den sortierten bekannten Optionen. Ein fehlender Wert meldet `{TKUTILS <MODUL> OPTION}` mit `missing value for option "<opt>"`. `tkudialog` und `tkufiledialog` prüfen selbst, mit demselben Aufbau; die `Known:`-Liste steht dort in der Reihenfolge des Codes.

Zehn Module prüfen ihre Optionen selbst: `tkuaction` (bei `define`), `tkuballoon`, `tkuform`, `tkukeynav`, `tkulayoutcanvas`, `tkumarquee`, `tkuscrolledframe`, `tkutoolbar`, `tkuvalidate` und `tkutablelist` (das prüft vor `tkuopts`). Ab 0.47.0 folgen sie dem Aufbau von `tkuopts`: Code `{TKUTILS <MODUL> OPTION <opt>}`, Meldung `unknown option "<opt>"` und in der zweiten Zeile `Known:` mit den sortierten bekannten Optionen. Bis 0.46.0 fehlte dort der Optionsname im Code, die `Known:`-Liste fehlte, und die Meldung lautete `unknown option '<opt>'`. Wer den Fehlercode vergleicht, muss ab 0.47.0 die Option als viertes Element erwarten (Anhang Fallen).

Nur wenige Module nutzen die tclutils-Prüfung und melden `{TCLUTILS COMMON OPTION <opt>}`.

Nicht jedes Modul folgt dem Schema: `tkutlclip` und `tkutlfind` melden `{TKUTILS <MODUL> OPTION}` ohne Optionsnamen und ohne `Known:`-Zeile, `tkutlstate::autosave` den Grund `badoption` in Kleinbuchstaben, und `tkutlfooter` prüft Optionen nicht. Die neuen Helfer `tkutlfilter`, `tkutlgroup` und `tkutldrag` melden `OPTION` ohne Optionsnamen im Code, nennen die bekannten Optionen aber in der Meldung (`unknown option "-x": must be …`).

Modul-spezifische Codes: jeweiliges `docs/<modul>.md`.

20.1 Zusammenfassung

- Fehlercodes folgen `{TKUTILS <MODUL> <GRUND>}`; Ausnahmen sind tkuload (`TKULOAD`) und Module mit tclutils-Prüfung (`TCLUTILS COMMON`).
- Ab 0.47.0 nennen die zehn Module mit eigener Optionsprüfung den Optionsnamen im Code und die bekannten Optionen in der Meldung; bis 0.46.0 fehlte beides. Module mit tclutils-Prüfung melden dasselbe unter `{TCLUTILS COMMON OPTION <opt>}`. Abweichend:

`tkutlclip`, `tkutlfind`, `tkutlstate`, `tkutlfooter`.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux und Xvfb: Codes und Meldungen von `tkudialog`, `tkutablelist`, `tkutl*`, `tkugeometry`, `tkuunits`, `tkuaction`, `tkuform`, `tkulauncher`, `tkucanvaspng` und `tkuload` wie hier und in den Kapiteln angegeben. Die übrigen Zeilen sind aus dem Code gelesen.

21 API-Katalog: Tkutils-Module

Der Katalog nennt die Prozedursignaturen der öffentlichen Befehle, in der Reihenfolge der Quelldatei `lib/tm/tkutils/<modul>-<version>.tm`. Er ist aus den Quellen von tkutils 0.49.0 erzeugt. Optionen, Rückgaben, Fehlercodes und Nebenwirkungen stehen in den Kapiteln und in `docs/<modul>.md`.

Öffentlich heißt: von `namespace export` erfasst. Eine Signatur mit `args` nimmt Optionen oder eine variable Zahl von Argumenten; ein Paar `{name wert}` ist ein Argument mit Vorgabe.

Module mit Einträgen: **78** von 79. `tkutltools` hat keine eigenen Befehle; es lädt die zehn `tkutl*`-Helfer.

21.1 tkuaction

Quelle: `lib/tm/tkutils/tkuaction-0.2.tm`.

```
::tkutils::tkuaction::define {name args}
::tkutils::tkuaction::exists {name}
::tkutils::tkuaction::get {name {key ""}}
::tkutils::tkuaction::names {}
::tkutils::tkuaction::delete {name}
::tkutils::tkuaction::register {name widget}
::tkutils::tkuaction::unregister {name widget}
::tkutils::tkuaction::registerMenu {name menu {index end}}
::tkutils::tkuaction::unregisterMenu {name menu {index ""}}
::tkutils::tkuaction::addMenuItem {menu name}
::tkutils::tkuaction::setEnabled {name enabled {win ""}}
::tkutils::tkuaction::setChecked {name checked}
::tkutils::tkuaction::getEnabled {name {win ""}}
::tkutils::tkuaction::getChecked {name}
::tkutils::tkuaction::toggle {name}
::tkutils::tkuaction::invoke {name}
::tkutils::tkuaction::groupDefine {name actionList}
::tkutils::tkuaction::groupSet {name enabled {win ""}}
::tkutils::tkuaction::groupAdd {name action}
::tkutils::tkuaction::groupList {{name ""}}
::tkutils::tkuaction::reset {}
```

21.2 tkuballoon

Quelle: `lib/tm/tkutils/tkuballoon-0.1.tm`.

```
::tkutils::tkuballoon::configure {args}
::tkutils::tkuballoon::cget {option}
::tkutils::tkuballoon::enable {}
::tkutils::tkuballoon::disable {}
::tkutils::tkuballoon::add {w text args}
::tkutils::tkuballoon::clear {w}
```

21.3 tkubase64

Quelle: `lib/tm/tkutils/tkubase64-0.2.tm`.

```
::tkutils::tkubase64::widget {path args}
::tkutils::tkubase64::setInput {path text}
::tkutils::tkubase64::getInput {path}
::tkutils::tkubase64::getOutput {path}
::tkutils::tkubase64::encode {path}
::tkutils::tkubase64::decode {path}
```

21.4 tkubind

Quelle: `lib/tm/tkutils/tkubind-0.1.tm`.

```
::tkutils::tkubind::platform {{key ""}}
::tkutils::tkubind::accelerator {spec}
::tkutils::tkubind::isEditing {}
::tkutils::tkubind::key {spec script args}
::tkutils::tkubind::unbind {id}
::tkutils::tkubind::context {widget script}
::tkutils::tkubind::doubleClick {widget script}
::tkutils::tkubind::group {cmd args}
::tkutils::tkubind::getInfo {args}
::tkutils::tkubind::clear {}
```

21.5 tkucal

Quelle: `lib/tm/tkutils/tkucal-0.1.tm`.

```
::tkutils::tkucal::widget {path args}
::tkutils::tkucal::setMonth {path year month}
::tkutils::tkucal::getMonth {path}
::tkutils::tkucal::next {path}
::tkutils::tkucal::prev {path}
::tkutils::tkucal::today {path}
::tkutils::tkucal::getText {path}
```

21.6 tkucalc

Quelle: `lib/tm/tkutils/tkucalc-0.2.tm`.

```
::tkutils::tkucalc::evaluate {text}
::tkutils::tkucalc::widget {path args}
::tkutils::tkucalc::getHistory {path}
::tkutils::tkucalc::clearHistory {path}
::tkutils::tkucalc::clear {path}
```

21.7 tkucalendar

Quelle: `lib/tm/tkutils/tkucalendar-0.2.tm`.

```
::tkutils::tkucalendar::widget {path args}
::tkutils::tkucalendar::getDate {path}
::tkutils::tkucalendar::getFormatted {path}
::tkutils::tkucalendar::setDate {path iso}
::tkutils::tkucalendar::today {path}
::tkutils::tkucalendar::next {path}
::tkutils::tkucalendar::prev {path}
::tkutils::tkucalendar::getMonth {path}
```

21.8 tkucanvaspng

Quelle: `lib/tm/tkutils/tkucanvaspng-0.2.tm`.

```
::tkutils::tkucanvaspng::render {canvas args}
::tkutils::tkucanvaspng::write {file canvas args}
```

21.9 tkucontextmenu

Quelle: `lib/tm/tkutils/tkucontextmenu-0.1.tm`.

```
::tkutils::tkucontextmenu::create {path args}
::tkutils::tkucontextmenu::addItem {menu label args}
::tkutils::tkucontextmenu::addSeparator {menu}
::tkutils::tkucontextmenu::addSubmenu {menu label submenu}
::tkutils::tkucontextmenu::addCheckItem {menu label args}
::tkutils::tkucontextmenu::addRadioItem {menu label args}
::tkutils::tkucontextmenu::attach {menu widget args}
::tkutils::tkucontextmenu::detach {menu widget}
::tkutils::tkucontextmenu::show {menu x y}
::tkutils::tkucontextmenu::enable {menu label}
::tkutils::tkucontextmenu::disable {menu label}
::tkutils::tkucontextmenu::setCallback {menu label command}
::tkutils::tkucontextmenu::setUpdateHandler {menu handler}
::tkutils::tkucontextmenu::destroyMenu {menu}
::tkutils::tkucontextmenu::createStandardEdit {path args}
::tkutils::tkucontextmenu::createFromSpec {path spec}
```

21.10 tkucsv

Quelle: `lib/tm/tkutils/tkucsv-0.1.tm`.

```
::tkutils::tkucsv::widget {path args}
::tkutils::tkucsv::setData {path csvText args}
::tkutils::tkucsv::loadFile {path filename args}
::tkutils::tkucsv::getRows {path}
```

21.11 tkudateentry

Quelle: `lib/tm/tkutils/tkudateentry-0.1.tm`.

```
::tkutils::tkudateentry::widget {path args}
::tkutils::tkudateentry::getText {path}
::tkutils::tkudateentry::getDate {path}
::tkutils::tkudateentry::setDate {path iso}
::tkutils::tkudateentry::today {path}
::tkutils::tkudateentry::clear {path}
```

21.12 tkudavaccount

Quelle: `lib/tm/tkutils/tkudavaccount-0.1.tm`.

```
::tkutils::tkudavaccount::widget {path args}
::tkutils::tkudavaccount::getConfig {path}
::tkutils::tkudavaccount::setConfig {path cfg}
::tkutils::tkudavaccount::clear {path}
::tkutils::tkudavaccount::focusUrl {path}
::tkutils::tkudavaccount::testConnection {path}
```

21.13 tkudavbrowser

Quelle: `lib/tm/tkutils/tkudavbrowser-0.2.tm`.

```
::tkutils::tkudavbrowser::widget {path args}
::tkutils::tkudavbrowser::treeWidget {path}
::tkutils::tkudavbrowser::setClient {path client}
::tkutils::tkudavbrowser::refresh {path args}
::tkutils::tkudavbrowser::setData {path cols}
::tkutils::tkudavbrowser::collections {path}
::tkutils::tkudavbrowser::selected {path}
::tkutils::tkudavbrowser::createCalendar {path displayname}
::tkutils::tkudavbrowser::createAddressbook {path displayname}
::tkutils::tkudavbrowser::deleteCollection {path {href ""}}
```

21.14 tkudhash

Quelle: `lib/tm/tkutils/tkudhash-0.1.tm`.

```
::tkutils::tkudhash::fromFile {path args}
::tkutils::tkudhash::distance {a b}
::tkutils::tkudhash::similar {a b {maxDist 10}}
::tkutils::tkudhash::similarFiles {pathA pathB {maxDist 10}}
```

21.15 tkudialog

Quelle: `lib/tm/tkutils/tkudialog-0.3.tm`.

```
::tkutils::tkudialog::choose {win id}
::tkutils::tkudialog::build {win args}
::tkutils::tkudialog::getText {win}
::tkutils::tkudialog::getDetail {win}
::tkutils::tkudialog::result {win}
::tkutils::tkudialog::copyText {win}
::tkutils::tkudialog::show {win args}
::tkutils::tkudialog::showInfo {message args}
::tkutils::tkudialog::showWarning {message args}
::tkutils::tkudialog::showError {message args}
::tkutils::tkudialog::confirm {message args}
::tkutils::tkudialog::input {args}
::tkutils::tkudialog::form {fieldspec args}
```

21.16 tkudiff

Quelle: `lib/tm/tkutils/tkudiff-0.1.tm`.

```
::tkutils::tkudiff::widget {path args}
::tkutils::tkudiff::setTexts {path oldText newText args}
::tkutils::tkudiff::loadFiles {path oldFile newFile args}
::tkutils::tkudiff::getOps {path}
```

21.17 tkueditor

Quelle: `lib/tm/tkutils/tkueditor-0.4.tm`.

```
::tkutils::tkueditor::widget {path args}
::tkutils::tkueditor::toolbarWidget {path}
::tkutils::tkueditor::statusbarWidget {path}
::tkutils::tkueditor::setStatus {path text}
::tkutils::tkueditor::refreshStatus {path}
::tkutils::tkueditor::setText {path text}
::tkutils::tkueditor::getText {path}
::tkutils::tkueditor::loadFile {path filename args}
::tkutils::tkueditor::saveFile {path args}
::tkutils::tkueditor::currentFile {path}
::tkutils::tkueditor::encoding {path args}
::tkutils::tkueditor::eol {path args}
::tkutils::tkueditor::find {path needle args}
::tkutils::tkueditor::findAll {path needle args}
::tkutils::tkueditor::findNext {path needle args}
::tkutils::tkueditor::replace {path needle repl args}
::tkutils::tkueditor::highlightAll {path needle args}
::tkutils::tkueditor::clearHighlight {path args}
::tkutils::tkueditor::gotoLine {path n}
```

```
::tkutils::tkueditor::cursor {path}
::tkutils::tkueditor::readonly {path args}
::tkutils::tkueditor::isModified {path}
::tkutils::tkueditor::menuWidget {path}
::tkutils::tkueditor::addMenuItem {path label command}
::tkutils::tkueditor::addMenuSeparator {path}
::tkutils::tkueditor::selectAll {path}
```

21.18 tkufiledialog

Quelle: `lib/tm/tkutils/tkufiledialog-0.2.tm`.

```
::tkutils::tkufiledialog::open {args}
::tkutils::tkufiledialog::save {args}
::tkutils::tkufiledialog::directory {args}
```

21.19 tkufilelist

Quelle: `lib/tm/tkutils/tkufilelist-0.2.tm`.

```
::tkutils::tkufilelist::widget {path args}
::tkutils::tkufilelist::setDir {path dir}
::tkutils::tkufilelist::refresh {path}
::tkutils::tkufilelist::setFilter {path glob}
::tkutils::tkufilelist::dir {path}
::tkutils::tkufilelist::selectedEntry {path}
::tkutils::tkufilelist::selectedEntries {path}
```

21.20 tkufiletree

Quelle: `lib/tm/tkutils/tkufiletree-0.2.tm`.

```
::tkutils::tkufiletree::widget {path args}
::tkutils::tkufiletree::setRoot {path dir}
::tkutils::tkufiletree::refresh {path}
::tkutils::tkufiletree::refreshDir {path dir}
::tkutils::tkufiletree::up {path}
::tkutils::tkufiletree::selectedPath {path}
::tkutils::tkufiletree::treeview {path}
::tkutils::tkufiletree::root {path}
::tkutils::tkufiletree::reveal {path target}
```

21.21 tkufilterbar

Quelle: `lib/tm/tkutils/tkufilterbar-0.2.tm`.

```
::tkutils::tkufilterbar::widget {path args}
::tkutils::tkufilterbar::setColumns {path cols}
::tkutils::tkufilterbar::columns {path}
::tkutils::tkufilterbar::getFilters {path}
::tkutils::tkufilterbar::getFilter {path col}
```

```
::tkutils::tkufilterbar::setFilter {path col text}
::tkutils::tkufilterbar::clear {path}
::tkutils::tkufilterbar::focusFirst {path}
```

21.22 tkuform

Quelle: `lib/tm/tkutils/tkuform-0.1.tm`.

```
::tkutils::tkuform::widget {path fieldspec args}
::tkutils::tkuform::get {path name}
::tkutils::tkuform::setField {path name value}
::tkutils::tkuform::fieldNames {path}
::tkutils::tkuform::values {path}
::tkutils::tkuform::setValues {path dict}
::tkutils::tkuform::widgetOf {path name}
```

21.23 tkufuzzy

Quelle: `lib/tm/tkutils/tkufuzzy-0.1.tm`.

```
::tkutils::tkufuzzy::widget {path args}
::tkutils::tkufuzzy::setItems {path items}
::tkutils::tkufuzzy::filter {path pattern}
::tkutils::tkufuzzy::getMatches {path}
::tkutils::tkufuzzy::getSelection {path}
```

21.24 tkugeometry

Quelle: `lib/tm/tkutils/tkugeometry-0.1.tm`.

```
::tkutils::tkugeometry::key {w}
::tkutils::tkugeometry::save {w app args}
::tkutils::tkugeometry::restore {w app args}
::tkutils::tkugeometry::track {w app args}
```

21.25 tkuhexedit

Quelle: `lib/tm/tkutils/tkuhexedit-0.2.tm`.

```
::tkutils::tkuhexedit::widget {path args}
::tkutils::tkuhexedit::setData {path data}
::tkutils::tkuhexedit::getData {path}
::tkutils::tkuhexedit::loadFile {path filename}
::tkutils::tkuhexedit::saveFile {path {filename ""}}
::tkutils::tkuhexedit::render {path {startOffset 0}}
::tkutils::tkuhexedit::gotoOffset {path offset}
::tkutils::tkuhexedit::findText {path pattern {start 0}}
::tkutils::tkuhexedit::findHex {path hex {start 0}}
::tkutils::tkuhexedit::patchHex {path offset hex}
```

21.26 tkuical

Quelle: `lib/tm/tkutils/tkuical-0.3.tm`.

```
::tkutils::tkuical::widget {path args}
::tkutils::tkuical::treeWidget {path}
::tkutils::tkuical::loadText {path ics}
::tkutils::tkuical::loadFile {path file}
::tkutils::tkuical::setComponents {path comps}
::tkutils::tkuical::events {path}
::tkutils::tkuical::count {path}
::tkutils::tkuical::addEvent {path {summary ""}}
::tkutils::tkuical::removeEvent {path index}
::tkutils::tkuical::setField {path index field value}
::tkutils::tkuical::setEventProperty {path index name value {params
    {}}}
::tkutils::tkuical::addEventProperty {path index name value {params
    {}}}
::tkutils::tkuical::removeEventProperty {path index name}
::tkutils::tkuical::toText {path}
::tkutils::tkuical::save {path file}
```

21.27 tkuicon

Quelle: `lib/tm/tkutils/tkuicon-0.1.tm`.

```
::tkutils::tkuicon::hassvg {}
::tkutils::tkuicon::load {filename args}
::tkutils::tkuicon::create {iconName size args}
::tkutils::tkuicon::rescale {imgName newHeight}
::tkutils::tkuicon::available {}
::tkutils::tkuicon::clearCache {}
::tkutils::tkuicon::loadToolbarSet {size args}
```

21.28 tkuimage

Quelle: `lib/tm/tkutils/tkuimage-0.2.tm`.

```
::tkutils::tkuimage::fit {imgW imgH boxW boxH {mode contain}}
::tkutils::tkuimage::load {file {dst ""}}
::tkutils::tkuimage::fromData {bytes {dst ""}}
::tkutils::tkuimage::scale {src w h {dst ""}}
::tkutils::tkuimage::thumbnail {src maxSize {dst ""}}
::tkutils::tkuimage::view {path args}
::tkutils::tkuimage::openFile {path file}
::tkutils::tkuimage::getImage {path}
::tkutils::tkuimage::zoomLevel {path}
::tkutils::tkuimage::zoomIn {path}
::tkutils::tkuimage::zoomOut {path}
::tkutils::tkuimage::zoom1 {path}
::tkutils::tkuimage::fitView {path}
```

21.29 tkuini

Quelle: `lib/tm/tkutils/tkuini-0.3.tm`.

```
::tkutils::tkuini::widget {path args}
::tkutils::tkuini::sectionWidget {path}
::tkutils::tkuini::treeWidget {path}
::tkutils::tkuini::loadText {path ini}
::tkutils::tkuini::loadFile {path file}
::tkutils::tkuini::setData {path d}
::tkutils::tkuini::selectSection {path section}
::tkutils::tkuini::data {path}
::tkutils::tkuini::sections {path}
::tkutils::tkuini::currentSection {path}
::tkutils::tkuini::count {path}
::tkutils::tkuini::setKey {path section key value}
::tkutils::tkuini::removeKey {path section key}
::tkutils::tkuini::addSection {path section}
::tkutils::tkuini::removeSection {path section}
::tkutils::tkuini::toText {path}
::tkutils::tkuini::save {path file}
```

21.30 tkujson

Quelle: `lib/tm/tkutils/tkujson-0.1.tm`.

```
::tkutils::tkujson::widget {path args}
::tkutils::tkujson::setJson {path jsonText}
::tkutils::tkujson::loadFile {path filename}
::tkutils::tkujson::getTree {path}
```

21.31 tkukeynav

Quelle: `lib/tm/tkutils/tkukeynav-0.1.tm`.

```
::tkutils::tkukeynav::next {w}
::tkutils::tkukeynav::prev {w}
::tkutils::tkukeynav::enable {w args}
::tkutils::tkukeynav::disable {w}
::tkutils::tkukeynav::form {container args}
```

21.32 tkulabeled

Quelle: `lib/tm/tkutils/tkulabeled-0.1.tm`.

```
::tkutils::tkulabeled::add {path type args}
::tkutils::tkulabeled::value {path args}
::tkutils::tkulabeled::control {path}
::tkutils::tkulabeled::labelwidget {path}
::tkutils::tkulabeled::configure {path args}
```

21.33 tkulauncher

Quelle: `lib/tm/tkutils/tkulauncher-0.4.tm`.

```
::tkutils::tkulauncher::widget {path args}
::tkutils::tkulauncher::available {entry}
::tkutils::tkulauncher::resolve {entry}
::tkutils::tkulauncher::load {file}
::tkutils::tkulauncher::reload {path}
::tkutils::tkulauncher::editConfig {path}
::tkutils::tkulauncher::openLocation {path}
::tkutils::tkulauncher::saveEntries {items file}
::tkutils::tkulauncher::showCalendar {path}
::tkutils::tkulauncher::showCalculator {path}
::tkutils::tkulauncher::editEntries {path}
::tkutils::tkulauncher::systemTitle {id}
::tkutils::tkulauncher::resolveSystem {id args}
::tkutils::tkulauncher::systemItems {args}
::tkutils::tkulauncher::systemIds {}
```

21.34 tkulayoutcanvas

Quelle: `lib/tm/tkutils/tkulayoutcanvas-0.1.tm`.

```
::tkutils::tkulayoutcanvas::widget {path args}
::tkutils::tkulayoutcanvas::configure {path args}
::tkutils::tkulayoutcanvas::cget {path opt}
::tkutils::tkulayoutcanvas::getCanvas {path}
::tkutils::tkulayoutcanvas::getBlocks {path}
::tkutils::tkulayoutcanvas::setBlocks {path blocks}
::tkutils::tkulayoutcanvas::select {path id}
::tkutils::tkulayoutcanvas::mergePreset {path preset args}
::tkutils::tkulayoutcanvas::redraw {path}
```

21.35 tkuldif

Quelle: `lib/tm/tkutils/tkuldif-0.3.tm`.

```
::tkutils::tkuldif::widget {path args}
::tkutils::tkuldif::treeWidget {path}
::tkutils::tkuldif::loadText {path ldif}
::tkutils::tkuldif::loadFile {path file}
::tkutils::tkuldif::setEntries {path entries}
::tkutils::tkuldif::entries {path}
::tkutils::tkuldif::count {path}
::tkutils::tkuldif::addEntry {path {dn "cn=New"}}
::tkutils::tkuldif::removeEntry {path index}
::tkutils::tkuldif::addAttr {path entryIndex attr value}
::tkutils::tkuldif::setAttr {path entryIndex pairIndex attr value}
::tkutils::tkuldif::removeAttr {path entryIndex pairIndex}
::tkutils::tkuldif::toText {path}
::tkutils::tkuldif::save {path file}
```

21.36 tkuload

Quelle: `lib/tm/tkutils/tkuload-0.3.tm`.

```
::tkuload::build {parent spec}
::tkuload::buildFromFile {parent path}
::tkuload::slot {ui name}
::tkuload::widgetByName {ui name}
::tkuload::setValue {ui name value}
::tkuload::getValue {ui name}
::tkuload::fill {ui data}
::tkuload::collect {ui {names ""}}
```

21.37 tkumarquee

Quelle: `lib/tm/tkutils/tkumarquee-0.1.tm`.

```
::tkutils::tkumarquee::enable {c args}
::tkutils::tkumarquee::disable {c}
::tkutils::tkumarquee::region {c}
::tkutils::tkumarquee::active {c}
```

21.38 tkumd

Quelle: `lib/tm/tkutils/tkumd-0.1.tm`.

```
::tkutils::tkumd::widget {path args}
::tkutils::tkumd::setMarkdown {path markdown}
::tkutils::tkumd::loadFile {path filename}
::tkutils::tkumd::getHeadings {path}
```

21.39 tkumdview

Quelle: `lib/tm/tkutils/tkumdview-0.1.tm`.

```
::tkutils::tkumdview::widget {path args}
::tkutils::tkumdview::tocWidget {path}
::tkutils::tkumdview::textWidget {path}
::tkutils::tkumdview::getMarkdown {path}
::tkutils::tkumdview::headings {path}
::tkutils::tkumdview::toHtml {path}
::tkutils::tkumdview::loadFile {path filename}
::tkutils::tkumdview::setMarkdown {path md}
```

21.40 tkumonthcanvas

Quelle: `lib/tm/tkutils/tkumonthcanvas-0.5.tm`.

21.41 tkunotes

Quelle: `lib/tm/tkutils/tkunotes-0.4.tm`.

```
::tkutils::tkunotes::widget {path args}
::tkutils::tkunotes::store {path}
::tkutils::tkunotes::count {path}
::tkutils::tkunotes::current {path}
::tkutils::tkunotes::treeWidget {path}
::tkutils::tkunotes::refresh {path}
::tkutils::tkunotes::select {path id}
::tkutils::tkunotes::addRoot {path title content {tags {}}}
::tkutils::tkunotes::addChild {path parentId title content {tags {}}}
::tkutils::tkunotes::delete {path id {cascade 1}}
::tkutils::tkunotes::search {path query}
::tkutils::tkunotes::move {path id newParentId}
::tkutils::tkunotes::load {path file}
::tkutils::tkunotes::save {path {file ""}}
::tkutils::tkunotes::expandAll {path}
::tkutils::tkunotes::collapseAll {path}
::tkutils::tkunotes::tags {path}
::tkutils::tkunotes::byTag {path tag}
::tkutils::tkunotes::addTag {path id tag}
::tkutils::tkunotes::removeTag {path id tag}
::tkutils::tkunotes::subtree {path id}
::tkutils::tkunotes::saveSubtree {path id file}
```

21.42 tkunumentry

Quelle: `lib/tm/tkutils/tkunumentry-0.1.tm`.

```
::tkutils::tkunumentry::widget {path args}
::tkutils::tkunumentry::getValue {path}
::tkutils::tkunumentry::setValue {path value}
::tkutils::tkunumentry::clear {path}
```

21.43 tkuopts

Quelle: `lib/tm/tkutils/tkuopts-0.1.tm`.

```
::tkutils::tkuopts::merge {mod defaults argv}
```

21.44 tkupath

Quelle: `lib/tm/tkutils/tkupath-0.3.tm`.

```
::tkutils::tkupath::widget {path args}
::tkutils::tkupath::setPath {path p}
::tkutils::tkupath::getPath {path}
```

21.45 tkupdfinspect

Quelle: `lib/tm/tkutils/tkupdfinspect-0.1.tm`.

```
::tkutils::tkupdfinspect::widget {path args}
::tkutils::tkupdfinspect::treeWidget {path}
::tkutils::tkupdfinspect::textWidget {path}
::tkutils::tkupdfinspect::currentFile {path}
::tkutils::tkupdfinspect::summary {path}
::tkutils::tkupdfinspect::loadFile {path filename}
```

21.46 tkupreview

Quelle: `lib/tm/tkutils/tkupreview-0.2.tm`.

```
::tkutils::tkupreview::widget {path}
::tkutils::tkupreview::text {path title content}
::tkutils::tkupreview::markdown {path title content}
::tkutils::tkupreview::message {path msg}
::tkutils::tkupreview::html {path title content}
::tkutils::tkupreview::json {path title content}
::tkutils::tkupreview::xml {path title content}
::tkutils::tkupreview::ini {path title content}
::tkutils::tkupreview::hex {path title data}
::tkutils::tkupreview::csv {path title content args}
::tkutils::tkupreview::photo {path title img}
::tkutils::tkupreview::sqlite {path title file}
::tkutils::tkupreview::image {path title file}
::tkutils::tkupreview::pdf {path title file}
::tkutils::tkupreview::clear {path}
::tkutils::tkupreview::kind {path}
```

21.47 tkurender

Quelle: `lib/tm/tkutils/tkurender-0.2.tm`.

```
::tkurender::isContainer {type}
::tkurender::optspecOf {type}
::tkurender::labelOf {type}
::tkurender::pkgFor {type}
::tkurender::pkgAvail {type}
::tkurender::newModel {}
::tkurender::addNode {type parent {internal 0}}
::tkurender::deleteNode {id}
::tkurender::nodeType {id}
::tkurender::nodeOpt {id o}
::tkurender::setOpt {id o v}
::tkurender::nodeGeom {id k}
::tkurender::setGeom {id k v}
::tkurender::nodeLayout {id}
::tkurender::setLayout {id v}
::tkurender::nodeName {id}
::tkurender::setName {id v}
::tkurender::setStretch {id which v}
::tkurender::renderChildren {mid tkparent pv}
::tkurender::serialize {}
::tkurender::deserialize {spec}
```

21.48 tkuscrolledframe

Quelle: `lib/tm/tkutils/tkuscrolledframe-0.1.tm`.

```
::tkutils::tkuscrolledframe::widget {path args}
::tkutils::tkuscrolledframe::content {path}
::tkutils::tkuscrolledframe::scrollableframe {path}
::tkutils::tkuscrolledframe::scrollarea {path}
::tkutils::tkuscrolledframe::configure {path args}
::tkutils::tkuscrolledframe::see {path w {corner nw}}
::tkutils::tkuscrolledframe::seerect {path x1 y1 x2 y2 {corner nw}}
::tkutils::tkuscrolledframe::xview {path args}
::tkutils::tkuscrolledframe::yview {path args}
::tkutils::tkuscrolledframe::autosize {path dimensions}
::tkutils::tkuscrolledframe::autofillx {path boolean}
::tkutils::tkuscrolledframe::autofilly {path boolean}
```

21.49 tkusearchbar

Quelle: `lib/tm/tkutils/tkusearchbar-0.1.tm`.

```
::tkutils::tkusearchbar::widget {path args}
::tkutils::tkusearchbar::getText {path}
::tkutils::tkusearchbar::setText {path text}
::tkutils::tkusearchbar::clear {path}
::tkutils::tkusearchbar::getFilter {path}
::tkutils::tkusearchbar::setFilter {path value}
::tkutils::tkusearchbar::setFilters {path filters}
::tkutils::tkusearchbar::focusSearch {path}
```

21.50 tkusqlite

Quelle: `lib/tm/tkutils/tkusqlite-0.1.tm`.

```
::tkutils::tkusqlite::widget {path args}
::tkutils::tkusqlite::openFile {path dbfile}
::tkutils::tkusqlite::tables {path}
::tkutils::tkusqlite::showTable {path table}
::tkutils::tkusqlite::getRows {path}
::tkutils::tkusqlite::closeDb {path}
```

21.51 tkustatus

Quelle: `lib/tm/tkutils/tkustatus-0.2.tm`.

```
::tkutils::tkustatus::widget {path args}
::tkutils::tkustatus::setText {path text}
::tkutils::tkustatus::getText {path}
::tkutils::tkustatus::addField {path id args}
::tkutils::tkustatus::setField {path id text}
::tkutils::tkustatus::fieldText {path id}
::tkutils::tkustatus::progress {path {value ""}}
::tkutils::tkustatus::flash {path text {ms 2000}}
```

21.52 tkustrings

Quelle: `lib/tm/tkutils/tkustrings-0.1.tm`.

```
::tkutils::tkustrings::widget {path args}
::tkutils::tkustrings::setData {path data args}
::tkutils::tkustrings::loadFile {path filename args}
::tkutils::tkustrings::getStrings {path}
```

21.53 tkusysinfo

Quelle: `lib/tm/tkutils/tkusysinfo-0.1.tm`.

```
::tkutils::tkusysinfo::widget {path args}
::tkutils::tkusysinfo::screen {win}
::tkutils::tkusysinfo::refresh {path}
::tkutils::tkusysinfo::data {path}
::tkutils::tkusysinfo::report {path args}
::tkutils::tkusysinfo::json {path}
::tkutils::tkusysinfo::save {path file args}
```

21.54 tkutab

Quelle: `lib/tm/tkutils/tkutab-0.2.tm`.

```
::tkutils::tkutab::widget {path args}
::tkutils::tkutab::add {path label}
::tkutils::tkutab::close {path f}
::tkutils::tkutab::current {path}
::tkutils::tkutab::setLabel {path f label}
::tkutils::tkutab::tabs {path}
::tkutils::tkutab::count {path}
```

21.55 tkutablelist

Quelle: `lib/tm/tkutils/tkutablelist-0.5.tm`.

```
::tkutils::tkutablelist::widget {path args}
::tkutils::tkutablelist::footerWidget {path}
::tkutils::tkutablelist::footerSet {path values}
::tkutils::tkutablelist::footerSum {path args}
::tkutils::tkutablelist::tableWidget {path}
::tkutils::tkutablelist::insert {path row {id ""}}
::tkutils::tkutablelist::setRows {path rows args}
::tkutils::tkutablelist::rowId {path index}
::tkutils::tkutablelist::findId {path id}
::tkutils::tkutablelist::selectedIds {path}
::tkutils::tkutablelist::selectIds {path ids}
::tkutils::tkutablelist::rows {path}
::tkutils::tkutablelist::clear {path}
::tkutils::tkutablelist::size {path}
::tkutils::tkutablelist::getRow {path index}
::tkutils::tkutablelist::setRow {path index row}
::tkutils::tkutablelist::deleteRow {path index}
::tkutils::tkutablelist::cellText {path row col}
::tkutils::tkutablelist::setCell {path row col value}
::tkutils::tkutablelist::setColumns {path titles}
::tkutils::tkutablelist::columns {path}
::tkutils::tkutablelist::configureColumn {path col args}
::tkutils::tkutablelist::configureRow {path row args}
::tkutils::tkutablelist::configureCell {path cell args}
::tkutils::tkutablelist::insertChild {path parent values {index end}}
```

```
::tkutils::tkutablelist::expand {path {row ""}}
::tkutils::tkutablelist::collapse {path {row ""}}
::tkutils::tkutablelist::selection {path}
::tkutils::tkutablelist::selectedRows {path}
::tkutils::tkutablelist::selectRows {path indices}
::tkutils::tkutablelist::sortBy {path col {order -increasing}}
::tkutils::tkutablelist::loadCsv {path csv args}
::tkutils::tkutablelist::fromNotes {path store args}
::tkutils::tkutablelist::editEndCommand {path cmd}
::tkutils::tkutablelist::selectCommand {path args}
::tkutils::tkutablelist::doubleCommand {path args}
::tkutils::tkutablelist::toCsv {path args}
::tkutils::tkutablelist::saveCsv {path file args}
```

21.56 tkutags

Quelle: `lib/tm/tkutils/tkutags-0.1.tm`.

```
::tkutils::tkutags::widget {path args}
::tkutils::tkutags::getTags {path}
::tkutils::tkutags::setTags {path tags}
::tkutils::tkutags::addTag {path tag}
::tkutils::tkutags::removeTag {path tag}
::tkutils::tkutags::clear {path}
```

21.57 tkutical

Quelle: `lib/tm/tkutils/tkutical-0.4.tm`.

```
::tkutils::tkutical::widget {path args}
::tkutils::tkutical::refresh {path}
::tkutils::tkutical::setView {path view}
::tkutils::tkutical::getView {path}
::tkutils::tkutical::setDate {path iso}
::tkutils::tkutical::getDate {path}
::tkutils::tkutical::setMonth {path year month}
::tkutils::tkutical::getMonth {path}
::tkutils::tkutical::next {path}
::tkutils::tkutical::prev {path}
::tkutils::tkutical::today {path}
::tkutils::tkutical::selectMode {path mode}
::tkutils::tkutical::getSelection {path}
::tkutils::tkutical::setSelection {path dates}
::tkutils::tkutical::clearSelection {path}
::tkutils::tkutical::canvasWidget {path}
::tkutils::tkutical::setEvents {path events}
::tkutils::tkutical::getEvents {path}
```

21.58 tkutimeentry

Quelle: `lib/tm/tkutils/tkutimeentry-0.1.tm`.

```
::tkutils::tkutimeentry::widget {path args}
::tkutils::tkutimeentry::getTime {path}
::tkutils::tkutimeentry::setTime {path str}
```

21.59 tkutlclip

Quelle: `lib/tm/tkutils/tkutlclip-0.3.tm`.

```
::tkutils::tkutlclip::asText {tbl rows args}
::tkutils::tkutlclip::copySelection {tbl args}
::tkutils::tkutlclip::copyAll {tbl args}
::tkutils::tkutlclip::installBindings {tbl args}
```

21.60 tkutldrag

Quelle: `lib/tm/tkutils/tkutldrag-0.1.tm`.

```
::tkutils::tkutldrag::enable {tbl args}
::tkutils::tkutldrag::disable {tbl}
::tkutils::tkutldrag::order {tbl {parent root}}
```

21.61 tkutlfilter

Quelle: `lib/tm/tkutils/tkutlfilter-0.1.tm`.

```
::tkutils::tkutlfilter::filter {tbl query args}
::tkutils::tkutlfilter::clear {tbl}
::tkutils::tkutlfilter::query {tbl}
::tkutils::tkutlfilter::later {tbl ms query args}
```

21.62 tkutlfind

Quelle: `lib/tm/tkutils/tkutlfind-0.3.tm`.

```
::tkutils::tkutlfind::find {tbl query args}
::tkutils::tkutlfind::matches {tbl}
::tkutils::tkutlfind::next {tbl}
::tkutils::tkutlfind::prev {tbl}
::tkutils::tkutlfind::clear {tbl}
```

21.63 tkutlfmt

Quelle: `lib/tm/tkutils/tkutlfmt-0.3.tm`.

```
::tkutils::tkutlfmt::column {tbl col type args}
::tkutils::tkutlfmt::columns {tbl spec}
```

21.64 tkutlfooter

Quelle: `lib/tm/tkutils/tkutlfooter-0.4.tm`.

```
::tkutils::tkutlfooter::attach {tbl foot args}
::tkutils::tkutlfooter::detach {tbl foot}
::tkutils::tkutlfooter::setvals {foot values}
::tkutils::tkutlfooter::update {tbl foot}
::tkutils::tkutlfooter::autosum {tbl foot args}
```

21.65 tkutlgroup

Quelle: `lib/tm/tkutils/tkutlgroup-0.1.tm`.

```
::tkutils::tkutlgroup::group {tbl rows keyCol args}
::tkutils::tkutlgroup::ungroup {tbl}
```

21.66 tkutlsort

Quelle: `lib/tm/tkutils/tkutlsort-0.3.tm`.

```
::tkutils::tkutlsort::column {tbl col type args}
::tkutils::tkutlsort::columns {tbl spec args}
```

21.67 tkutlstate

Quelle: `lib/tm/tkutils/tkutlstate-0.4.tm`.

```
::tkutils::tkutlstate::save {tbl}
::tkutils::tkutlstate::restore {tbl state}
::tkutils::tkutlstate::saveToFile {tbl path}
::tkutils::tkutlstate::restoreFromFile {tbl path}
::tkutils::tkutlstate::saveVia {tbl key setPrefix}
::tkutils::tkutlstate::restoreVia {tbl key getPrefix}
::tkutils::tkutlstate::autosave {tbl key setPrefix args}
::tkutils::tkutlstate::cancelAutosave {tbl}
::tkutils::tkutlstate::lastError {tbl}
```

21.68 tkutltree

Quelle: `lib/tm/tkutils/tkutltree-0.3.tm`.

```
::tkutils::tkutltree::fromData {tbl data args}
::tkutils::tkutltree::toData {tbl args}
::tkutils::tkutltree::clear {tbl args}
::tkutils::tkutltree::lazy {tbl loadCmd args}
::tkutils::tkutltree::reveal {tbl index}
::tkutils::tkutltree::findId {tbl id}
```

21.69 tkutodo

Quelle: `lib/tm/tkutils/tkutodo-0.2.tm`.

```
::tkutils::tkutodo::widget {path args}
::tkutils::tkutodo::treeWidget {path}
::tkutils::tkutodo::setTodos {path comps}
::tkutils::tkutodo::loadText {path ics}
::tkutils::tkutodo::todos {path}
::tkutils::tkutodo::count {path}
::tkutils::tkutodo::toggle {path {index ""}}
```

21.70 tkutoolbar

Quelle: `lib/tm/tkutils/tkutoolbar-0.2.tm`.

```
::tkutils::tkutoolbar::widget {path args}
::tkutils::tkutoolbar::addButton {path id label command args}
::tkutils::tkutoolbar::addToggle {path id label varName args}
::tkutils::tkutoolbar::addDropdown {path id label args}
::tkutils::tkutoolbar::addSeparator {path}
::tkutils::tkutoolbar::addWidget {path id w {side left}}
::tkutils::tkutoolbar::setEnabled {path id enabled}
::tkutils::tkutoolbar::buttonWidget {path id}
::tkutils::tkutoolbar::items {path}
::tkutils::tkutoolbar::configureButton {path id args}
::tkutils::tkutoolbar::setCallback {path id command}
::tkutils::tkutoolbar::setDisplayMode {path mode}
::tkutils::tkutoolbar::getDisplayMode {path}
::tkutils::tkutoolbar::addAction {path name {id ""}}
```

21.71 tkutree

Quelle: `lib/tm/tkutils/tkutree-0.2.tm`.

```
::tkutils::tkutree::widget {path args}
::tkutils::tkutree::insert {path parent text args}
::tkutils::tkutree::loadTree {path nodes {parent ""}}
::tkutils::tkutree::clear {path}
::tkutils::tkutree::selection {path}
::tkutils::tkutree::itemText {path id}
::tkutils::tkutree::itemValues {path id}
::tkutils::tkutree::children {path {id ""}}
::tkutils::tkutree::treeview {path}
```

21.72 tkuunits

Quelle: `lib/tm/tkutils/tkuunits-0.1.tm`.

```
::tkutils::tkuunits::units {}
::tkutils::tkuunits::toPixels {value unit {w .}}
::tkutils::tkuunits::fromPixels {px {w .}}
::tkutils::tkuunits::convert {value from to {w .}}
```

21.73 tkuvalidate

Quelle: `lib/tm/tkutils/tkuvalidate-0.1.tm`.

```
::tkutils::tkuvalidate::attach {w validator args}
::tkutils::tkuvalidate::check {w}
::tkutils::tkuvalidate::valid {w}
::tkutils::tkuvalidate::clear {w}
::tkutils::tkuvalidate::allValid {widgets}
```

21.74 tkuvcard

Quelle: `lib/tm/tkutils/tkuvcard-0.3.tm`.

```
::tkutils::tkuvcard::widget {path args}
::tkutils::tkuvcard::treeWidget {path}
::tkutils::tkuvcard::loadText {path vcf}
::tkutils::tkuvcard::loadFile {path file}
::tkutils::tkuvcard::setCards {path cards}
::tkutils::tkuvcard::cards {path}
::tkutils::tkuvcard::count {path}
::tkutils::tkuvcard::addCard {path {fn "New Contact"}}
::tkutils::tkuvcard::removeCard {path index}
::tkutils::tkuvcard::addProperty {path cardIndex name value {type ""}}
::tkutils::tkuvcard::removeProperty {path cardIndex propIndex}
::tkutils::tkuvcard::setProperty {path cardIndex propIndex name value
    {type ""}}
::tkutils::tkuvcard::toText {path}
::tkutils::tkuvcard::save {path file}
```

21.75 tkuwheel

Quelle: `lib/tm/tkutils/tkuwheel-0.2.tm`.

```
::tkutils::tkuwheel::redirect {target w args}
::tkutils::tkuwheel::unbind {w args}
```

21.76 tkuwinico

Quelle: `lib/tm/tkutils/tkuwinico-0.1.tm`.

```
::tkutils::tkuwinico::defaultSizes {}
::tkutils::tkuwinico::fromSvg {svgFile outFile args}
::tkutils::tkuwinico::fromPhoto {photo outFile args}
::tkutils::tkuwinico::fromPhotos {photos outFile}
```

21.77 tkuxml

Quelle: `lib/tm/tkutils/tkuxml-0.1.tm`.

```
::tkutils::tkuxml::widget {path args}
::tkutils::tkuxml::setXml {path xml}
::tkutils::tkuxml::loadFile {path filename}
::tkutils::tkuxml::getRoot {path}
```

21.78 tkuzip

Quelle: `lib/tm/tkutils/tkuzip-0.1.tm`.

```
::tkutils::tkuzip::widget {path args}
::tkutils::tkuzip::openFile {path zipfile}
::tkutils::tkuzip::getEntries {path}
::tkutils::tkuzip::selectedMember {path}
```

22 Praxis: Tk-Integration, Maße und Fensterzustand

22.1 Tk zuerst laden

`tkutils` ergänzt Tk; es ersetzt die Tk-Laufzeit nicht. Für grafische Beispiele wird ein Display benötigt. Unter Linux kann ein Testlauf mit Xvfb sinnvoll sein, während der reine `tclsh` ohne Display keine Fenster erstellt.

```
package require Tk
package require tkutils::tkuunits

label .label -text "Maßeinheiten"
pack .label -padx 12 -pady 12
puts [::tkutils::tkuunits::toPixels 10 mm .]
```

`tkuunits-0.1.tm` exportiert `units`, `toPixels`, `fromPixels` und `convert`. Die Umrechnung kann das Fensterargument `w` verwenden, weil die physische Bildschirmauflösung relevant ist. Ein Pixelwert ist deshalb nicht auf allen Displays identisch.

22.2 Einheiten vergleichen

```
package require Tk
package require tkutils::tkuunits

set pixels [::tkutils::tkuunits::toPixels 25.4 mm .]
set inches [::tkutils::tkuunits::convert 25.4 mm inch .]
puts "pixels=$pixels inches=$inches"
```

Die Rückgabewerte sollten nicht als feste Pixelkonstanten in Dokumentation oder Tests eingetragen werden. Ein guter Test vergleicht die Konsistenz zweier Umrechnungen unter derselben Displaykonfiguration.

22.3 Fenstergeometrie

`tkugeometry-0.1.tm` exportiert `save`, `restore`, `track` und `key`. Die ersten drei Befehle erwarten unter anderem ein Fenster und eine Anwendungskennung. Die interne Implementierung enthält außerdem Logik zum Anpassen an verfügbare Bildschirmflächen.

```
package require Tk
package require tkutils::tkugeometry

set appId demo-window
wm title . "Fensterzustand"
# "." ergibt den Schlüssel main, ohne zu speichern.
set geometryKey [::tkutils::tkugeometry::key .]
puts $geometryKey
```

Das Beispiel führt keine persistenten Schreibzugriffe aus. Für `save`, `restore` und `track` sind vor dem Einsatz insbesondere Ablageort, Optionen und Verhalten bei mehreren Monitoren zu prüfen.

22.4 Fehlersuche bei optionalen Widgets

Ein Widget-Modul kann zusätzliche Pakete benötigen, etwa Tablelist, Img, tDOM oder scrollutil. Testen Sie solche Module einzeln, statt allein aus einem erfolgreichen `package require tkutils` auf die Verfügbarkeit sämtlicher Zusatzfunktionen zu schließen.

```
package require Tk
set code [catch {package require tkutils::tkutablelist} result options]
if {$code} {
    puts stderr "Tablelist integration unavailable: $result"
} else {
    puts "tkutablelist $result"
}
```

Ein fehlendes optionales Paket ist kein Fehler der Tk-Laufzeit. Die Anwendung sollte dann entweder eine verständliche Diagnose ausgeben oder die betroffene Funktion deaktivieren.

Getestet mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux und Xvfb: alle Beispiele dieses Anhangs. Bei `tk scaling` 1,39 gibt das erste Beispiel `39.38461538461539` aus, das zweite `pixels=100.03692307692307 inches=1.0`, das dritte `main`.

Nicht getestet: verschiedene Fenstermanager.

23 Testumgebung

23.1 Einführung

Dieser Anhang nennt die Umgebung, mit der die Angaben des Buchs geprüft sind, das Verhalten älterer Fassungen und die Punkte, die noch nicht geprüft sind.

23.2 Umgebung

Was	Stand
System	Linux, GUI unter Xvfb ohne Fenstermanager, Bildschirm 1280×1024
Tcl/Tk 8.6	8.6.14 mit tDOM 0.9.3, sqlite3 3.45.1, tklib 0.8 (Tablelist 6.20, scrollutil 1.17), Img 1.4.16; ohne tksvg

Was	Stand
Tcl/Tk 9	9.0.4 mit tDOM 0.9.7, sqlite3 3.53.0, Tablelist 7.11, scrollutil 2.10, Img 2.1.1
tkutils	0.49.0
tclutils	0.69.0
ctrlutils	0.4
tical	1.1, über TICAL_DIR oder Geschwisterordner

Die Ergebnisse der Testsuite stehen in Installation, Abschnitt *Testlauf*. Für den Lauf liegen tclutils, tkutils und tical als Geschwisterordner nebeneinander; `TCLUTILS_TM`, `TKUTILS_TM` und `TICAL_DIR` zeigen auf diese Bäume.

Prüfskripte: Jedes Beispiel mit einer Ausgabe `;# -> …` läuft unter `xvfb-run -a tclsh8.6` und `xvfb-run -a tclsh9.0`, die Module aus `tcl::tm::path add <tclutils>/lib/tm <tkutils>/lib/tm`.

`tkulauncher-9.4` (Mausrad im Scroll-Modus) war bis `tkulauncher` 0.2 unbeständig: Die Knöpfe bekamen die Mausrad-Bindung erst über ein späteres Ereignis, nach `reload` gar nicht. Seit 0.3 bindet der Launcher die Knöpfe beim Anlegen; `tkulauncher-9.5` prüft den Fall nach `reload`.

23.3 Ältere Fassungen

23.3.1 Testsuite 0.48.0

Gleiche Umgebung, tclutils 0.68.0: 78 Testdateien, 843 Tests; unter 8.6.14 831 bestanden und 12 übersprungen, unter 9.0.4 840 bestanden und 3 übersprungen, 0 Fehler. 0.49.0 bringt vier neue Testdateien (`tkutlfilter`, `tkutlgroup`, `tkutldrag`, `tkusysinfo`) und 64 Tests mehr.

23.3.2 Testsuite 0.46.0

Tcl/Tk 8.6.14, Linux, Xvfb, tclutils 0.65.0, tkutils 0.46.0 (Commit `401ebde`), tklib 0.8, tDOM 0.9.3, sqlite3 3.45.1, ohne tksvg: 78 Testdateien, 837 Tests, 811 bestanden, 25 übersprungen, 1 Fehler.

- Der Fehler war `tkulauncher-9.4` (Mausrad im Scroll-Modus). Er ließ sich danach nicht reproduzieren, auch nicht im unveränderten Baum (837 Tests, 0 Fehler unter 8.6.14 und 9.0.4); `tkulauncher.test` allein bestand mit 99 von 99 Tests. Die Ursache ist in `tkulauncher` 0.3 behoben (Abschnitt *Umgebung*).
- Übersprungen waren u. a. alle 13 Tests von `tkumonthcanvas.test` und 7 SVG-Tests von `tkuwinico.test`.
- In 0.47.0 kamen `tkufiledialog-1.4a` (Doppelklick), `tkutltools-2.1/-2.2` (beide Ladereihenfolgen mit `tablelist_tile`) und `tkuform-3.7` (Optionsfehler) hinzu; die ersten drei schlagen mit dem Code von 0.46.0 fehl.

Ohne Display. In 0.46.0 brechen `stack.test`, `tkumdview.test`, `tkupdfinspect.test` und `tkuwinico.test` ohne `DISPLAY` beim Laden ab, schreiben keine Zusammenfassung, und der Lauf endet mit Exit-Code 1. Abhilfe: die Suite immer unter Xvfb oder mit Display starten.

tical. `tkumonthcanvas.test` aus 0.46.0 sucht ohne `TICAL_DIR` nur `tical*/tical` und

überspringt bei flachem Checkout alle 13 Tests. Abhilfe: `TICAL_DIR=/pfad/zu/tical` setzen; dann bestehen 13 von 13.

23.3.3 Zwei Tablelist-Varianten bis 0.46.0

`tkutablelist` lädt zuerst `tablelist_tile` und nur, wenn das fehlt, `Tablelist`. Die Helfer `tkutl*` laden dagegen `tablelist`. Mit Tablelist 6.20 aus tklib 0.8 scheitert nach `package require tkutils` deshalb `package require tkutils::tkutltools` (ebenso `tkutlsort` allein) mit

```
can't set "usingTile": it is not supported to use both Tablelist and
    Tablelist_tile in the same application
```

Abhilfe: `package require tablelist` (oder `tkutils::tkutltools`) **vor** dem Umbrella. Dann nimmt `tkutablelist` die nicht-ttk-Variante, und beide Familien laufen zusammen.

23.3.4 tkufiledialog 0.1 (tkutils 0.46.0)

Dialog jeweils programmatisch bedient (Tcl/Tk 8.6.14, Linux, Xvfb):

Bedienung	**Ergebnis**
Name a.txt eintippen, Return	voller Pfad zu a.txt
Escape	""
save -initialfile bericht -defaultextension .txt, OK	.../bericht.txt
directory, OK ohne Auswahl	der Startordner
Doppelklick-Handler auf Datei a.txt	""

Der letzte Fall ist ein Fehler im Code von 0.46.0: Der Doppelklick auf eine Datei trägt den Namen ein und schließt mit *OK*, setzt aber den Rückgabewert nicht. Abhilfe: Datei markieren und *OK* oder Return benutzen. In 0.47.0 (`tkufiledialog` 0.2) behoben.

23.4 Noch nicht geprüft

Kapitel	**offen**
alle	unter Windows und macOS nur die Testsuite
Dialoge	Darstellung, Fokus und Platzierung auf einem echten Bildschirm
Datei-Widgets	Navigation eines Dateimanagers mit Windows-Pfaden
Geometrie und Units	Konfig-Ordner unter Windows und macOS; mehrere Monitore; Zoom-Zustand (Xvfb ohne Fenstermanager kennt keinen)
Aktionen und Layout	Start von app-Einträgen unter Windows (nur im Code nachgelesen)
Designer und load	die App Dateiblick (Code nicht in tclutils, tkutils oder ctrlutils)

Kapitel	offen
tkdevtools	Anzeige der Kodierungsprofile unter Tcl 9; Schrift-Aliase unter Windows

Ein Ergebnis zu einem dieser Punkte gehört in das jeweilige Kapitel und wird hier gestrichen.

24 Lizenz

```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Namensnennung - Weitergabe unter gleichen
Bedingungen 4.0 International (CC BY-SA 4.0)
Codebeispiele des Autors: MIT-Lizenz (unten)

---

Text

Dieses Buch ist unter der Creative-Commons-Lizenz
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0) lizenziert.

Sie dürfen das Material in jedwedem Format oder Medium vervielfältigen
und weiterverbreiten sowie es remixen, verändern und darauf aufbauen,
auch zu kommerziellen Zwecken, unter den folgenden Bedingungen:

* Namensnennung: Sie müssen angemessene Urheber- und Rechteangaben
  machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen
  vorgenommen wurden.

* Weitergabe unter gleichen Bedingungen: Wenn Sie das Material
  remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen
  Sie Ihre Beiträge nur unter derselben Lizenz wie das Original
  verbreiten.

Verbindlich ist der vollständige Lizenztext:
https://creativecommons.org/licenses/by-sa/4.0/legalcode.de
Englische Originalfassung:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

---

Codebeispiele

Die Codebeispiele des Autors (in den Codeblöcken des Buchs und als
Skripte daneben) stehen zusätzlich unter der MIT-Lizenz. Übernommene
Codebeispiele und zitierte Texte aus anderen Projekten (Tcl/Tk,
pdf4tcl und andere) behalten die Lizenz ihres Originals; die Quelle ist
```

```
jeweils angegeben.

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Stichwortverzeichnis

B

Button-Id 65

E

exportselection 21, 65

F

Fenster-Override 65

M

msgcat 14

N

Namensraum 65

P

Package shadow 66
Pairing 66
placeholder 55, 65
Provider 17, 65

S

slot 55, 65

T

tablelist 20
tablelist_tile 66
tclutils 8
TCLUTILS_TM 12
tical 9
tkdevtools 60
tkuaction 34
tkuballoon 39
tkubase64 52
tkubind 40
tkucal 46
tkucalc 49
tkucalendar 45
tkucanvaspng 59
tkucontextmenu 39
tkucsv 49
tkudateentry 43
tkudavaccount 53
tkudavbrowser 53
tkudhash 59
tkudialog 13
tkudiff 51
tkueditor 53

tkufiledialog 19
tkufiletree 17
tkufilterbar 30, 41
tkuform 42
tkufuzzy 52
tkugeometry 32
tkuhexedit 51
tkuical 47
tkuicon 58
tkuimage 58
tkuini 50
tkujson 50
tkukeynav 40
tkulabeled 43
tkulauncher 36
tkulayoutcanvas 57
tkuldif 48
tkuload 8, 55
tkumarquee 41
tkumd 51
tkumdview 51
tkumonthcanvas 46
tkunotes 48
tkunumentry 44
tkuopts 67
tkupath 18
tkupdfinspect 52
tkurender 8, 55
tkuscrolledframe 41
tkusearchbar 41
tkusqlite 51
tkustatus 38
tkustrings 52
tkusysinfo 54
tkutab 39
tkutablelist 8
tkutags 44
tkutical 46
tkutils 8
tkutimeentry 44
tkutldrag 31
tkutlfilter 29
tkutlgroup 30
tkutltools 23
tkutodo 47
tkutoolbar 38
tkutree 39

tkuunits 33
tkuvalidate 43
tkuvcard 48
tkuwheel 41
tkuwinico 59
tkuxml 50
tkuzip 52
tutestrun 12, 65

U

Umbrella 8, 65